



Empowering and
Building Trust

Web Application Security Assessment Report

Teamwork

30/01/2026



Document and Record Control

Version Control

Document Control ID	GRM-WEB-APPSEC- TEAMWORK-JANUARY-2026
Issued Date:	30 th January 2026
Effective Date:	30 th January 2026
Owner:	Mr. Babu Gopinathan

Revision Table

Date	Version	Brief Description	Author
30 th January 2026	Draft	Teamwork Web Application Security Assessment Draft Report	Ms. Sampada Jadhav

Release Authorization

Task	Author	Title
Prepared by:	Sampada Jadhav	Cyber Security Consultant
Reviewed by:	Ms. Bhavani P	Assistant Manager-Cyber Security

Approval Authorization

Name	Title	Signature	Date
Mr. Babu Gopinathan	Director-Cyber Security		30 th January 2026

Document Distribution List

Name	Organization
CEO	Teamwork

Important note: This document is intended solely for the use of the individual or entity to whom it is transmitted, and others authorized to receive it. It may contain confidential or legally privileged information. If you are not the intended recipient, you are hereby notified that any disclosure, copying, distribution, or taking any action in reliance on the contents of this document is strictly prohibited and may be unlawful. If you have received this document in error, please notify us immediately.



About Us

GRM Technologies LLC FZ

GRM Technologies LLC FZ ("GRM") is a UAE-registered company, established in **2022**, with strong operational roots in **India since 2013**. Headquartered in **Chennai, India**, GRM is a **CERT-In empanelled** Cyber Security and Information Security Audit, Advisory, and Consulting firm, founded with a clear purpose: to protect and preserve the confidentiality, integrity, and availability of our clients' systems and data.

Over the years, GRM has evolved into a trusted cybersecurity partner for organizations across multiple industry verticals, including **Public Sector, IT & ITES, Oil & Gas, Banking and Financial Services, Manufacturing, Logistics, Consulting, Outsourcing, Retail, E-commerce, Hospitality**, and more. Our global delivery capability enables us to support clients across **India, the United Arab Emirates, Saudi Arabia, the United States, the Caribbean Islands, Canada, and Europe**. To date, our team has successfully delivered **1000+ cybersecurity and information security engagements**.

GRM's professionals bring deep expertise across cybersecurity, governance, and compliance, delivering end-to-end services that include **UAE IA, DESC ISR, CERT-In guideline alignment, ISO 27001, ISO 27701, ISO 22301, SOC 1, SOC 2, GDPR, PCI DSS, HIPAA, SOX consulting, Vulnerability Assessment and Penetration Testing (VAPT), Application Security Assessments, Digital Forensics, Incident Management, Regulatory Compliance, and Privacy Framework implementations**. We also support organizations with **UAE regulatory and sectoral compliance**, including alignment to **UAE Information Assurance and relevant local regulatory expectations** (as applicable), ensuring practical, audit-ready controls across policy, process, and technology layers.

In addition, GRM has specialized capability in **Public Key Infrastructure (PKI) and Certificate Lifecycle Management**, supporting organizations with **PKI strategy, architecture, implementation, operations, and compliance**. Our services include **PKI governance (CP/CPS development), CA hierarchy design (offline root / issuing CA), certificate policy enforcement, key management and HSM integrations, certificate automation, renewal and revocation processes, CRL/OCSP design, and integration with enterprise systems and applications**. This expertise enables our clients to strengthen identity, encryption, and trust services across critical business platforms and regulated environments.

Our leadership team and core security experts are highly experienced in engaging with **board members and senior stakeholders**, translating complex technical risks into **clear, business-relevant insights** that support informed decision-making across all levels of the organization. We take pride in building long-term relationships with our clients by working closely with client management and execution teams—guided by **trust, integrity, and accountability**—to recommend and implement the right security controls at the right time, helping protect their data, environments, and business processes against evolving internal and external threats.



GRM Contact Details

Name	Babu Gopinathan
Title	Director - Cyber Security
Company	GRM Technologies LLC FZ
Office address	Meydan FreeZone, Business Center 1 M Floor, The Meydan Hotel, Nad AL Sheba, Dubai - UAE
Mobile	+971-527982852, +971-26320588
Email	babug@grmtechnologies.com

Required Details

S. No	Particulars	Details to be furnished
1	Name of the Information System Auditor/Consultant/Company	GRM Technologies LLC FZ
2	Location of Registered Office	Meydan FreeZone, Business Center 1 M Floor, The Meydan Hotel, Nad AL Sheba, Dubai - UAE
3	Mailing Address	Meydan FreeZone, Business Center 1 M Floor, The Meydan Hotel, Nad AL Sheba, Dubai - UAE
4	Official Contact Numbers	+971-527982852, +971-26320588
5	E-mail address of the contact person	babug@grmtechnologies.com
6	Description of business and business background	• Cyber Security Consulting Services • Advisory Services • Managed Security Services • Privacy Services • Forensics Services • Training Services



Table of Contents

1. Introduction	7
2. Executive Summary	13
3. Detailed Technical Report.....	16
3.1.1 Reflected Cross-Site Scripting (XSS).....	17
3.1.2 HTML Injection.....	20
3.1.3 SMTP Injection.....	22
3.1.4 Unrestricted File Upload.....	26
3.1.5 PII Data Exposure.....	30
3.1.6 Missing Rate Limiting.....	34
3.1.7 Sensitive File and Information Disclosure	37
3.1.8 Improper session management.....	43
3.1.9 Business Logic Flaw	49
3.1.10 Password Disclosure in Response.....	53
3.1.11 Weak Password Policy	55
3.1.12 User Enumeration	57
3.1.13 Session Fixation.....	60
3.1.14 Improper OTP Configuration.....	63
3.1.15 Missing Autocomplete.....	65
3.1.16 Server Name and Version Disclosure.....	67
3.1.17 Improper Error Handling.....	70
3.1.18 Clear Text Submission of Credentials	72
3.1.19 Missing Security Headers	74
3.1.20 Clickjacking Attack.....	76
3.1.21 Improper Input Validation	78
3.1.22 Vulnerable and Outdated Component.....	81
3.1.23 Method Interchange	89
3.1.24 Missing CAPTCHA	91
3.1.25 Sensitive Information Disclosure via URL	93
3.1.26 Open Redirection	95



3.1.27	JSON Web Key set Disclosed.....	97
3.1.28	Missing Cookie Attribute.....	99
3.1.29	Concurrent Login Session.....	101
3.1.30	Breach & Lucky13 Attack.....	103

DRAFT REPORT



1. Introduction

1.1 Objective

The Cyber Security team of “GRM Technologies” was engaged to understand Teamwork application security posture by conducting security testing of the Teamwork Web Application. The objective of this security assessment was to identify vulnerabilities/weaknesses that Teamwork Web Application may possess and to provide recommendations for risk mitigation. Based on the outcome of the security assessment, the Teamwork team would develop an action plan to minimize risk in the Web application.

1.2 Scope

The security assessment was conducted from 27th January 2025 to 29th January 2025 on the Teamwork Web Application.

The following activities were completed as part of the testing:

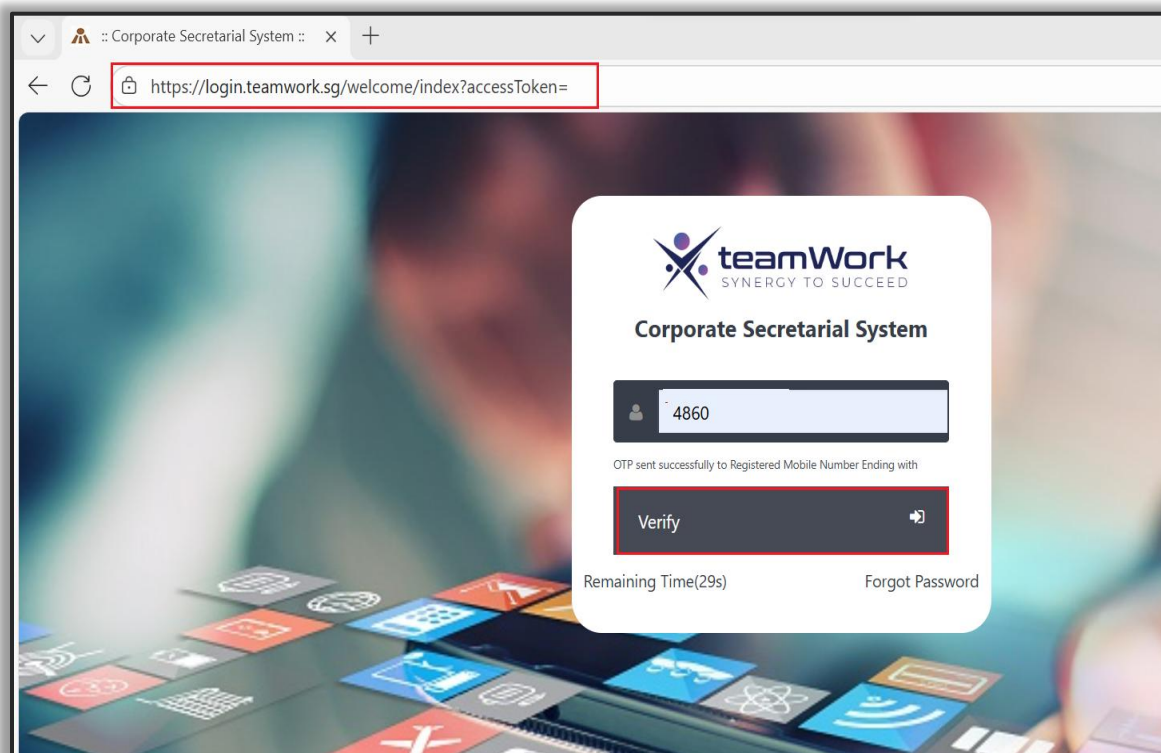
- Grey Box security testing of Teamwork Web Application.
- The security team was provided with the Teamwork URLs and Credentials to a UAT environment.

Details of the application tested are mentioned in the table below.

Web Application Security Assessment					
Application URL	Application Name	Hash Value	Application Version	Application Released Timestamps	Type
https://login.teamwork.sg/	Teamwork	NA	NA	NA	Grey box Assessment

1.3 Exceptions and Limitations

During the assessment, the OTP functionality did not operate as expected. After entering the OTP and clicking the Verify or Login button, the user was not able to log in.





1.4 Manpower Involved

S. No	Name of Employee	E-mail ID	Masters in Cyber Security	Certifications (Yes/No)												Experience in Cyber Security Audit (No. of Years)
				CISA	CISM	CRISC	CGEIT	DISA	OSCP	CEH	ISO 7001	PCIDSS	ISO 2301	CompTIA	CCNA	
1	Babu Gopinathan	babug@grmtechnologies.com		Y	Y	Y		Y	Y	Y	Y	Y	Y	Y	20	
2	Bhavani P	bhavanip@grmtechnologies.com								Y	Y				4	
3	Sampada Jadhav	sampada@grmtechnologies.com								Y					2	



1.5 Approach and Methodology

The following approach and methodology were followed for Security Assessment of Teamwork Web application.

PHASE I: Project Planning, Initiation and Pre-Assessment

Establish initial information, coordination, timelines, and methods of communication. In general, every type of assessment possesses a common phase of planning.

The objectives of this phase are:

- **Define Objectives:** Clearly define the goals and objectives of the web application security assessment, such as identifying vulnerabilities, assessing the effectiveness of security controls, or meeting compliance requirements. Incorporate OSSTMM3 control metrics to ensure the security assessment's objectives are measurable and align with OWASP Top 10 2022 vulnerabilities OWASP Web Security Testing Guide and OWASP ASVS.
- **Establish Communication:** Coordinate with stakeholders, including project managers, developers, and system administrators, to ensure their understanding of the assessment process and to gather relevant information about the web application.
- **Obtain Necessary Permissions:** Obtain authorized permissions to conduct the security assessment on the web application. Ensure verification of trust levels for all entities involved in the assessment, as suggested by OSSTMM3.

PHASE II: Information Gathering and Threat Modelling

The objective of this phase Includes Identify Application Components: Application's architecture, Secure Deployment and Configuration, Provision to Patch and update, Secure Development of Update, Patch & Release to Mitigate Against Supply Chain Risk from Developers, technology stack, frameworks, and libraries used.

Enumerate Functionalities: Understand the various functionalities and features of the web application. Overview and walk through, understanding application, fingerprinting of OS layer, application layer, web server layer, crawling, domain name lookup, port scanning, service identification

Threat Modeling: Define the Attack Surface by identifying all entry points, interfaces, and potential vulnerabilities within the web application. Use OWASP Threat Modeling techniques to assess risks and ensure alignment with OSSTMM3's multi-layer security approach. Analyze Threats and Risks associated with the identified attack surface and correlate them across various layers as per OSSTMM3.

Control Verification: Prioritize threats, determine the impact and likelihood of each threat, and verify the trustworthiness of components across different layers (OSSTMM3's trust level verification), ensuring that OWASP's API security guidelines and data validation practices are applied.

OWASP Top 10 Proactive Controls: 1. Implement Access Control, 2. Use Cryptography to Protect Data, 3. Validate all Input & Handle Exceptions, 4. Address Security from the Start, 5. Secure By Default Configurations, 6. Keep your components Secure, 7. Secure Digital Identities, 8. Leverage Browser Security Features, 9. Implement Security Logging and Monitoring, 10. Stop Server Side Request Forgery.



PHASE III: Vulnerability Assessment and Penetration Testing

Vulnerability Assessment and Penetration Testing: - The activities performed in this phase includes vulnerability detection by using Automated Scanning: Utilize automated scanning tools to identify common vulnerabilities, including outdated software versions, misconfigurations, and known web application vulnerabilities. Manual Testing: Conduct manual penetration testing to discover complex vulnerabilities that automated scanners might miss. This involves attempting to exploit identified vulnerabilities and testing for potential attack scenarios as per both OWASP Top 10 2022 vulnerabilities OWASP Web Security Testing Guide, OWASP ASVS and OSSTMM3 guidelines. Authentication and Authorization Testing: Verify the robustness of authentication and authorization mechanisms, such as weak passwords, insecure session management, and privilege escalation. Input Validation and Output Encoding: Validate and sanitize input data to prevent common web application vulnerabilities such as Cross-Site Scripting (XSS), SQL injection, and Command Injection. Session Management and Error Handling. Secure Communication. Data Protection and Storage. Web application security assessment team followed Grey Box testing approach. Any unexpected reaction from the Application(s) was noted and investigated. The goal of this method was to identify the vulnerabilities in the application(s) and underlying server(s).

PHASE IV: Analysis and Risk Assessment

The activities performed in this phase are:

- **Analyze Findings:** Consolidate and analyze the results of vulnerability assessments and penetration tests. Correlate findings to ISO/IEC, Cyber Security Audit Baseline Requirements, Open Source Security Testing Methodology Manual (OSSTMM3) multi-layer security principles to ensure comprehensive coverage, OWASP Web Security Testing Guide, Method of verifications to compliance with CERT-In Directions issued by CERT-In, OWASP Top 10 2022 vulnerabilities and OWASP ASVS.
- **Identify Risks:** Evaluate the impact and likelihood of identified vulnerabilities and prioritize them based on their severity.
- **Risk Mitigation:** Recommend appropriate mitigation strategies and countermeasures to address identified risks and vulnerabilities.

PHASE V: Report Generation and Deliverable

The deliverables consist of a report describing the activities performed, identified vulnerabilities and recommendations for mitigating the vulnerabilities and the areas that needs improvement, with the leading practice recommendations. The project management is performed throughout the engagement duration.

Penetration testing is essentially carried out to generate proof of concepts (POC) for the identified vulnerabilities in the application.

Following is the methodology:

- Perform security assessment using tools such as Acunetix, Burp Suite Professional, Nmap etc. to identify the vulnerabilities.
- Identify vulnerabilities through manual techniques.
- Identify and remove false positives



- Assign severity ratings to identified vulnerabilities and generate Proof of Concept (POC), with the focus on measurable outcomes in line with OSSTMM3 control metrics and OWASP guidelines.

1.6 CERT-In Guidelines followed during the assessment

GRM Technologies Private Limited includes the verification of compliance to CERT-In direction:

- “Directions under sub-section (6) of section 70B of the Information Technology Act, 2000 relating to information security practices, procedure, prevention, response and reporting of cyber incidents for Safe & Trusted Internet” dated 28 April 2022.
- “Method of verifications to compliance with CERT-In Directions issued on 28.04.2022”.
- “Guidelines for Secure Application Design, Development, Implementation & Operations” issued by CERT-In.

1.7 Risk Rating

The scale mentioned below has been used to rate the risk of the observed vulnerabilities:

Risk Rating	Description
Critical	Any vulnerability that has a high potential of causing program failure (inability to achieve minimum acceptable requirements) with high likelihood of occurrence and low attack complexity due to which, a threat actor or attacker could gain privileged access and disrupt the normal flow of operations or for which there is a publicly available mechanism to exploit the Vulnerability.
High	Any vulnerability that has a high potential of impacting business operations leading to the downtime or the disruption and provides an attacker with the privileged access, customer service, or SLA breach resulting in a significant outage or penalty to any of the customer services
Medium	Any vulnerability that has the potential of indirectly giving access to an intruder and/or does not have the features enabled for collecting evidence or notifying the attacker of unauthorized use for taking legal action in case the vulnerability gets exploited. This type of vulnerability upon exploitation might result in the elevation of privileges or slowing down the operations.
Low	Any vulnerability that has the potential of revealing the information about the device may lead to unauthorized access to the system leading to the compromise. The higher work factor would be involved in exploiting this type of vulnerability.
Informational	IT security risk is the potential harm to processor-related information resulting from some purposeful or accidental event that negatively impacts the process or the related information.



2. Executive Summary

The following sections in the report highlight the key findings as well as the analysis of the vulnerabilities identified in the Teamwork Web Application.

It is highly recommended that the recommendations should be applied to the complete application based on issues reported in this report.

2.1 Analysis

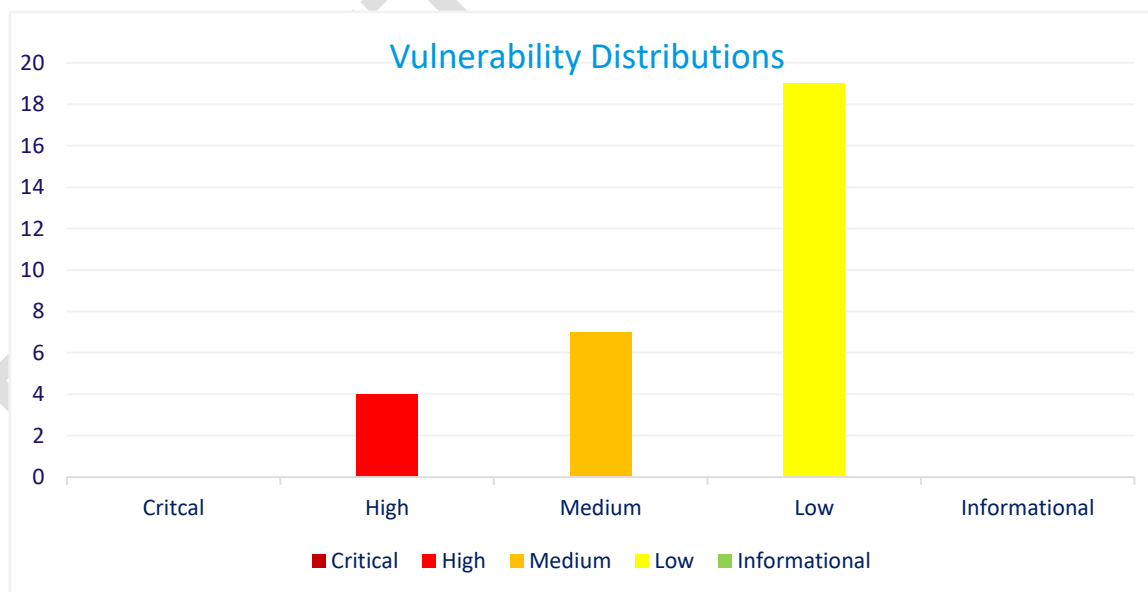
From the above-mentioned Teamwork Web Application in scope, 30 vulnerabilities were observed post-assessment for which breakup is mentioned below.

Critical	High	Medium	Low	Informational	Total
0	4	7	19	0	30

The table given below highlights the distribution of 30 vulnerabilities by components/platform.

#	Critical	High	Medium	Low	Informational	Total
Web Application	0	4	7	19	0	30
Total	0	4	7	19	0	30

The following graph shows the distribution of identified vulnerabilities with Critical, High, Medium, Low and Informational severity.





2.2 Vulnerability to Risk Mapping

The below table provides the mapping of each identified vulnerability along with the corresponding risk rating to vulnerable components.

S. No	Vulnerability	Risk Rating	OWASP Rating	Initial Status as on 30 th January 2025
1.	Reflected Cross-Site Scripting (XSS)	High	A03:2021	Open
2.	HTML Injection	High	A03:2021	Open
3.	SMTP Injection	High	A03:2021	Open
4.	Unrestricted File Upload	High	A05:2021	Open
5.	PII Data Exposure	Medium	A02:2021	Open
6.	Missing Rate Limiting	Medium	A07:2021	Open
7.	Sensitive File and Information Disclosure	Medium	A05:2021	Open
8.	Improper session management	Medium	A07:2021	Open
9.	Business Logic Flaw	Medium	A05:2021	Open
10.	Password Disclosure in Response	Medium	A02:2021	Open
11.	Weak Password Policy	Medium	A02:2021	Open
12.	User Enumeration	Low	A07:2021	Open
13.	Session Fixation	Low	A07:2021	Open
14.	Improper OTP Configuration	Low	A07:2021	Open
15.	Missing Autocomplete	Low	A05:2021	Open
16.	Server Name and Version Disclosure	Low	A05:2021	Open
17.	Improper Error Handling	Low	A05:2021	Open
18.	Clear Text Submission of Credentials	Low	A02:2021	Open
19.	Missing Security Headers	Low	A05:2021	Open
20.	Clickjacking Attack	Low	A05:2021	Open
21.	Improper Input Validation	Low	A05:2021	Open
22.	Vulnerable and Outdated Component	Low	A06:2021	Open
23.	Method Interchange	Low	A05:2021	Open
24.	Missing CAPTCHA	Low	A07:2021	Open
25.	Sensitive Information Disclosure via URL	Low	A05:2021	Open

26.	Open Redirection	Low	A01:2021	Open
27.	JSON Web Key set Disclosed	Low	A02:2021	Open
28.	Missing Cookie Attribute	Low	A05:2021	Open
29.	Concurrent Login Session	Low	A07:2021	Open
30.	Breach & Lucky13 Attack	Low	A05:2021	Open

2.3 Prioritization

Reported Critical and High vulnerabilities highlighted in audit reports should be patched by Teamwork immediately.

2.4 Overall Security Measure

The below table provides the overall security measure for the Teamwork web application.

Project Name	Description	Security Measure			Comments
		Poor	Controlled	Good	
Teamwork	The objective of this security assessment was to identify vulnerabilities/weaknesses that Teamwork Web Application may possess and to provide recommendations for risk mitigation https://login.teamwork.sg/				The current security posture is Controlled.



3. Detailed Technical Report

This section provides a detailed list of the observed vulnerabilities with the following information:

- Definition
- Description
- Risk Rating
- OWASP Rating
- Affected URL
- Root cause
- References
- Impact
- Recommendation(s) to minimize the risk
- Proof of Concept & Steps to reproduce the Vulnerability

3.1 Assessment Findings

Below are the vulnerabilities identified during the assessment and are listed according to the risk associated with them.



3.1.1 Reflected Cross-Site Scripting (XSS)

Parameters	Description
Vulnerability Severity	High
OWASP Rating	A03:2021 – Injection
Vulnerability Definition	Reflected XSS is a vulnerability where user-supplied input is immediately reflected back in the server's response without proper validation or encoding, allowing malicious scripts to execute in a victim's browser.
Vulnerability Description	During the assessment, we observed that the application allows user input in the Article Number field within the article/constitution page without proper validation or output encoding. When a JavaScript payload (e.g., <code><script>alert(reflected xss)</script></code>) was inserted into the Article Number field and the Save button was clicked, the application reflected the input in the response and executed the script, resulting in an alert pop-up. This behavior confirms that the application is vulnerable to Reflected Cross-Site Scripting (XSS).
Root Cause	User input is reflected in the response without proper output encoding or input sanitization.
Affected URL	• https://login.teamwork.sg/add_company • https://login.teamwork.sg/addon_settings • https://login.teamwork.sg/edit_admin/382 • https://login.teamwork.sg/tasks
Impact	An attacker can exploit this vulnerability by executing arbitrary JavaScript in a victim's browser, which may lead to session hijacking, account takeover, phishing attacks, or unauthorized actions performed on behalf of the victim.
References	• https://owasp.org/www-community/attacks/xss/ • https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/01-Testing_for_Reflected_Cross_Site_Scripting
Recommendation	• Validate and sanitize all user inputs on both client and server sides • Apply proper context-aware output encoding before reflecting user input • Restrict the use of special characters in sensitive fields such as Article Number • Implement a strong Content Security Policy (CSP)
Proof of Concept	

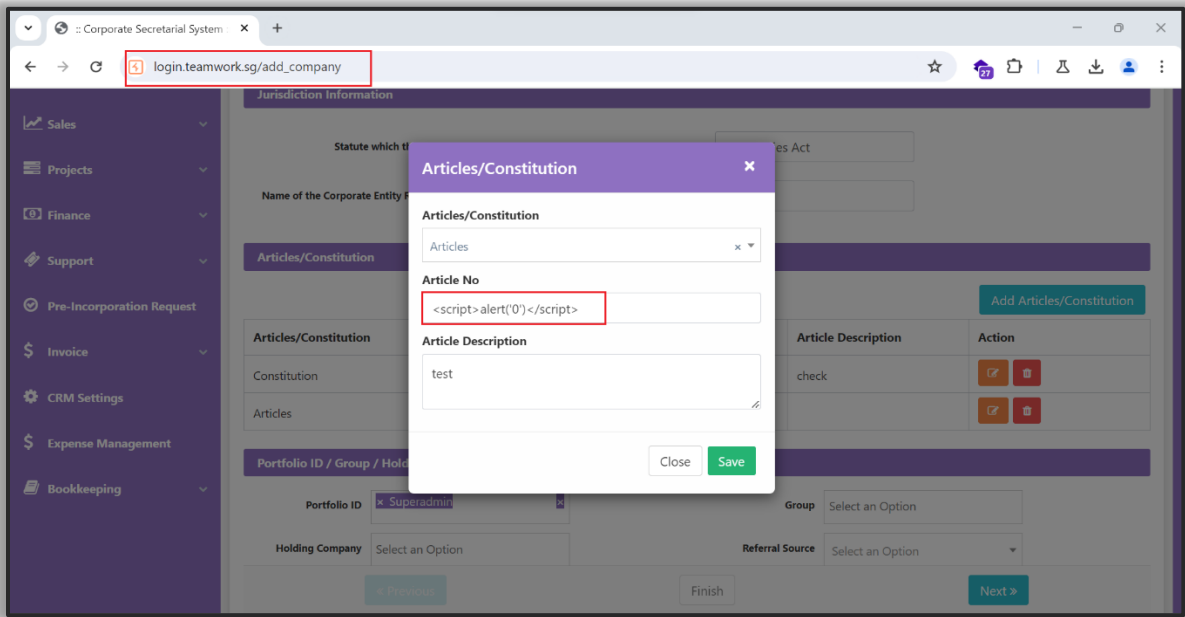


FIGURE 1: REFLECTED CROSS-SITE SCRIPTING (XSS) 01

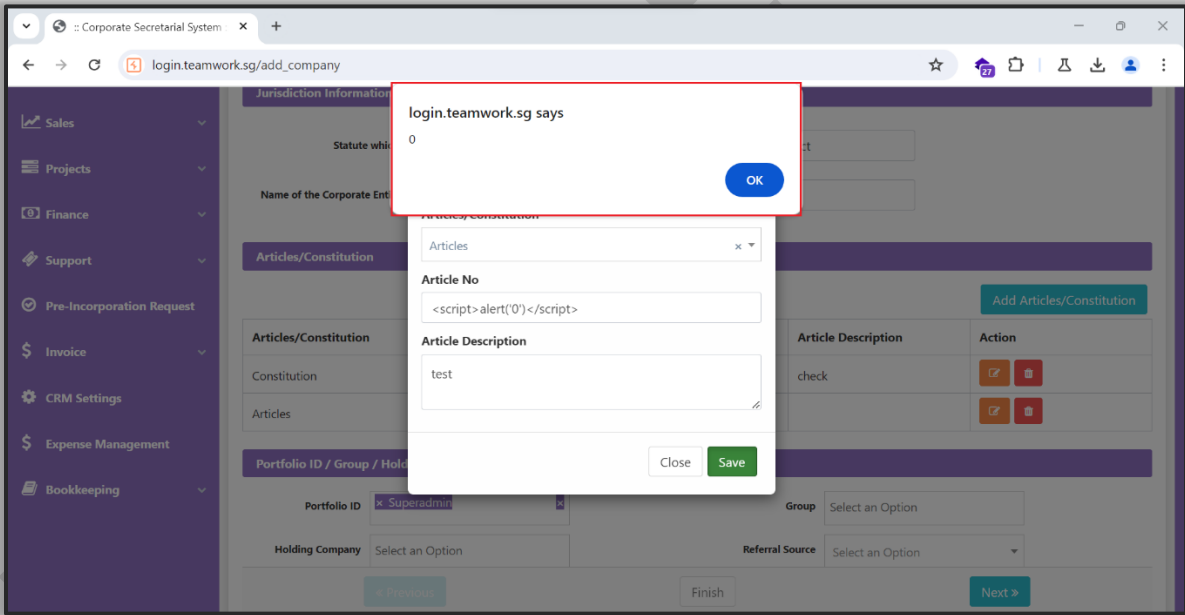


FIGURE 2: REFLECTED CROSS-SITE SCRIPTING (XSS) 02

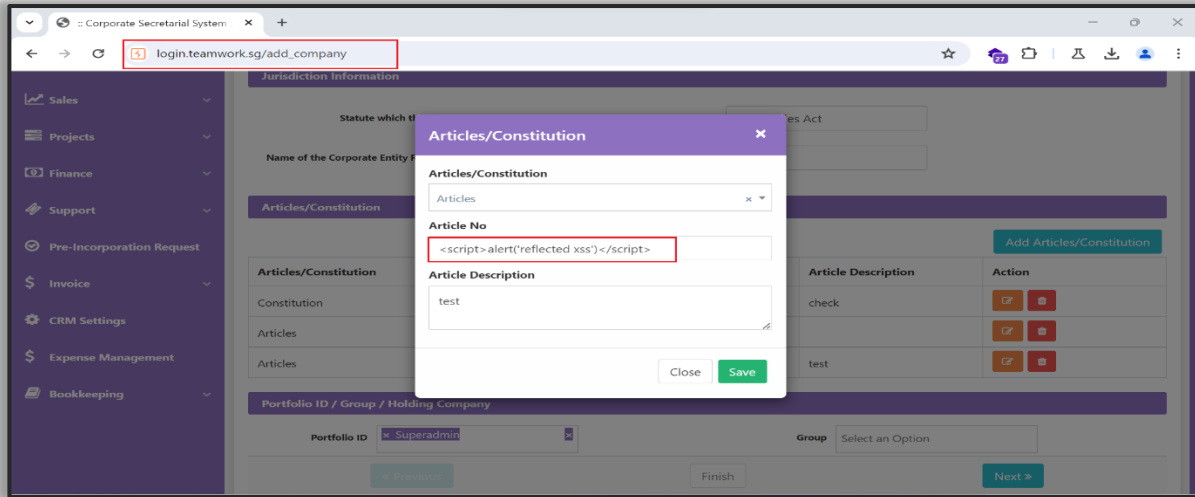


FIGURE 3: REFLECTED CROSS-SITE SCRIPTING (XSS) 03

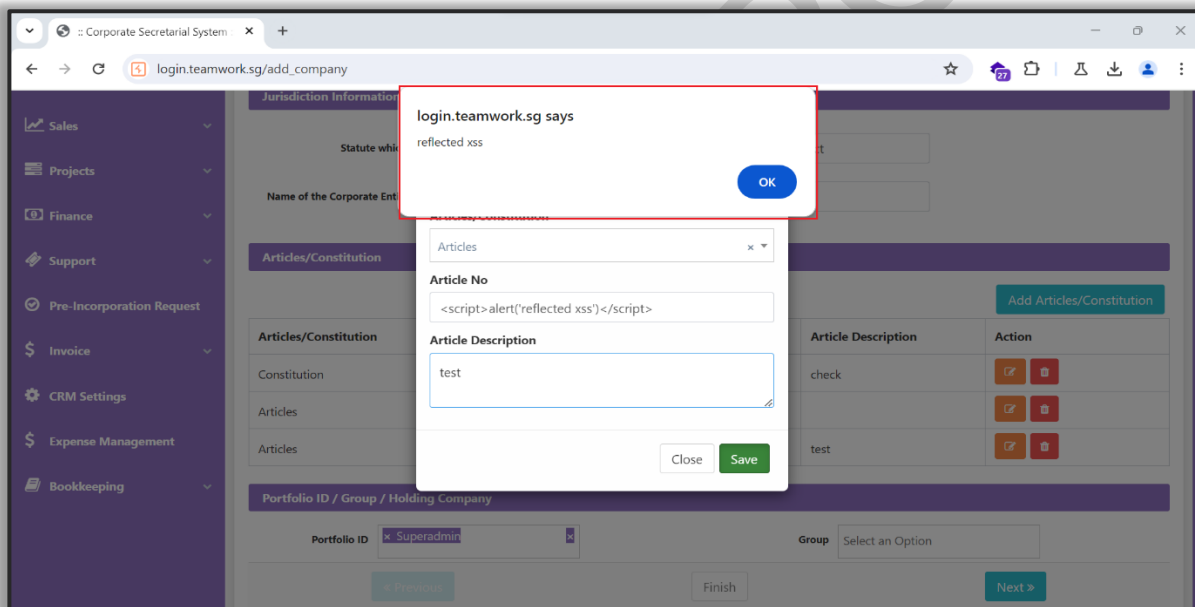


FIGURE 4: REFLECTED CROSS-SITE SCRIPTING (XSS) 04



3.1.2 HTML Injection

Parameters	Description
Vulnerability Severity	High
OWASP Rating	A03:2021 – Injection
Vulnerability Definition	HTML Injection is a vulnerability where an application allows an attacker to inject and execute arbitrary HTML content in a web page due to improper input validation and output encoding.
Vulnerability Description	During the assessment, we observed that the application does not properly validate user input in the Article Number field. When an HTML payload was inserted into this input field and the Save button was clicked, the injected HTML content was rendered directly in response. As a result, the HTML code was executed successfully and confirmed that HTML Injection is possible in the application.
Root Cause	User-supplied input is rendered in the application without proper sanitization or HTML encoding.
Affected URL	• https://login.teamwork.sg/add_company • https://login.teamwork.sg/settings/list_status • https://login.teamwork.sg/edit_admin/382
Impact	An attacker can inject malicious HTML content into the application, which may be used to deface pages, manipulate displayed content, perform phishing attacks, or act as a stepping stone for further client-side attacks such as XSS.
References	• https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection • https://www.acunetix.com/vulnerabilities/web/html-injection/
Recommendation	• Validate and sanitize all user inputs on the server side • Encode user input before rendering it in HTML responses • Restrict allowed characters in sensitive fields such as Article Number • Use secure frameworks or libraries that automatically escape HTML output
Proof of Concept	

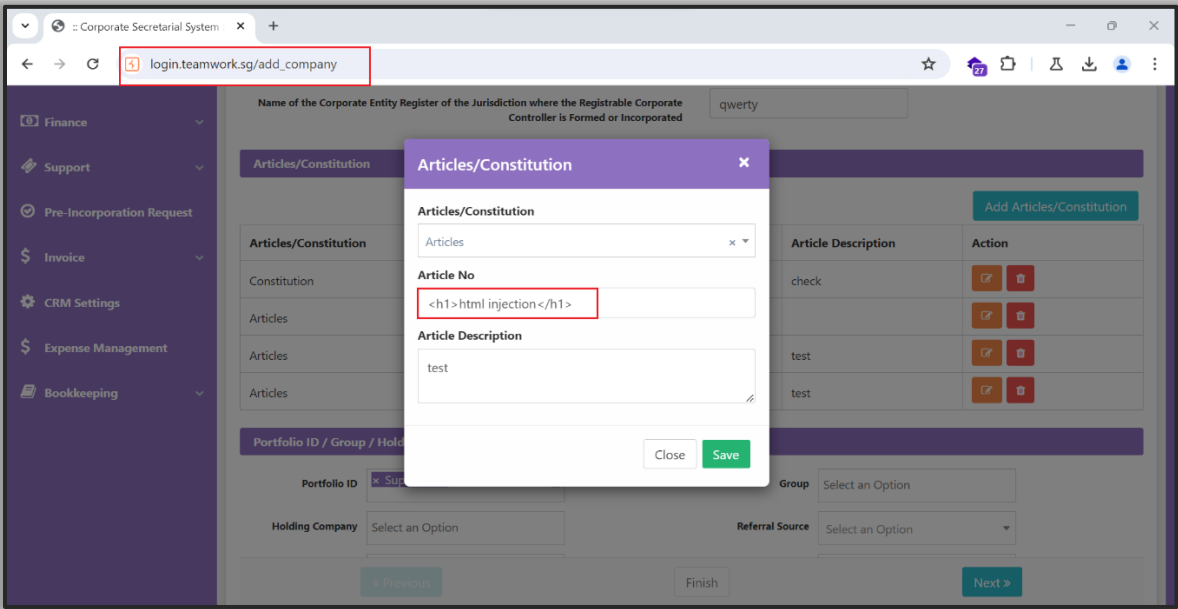


FIGURE 5: HTML INJECTION 01

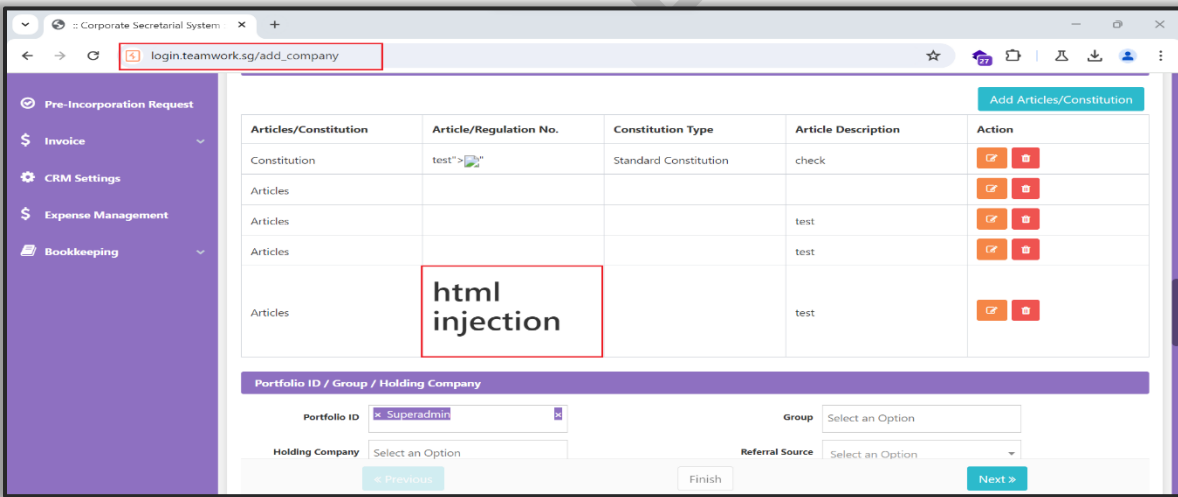


FIGURE 6: HTML INJECTION 02



3.1.3 SMTP Injection

Parameters	Description
Vulnerability Severity	High
OWASP Rating	A03:2021 – Injection
Vulnerability Definition	SMTP Injection is a vulnerability where an attacker manipulates email input fields to inject additional email addresses or headers, causing emails to be sent to unauthorized recipients.
Vulnerability Description	During the assessment, we observed that the application does not properly validate the email input. By adding another email address after the legitimate email and sending the request, the application processed both addresses and successfully sent the email, indicating improper handling of SMTP input.
Root Cause	Lack of proper validation and sanitization of user-supplied email input before sending SMTP requests.
Affected URL	https://login.teamwork.sg/edit_admin/382
Impact	An attacker can send emails to unauthorized recipients, abuse the email functionality for spam or phishing, impersonate legitimate users, and negatively impact the organization's reputation or cause email blacklisting.
References	https://portswigger.net/kb/issues/00200800_smtp-header-injection
Recommendation	• Strictly validate email input to allow only a single, properly formatted email address • Block special characters and separators such as commas, new lines (\r, \n) • Use secure email libraries instead of constructing SMTP requests manually • Implement monitoring and rate limiting on email functionality
Proof of Concept	

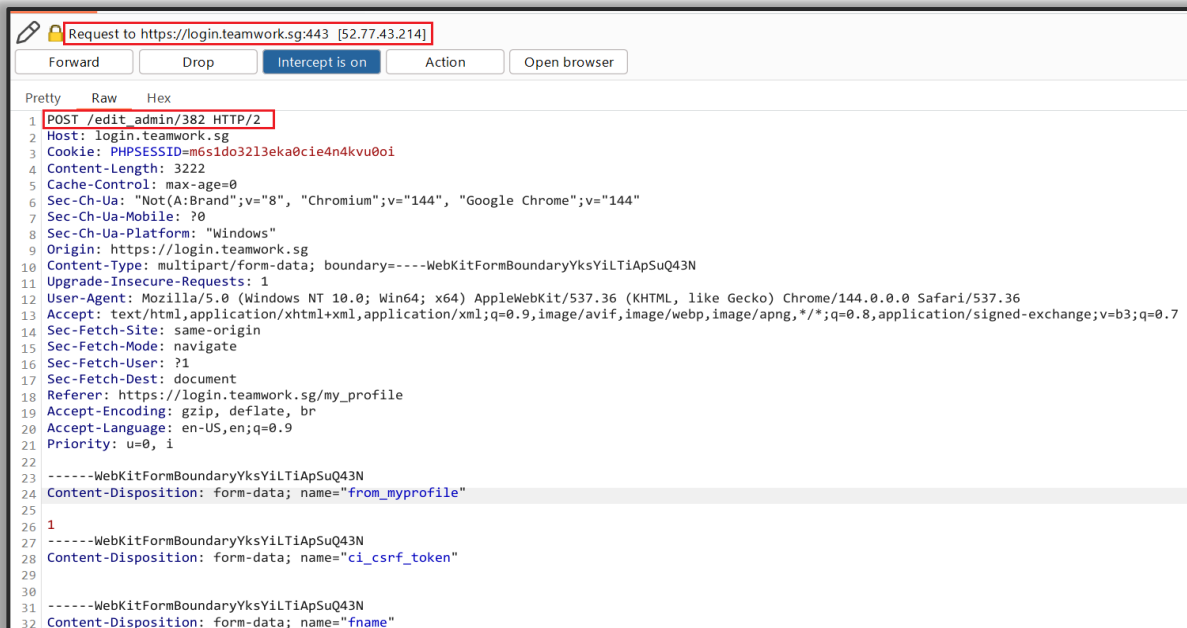


FIGURE 7: ORIGINAL REQUEST 01

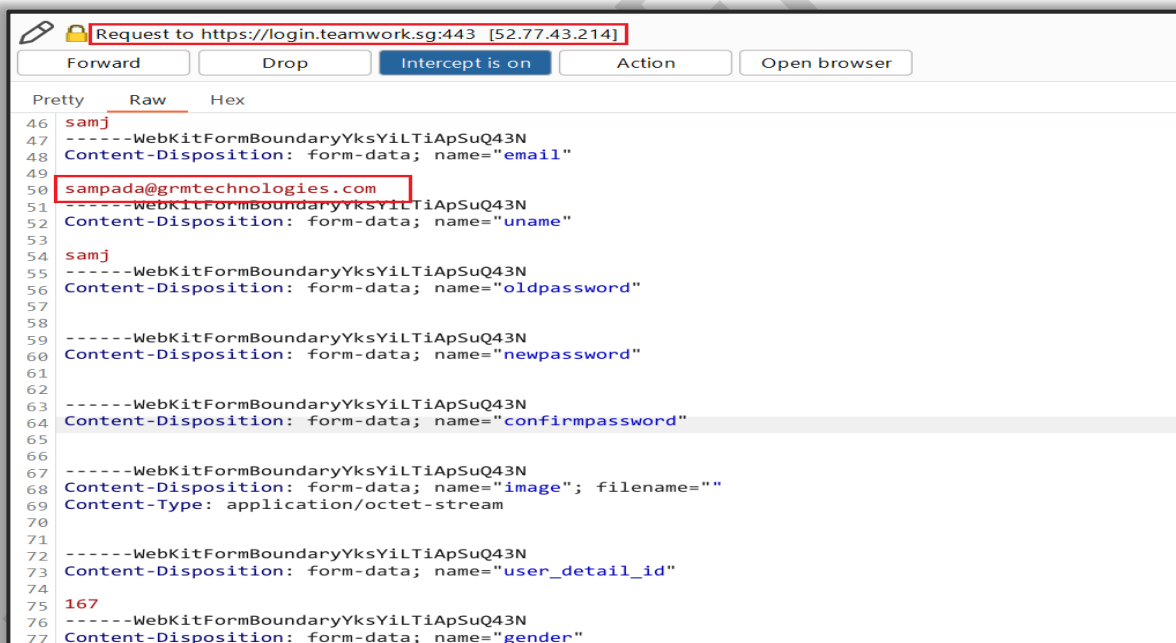


FIGURE 8: ORIGINAL REQUEST 02

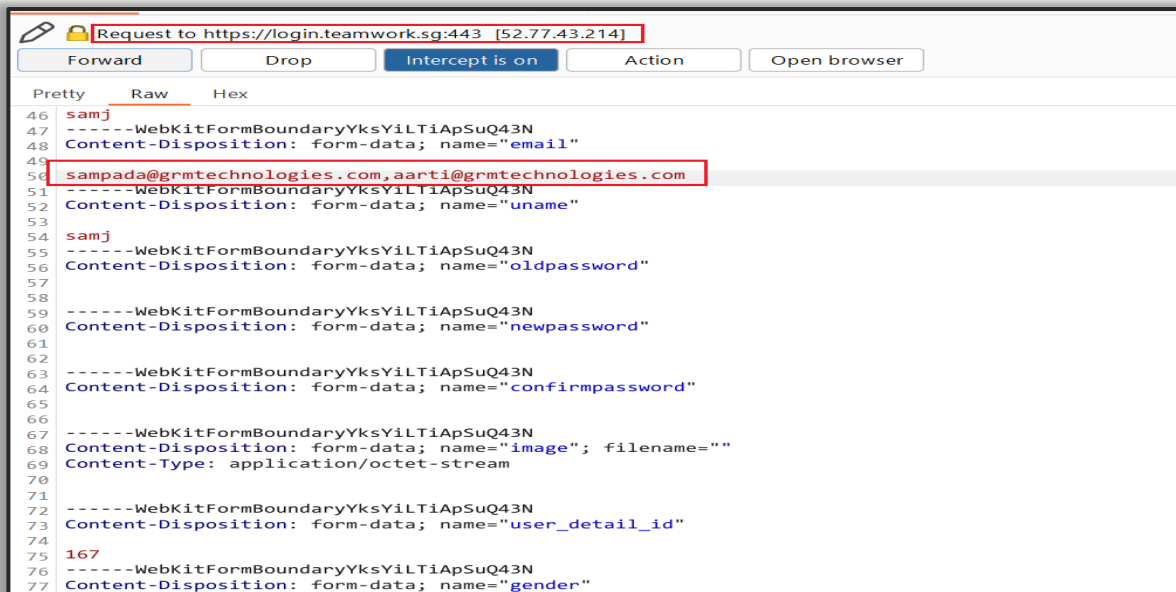


FIGURE 9: ADDING ANOTHER EMAIL ADDRESS AFTER THE LEGITIMATE USER.

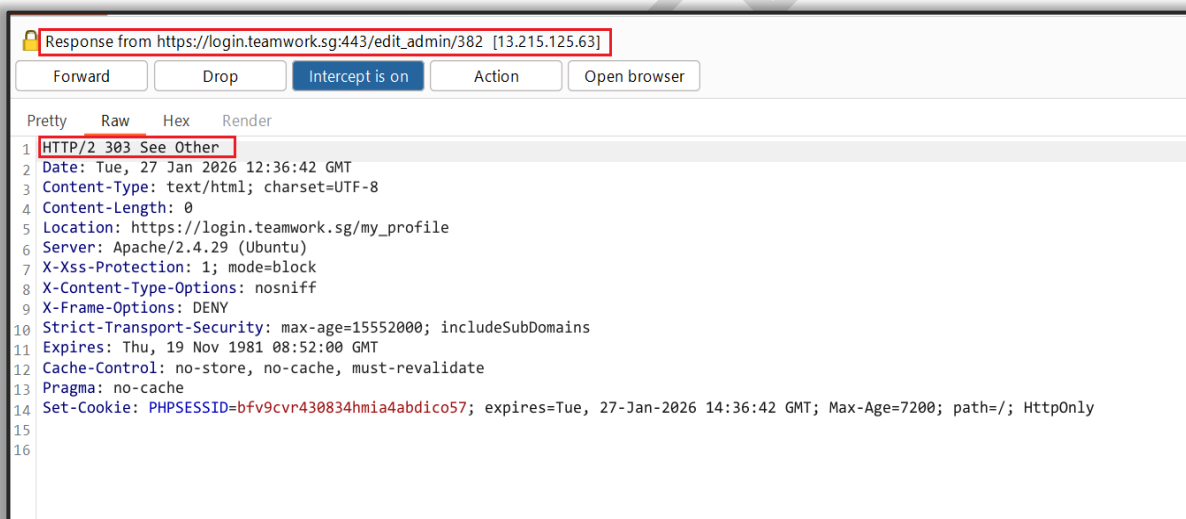
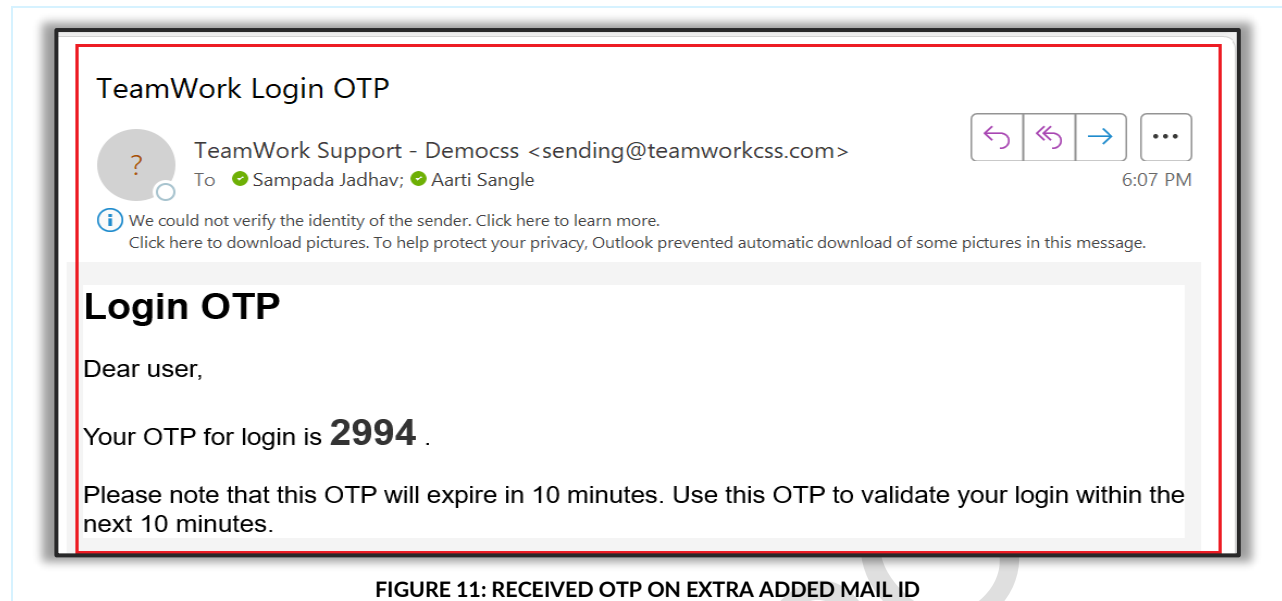


FIGURE 10: THE REQUEST RETURNS AN HTTP 303 RESPONSE WHICH CONFIRMS SMTP INJECTION.





3.1.4 Unrestricted File Upload

Parameters	Description
Vulnerability Severity	High
OWASP Rating	A05:2021 – Security Misconfiguration
Vulnerability Definition	Unrestricted File Upload is a vulnerability where an application allows users to upload files without properly validating their file type, extension, or content.
Vulnerability Description	During the assessment, we observed that the application does not properly validate uploaded files. The application allowed uploading files with double extensions such as abc.pdf.exe, executable files like test.exe.exe, and HTML files like html.html. These files were accepted and uploaded successfully, indicating insufficient file type and extension validation.
Root Cause	Improper validation of file extensions, MIME types, and file content during the upload process.
Affected URL	• https://login.teamwork.sg/edit_admin/382 • https://login.teamwork.sg/my_profile
Impact	An attacker can upload malicious files that may lead to malware distribution, server compromise, unauthorized code execution, phishing attacks, or defacement of the application.
References	• https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html • https://portswigger.net/web-security/file-upload • https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
Recommendation	• Allow only strictly required file types using a whitelist approach • Validate file extensions, MIME types, and file content on the server side • Block double extensions and executable file types • Rename uploaded files and store them outside the web root
Proof of Concept	

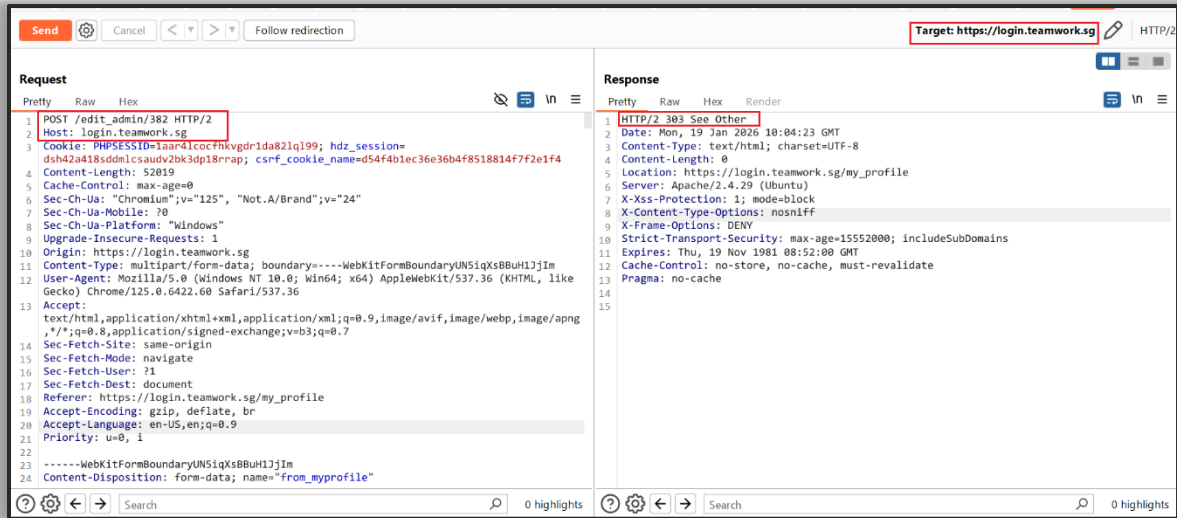


FIGURE 12 – CAPTURE THE REQUEST IN BURPSUITE 01

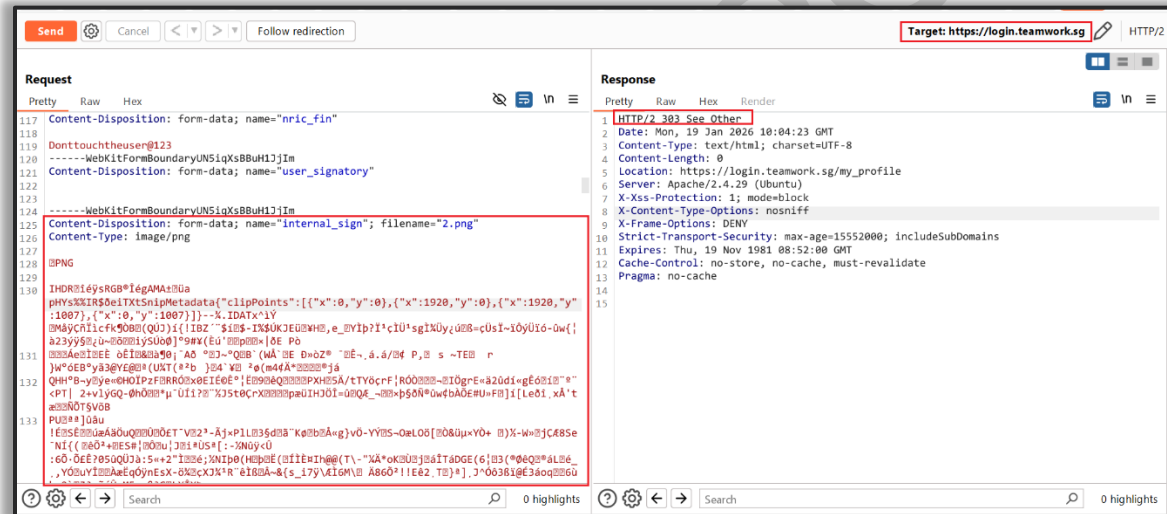


FIGURE 13 – CAPTURE THE REQUEST IN BURPSUITE 02

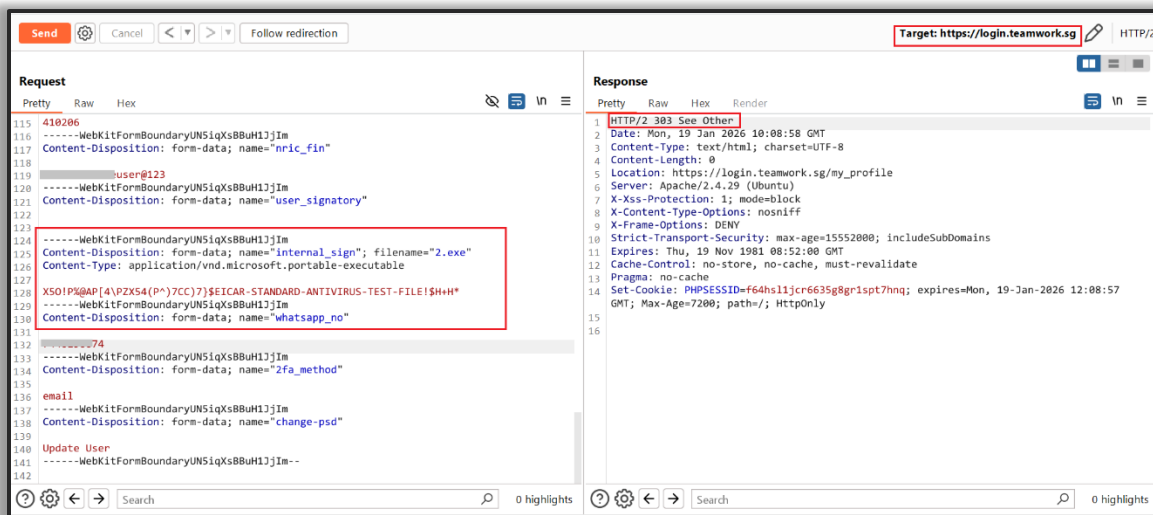


FIGURE 14 –UPLOADING EICAR ANTIVIRUS TEST STRING WITH MODIFIED EXTENTION

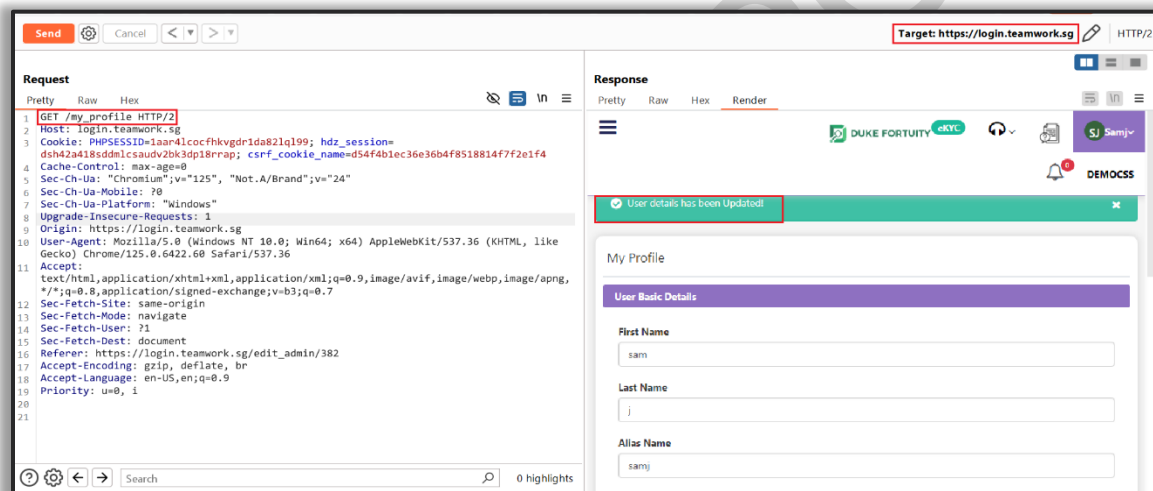


FIGURE 15 – MALICIOUS FILE UPLOADED SUCCESSFULLY

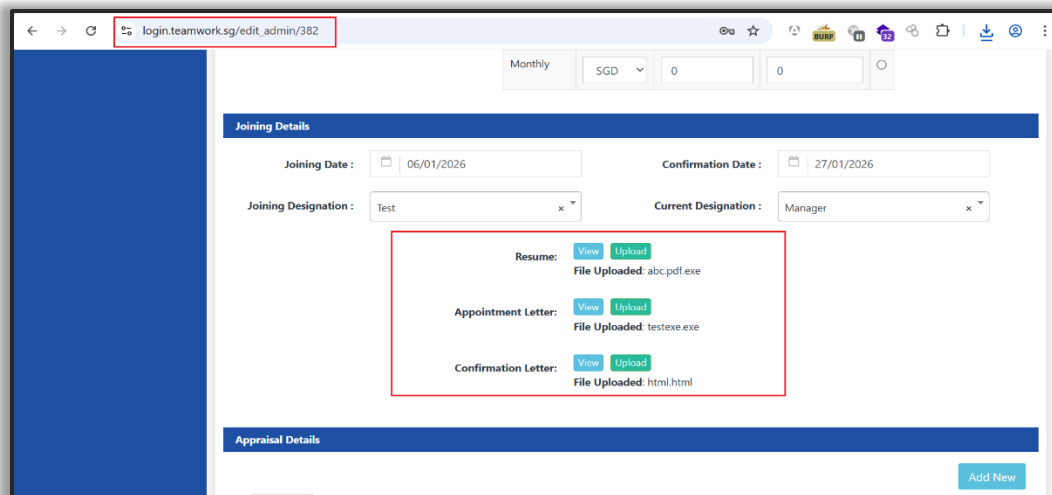


FIGURE 16 - MALICIOUS FILES ARE VISIBLE AND ACCESSIBLE, CONFIRMING UNRESTRICTED FILE UPLOAD.



3.1.5 PII Data Exposure

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A02:2021 – Cryptographic Failures
Vulnerability Definition	PII Data Exposure occurs when an application exposes sensitive personal or confidential information without proper masking or protection.
Vulnerability Description	During the assessment, we observed that the application exposes sensitive information such as Account number, PAN card number, Aadhar Number email IDs, passport numbers, client IDs, and client secret IDs in clear text within the application responses or UI. These sensitive details were not masked or protected, making them directly visible to users. Exposing such information increases the risk of unauthorized access and misuse.
Root Cause	Sensitive data is returned or displayed without proper masking, encryption, or access control.
Affected URL	• https://login.teamwork.sg/add_company • https://login.teamwork.sg/invoice/invoice_settings • https://login.teamwork.sg/my_profile • https://login.teamwork.sg/crm/insert_bank_of_user • https://login.teamwork.sg/duke_cdd/submit_artemis_configuration
Impact	An attacker or unauthorized user can access sensitive personal and confidential information, which may lead to identity theft, account compromise, unauthorized API access, financial fraud, and regulatory or compliance issues.
References	https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure
Recommendation	• Mask sensitive fields such as email IDs, passport numbers, and client secrets • Never expose client secrets in UI or API responses • Encrypt sensitive data both at rest and in transit • Apply proper access controls and least privilege principles
Proof of Concept	

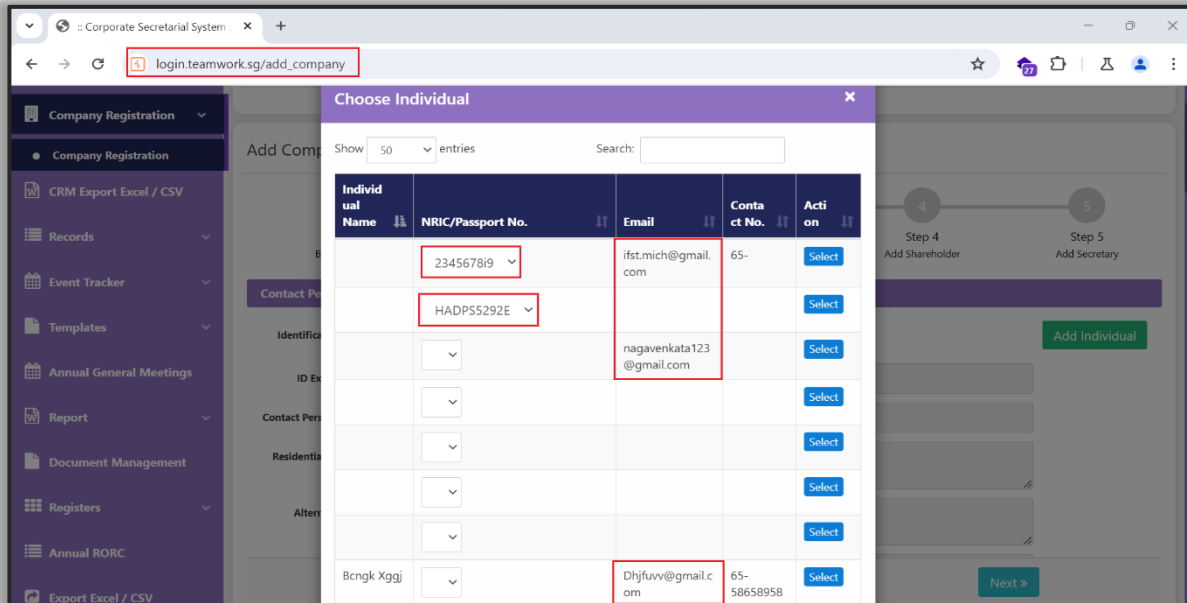


FIGURE 17 – SENSITIVE PII LIKE PASSPORT NUMBER AND EMAIL ARE NOT MASKED.

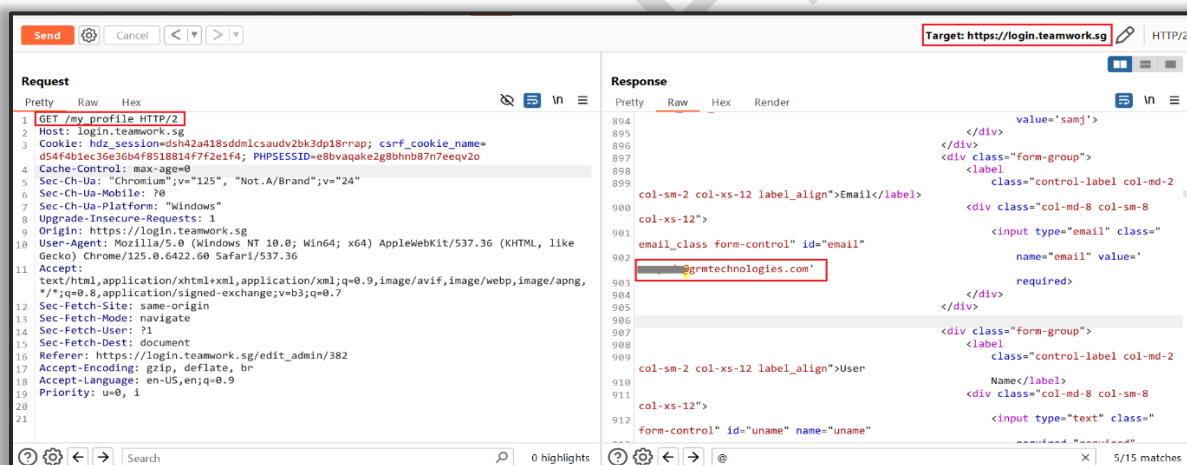


FIGURE 18 – EMAIL ID IS NOT MASKED



Actions	Bank Name	Branch Code	Bank Code	Bank Swift Code	Account Name	Account Type	Account Number
✕	DBS	1	23	DBS	Shine	Joinec	66

FIGURE 19- SENSITIVE PII LIKE ACCOUNT NUMBER IS NOT MASKED 01

```
Request
POST /crm/insert_bank_of_user HTTP/2
Host: login.teamwork.sg
Cookie: PHPSESSID=5p89e4kchoonhle7t68kk9n1c
Content-Length: 955
Sec-Ch-Ua-Platform: "Windows"
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
Accept: */*
Sec-Ch-Ua: "Not(A:Brand);v=8", "Chromium";v="144", "Google Chrome";v="144"
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryvmuaKKQ2ehPSU7tM
Sec-Ch-Ua-Mobile: 70
Origin: https://login.teamwork.sg
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://login.teamwork.sg/edit_admin/382
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Priority: u=1, i
-----WebKitFormBoundaryvmuaKKQ2ehPSU7tM
Content-Disposition: form-data; name="id"

Response
HTTP/2 200 OK
Date: Tue, 27 Jan 2026 06:46:43 GMT
Content-Type: application/json
Content-Length: 384
Server: Apache/2.4.29 (Ubuntu)
X-Xss-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=15552000; includeSubDomains
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache

{
  "message": "",
  "status": true,
  "data": {
    "user_id": "382",
    "bank_id": "9",
    "account_no": "909098909890",
    "account_type": "",
    "status": "Active",
    "account_holder_full_name": "sam j",
    "deleted": "0",
    "created_by": "382"
  }
}
```

FIGURE 20- SENSITIVE PII LIKE ACCOUNT NUMBER IS NOT MASKED 02



The screenshot shows the 'CDD Configuration - Artemis Details' page in the Teamwork web application. The page has a blue sidebar with navigation links: Dashboard, Client On-boarding, Commission, Company Registration, CRM Export Excel / CSV, Records, Event Tracker, Templates, Annual General Meetings, and Report. The main content area contains a form with the following fields:

Authentication URL	Client ID	Client Secret ID
https://crm.cynopsis.co	37d8c792-8035-4267-a072-9182a20807fd	84961622c6cca99d77bfe870963668a7
Backend URL	Domain ID	Assignee ID
https://api.artemis.cynopsis.co	5151	

A green 'Submit' button is located below the form fields. The Client ID and Client Secret ID fields contain sensitive PII-like information and are not masked.

FIGURE 21 – SENSITIVE PII LIKE CLIENT ID AND CLIENT SECRET ID IN NOT MASKED



3.1.6 Missing Rate Limiting

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A07:2021-Identification and Authentication Failures
Vulnerability Definition	Missing Rate Limiting is a vulnerability where an application does not restrict the number of requests a user can make within a specific time frame.
Vulnerability Description	During the assessment, we observed that the application does not enforce rate limiting on sensitive endpoints such as authentication or form submission. Multiple requests could be sent continuously without any restriction or blocking, indicating that rate limiting controls are not implemented.
Root Cause	Absence of request throttling or rate-limiting controls on critical application endpoints.
Affected URL	• https://login.teamwork.sg/crm/insert_bank_of_user • https://login.teamwork.sg/edit_admin/382
Impact	An attacker can perform brute-force attacks, credential stuffing, enumeration, or cause denial-of-service-like conditions by sending a large number of requests, potentially leading to account compromise or service disruption.
References	• https://www.geeksforgeeks.org/no-rate-limiting-flaw-in-cyber-security/ • https://learn.snyk.io/lesson/no-rate-limiting/?ecosystem=javascript
Recommendation	• Implement rate limiting sensitive endpoints. • Apply IP-based and user-based request throttling • Implement temporary account lockout after multiple failed attempts • Monitor and log excessive request activity
Proof of Concept	

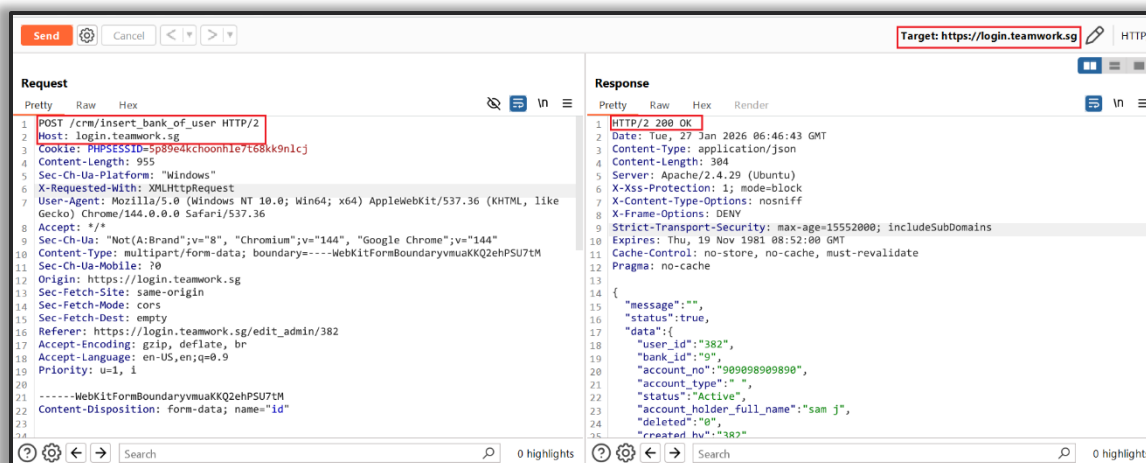


FIGURE 22: MISSING RATE LIMITING 01

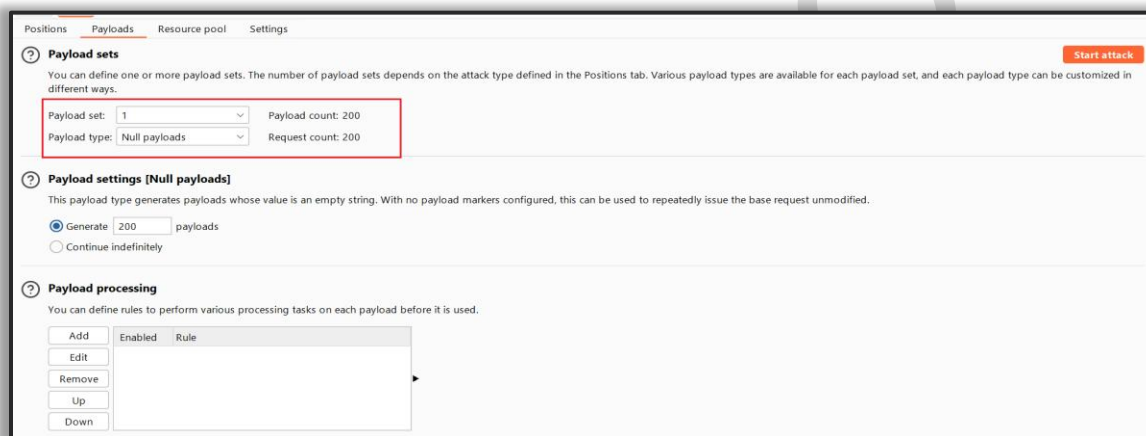


FIGURE 23: MISSING RATE LIMITING 02



Attack Save

5. Intruder attack of https://login.teamwork.sg

Attack Save ?

Results Positions Payloads Resource pool Settings

Intruder attack results filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
187	null	200	155			706	
188	null	200	155			706	
189	null	200	143			706	
190	null	200	142			706	
191	null	200	143			706	
192	null	200	142			706	
193	null	200	142			706	
194	null	200	150			706	
195	null	200	158			706	
196	null	200	144			706	
197	null	200	170			706	
198	null	200	153			706	
199	null	200	154			706	
200	null	200	154			706	

Request Response

Pretty Raw Hex Render

```
19 {
20   "bank_id": "9",
21   "account_no": "909098909890",
22   "account_type": " ",
23   "status": "Active",
24   "account_holder_full_name": "sam j",
25   "deleted": "0",
26   "created_by": "382"
27 }
```

Finished

FIGURE 24: MISSING RATE LIMITING 03

login.teamwork.sg/edit_admin/382

Google API Key

Pre-Incorporation Request

Invoice

CRM Settings

Expense Management

Bookkeeping

Bank Details

Show 10 entries Search Add Bank

Holder Name	Bank	Bank Code	Branch Code	Account Number	Status
1 sam j	asdas	sadas	asdasda	909098909890	Active
2 sam j	asdas	sadas	asdasda	909098909890	Active
3 sam j	asdas	sadas	asdasda	909098909890	Active
4 sam j	asdas	sadas	asdasda	909098909890	Active
5 sam j	asdas	sadas	asdasda	909098909890	Active
6 sam j	asdas	sadas	asdasda	909098909890	Active
7 sam j	asdas	sadas	asdasda	909098909890	Active
8 sam j	asdas	sadas	asdasda	909098909890	Active
9 sam j	asdas	sadas	asdasda	909098909890	Active
10 sam j	asdas	sadas	asdasda	909098909890	Active

Showing 1 to 10 of 204 entries Previous 1 2 3 4 5 ... 21

FIGURE 25: MISSING RATE LIMITING 04



3.1.7 Sensitive File and Information Disclosure

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A05: Security Misconfiguration
Vulnerability Definition	Sensitive File and Information Disclosure occurs when an application unintentionally exposes internal files, configuration details, source code paths, database information, or debug data to unauthorized users, which should not be publicly accessible.
Vulnerability Description	During assessment we observed that the application discloses sensitive internal information through publicly accessible files i.e. phpinfo file, sql file and verbose PHP error messages. The application exposes internal server paths, source code file names, line numbers, framework structure, and database-related errors.
Root Cause	Lack of access restrictions on sensitive files
Affected URL	• https://login.teamwork.sg/phpinfo.php • https://login.teamwork.sg/Companyform/download_all_templates • https://login.teamwork.sg/Companyform/company_form • https://login.teamwork.sg/settings/list_crm_master_cycle
Impact	• Identify framework, database type, and version • Discover internal directory structure
References	https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure
Recommendation	• Remove sensitive files. • Enable custom error pages



Proof of Concept

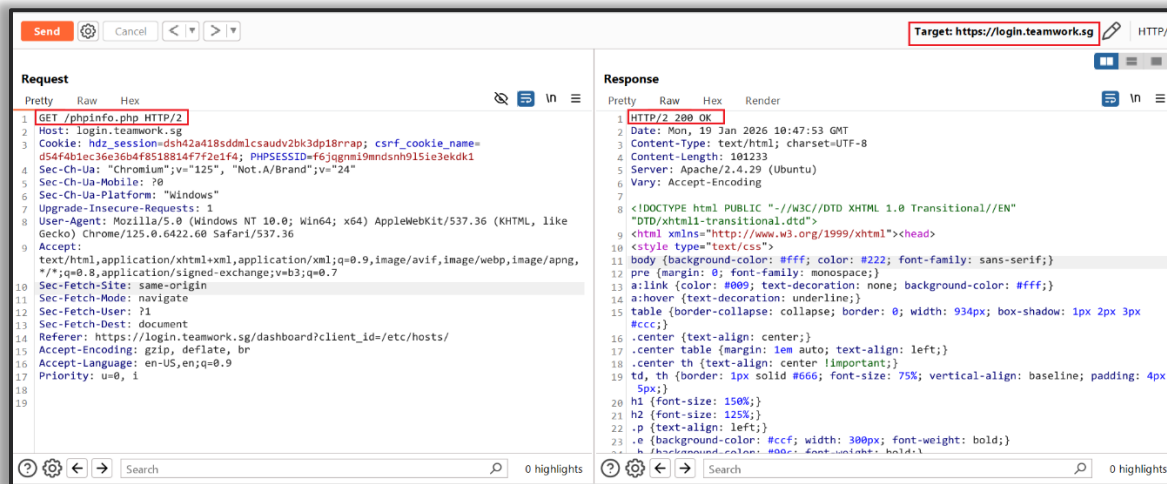


FIGURE 26 - DISCLOSURE OF PHPINFO FILE 01

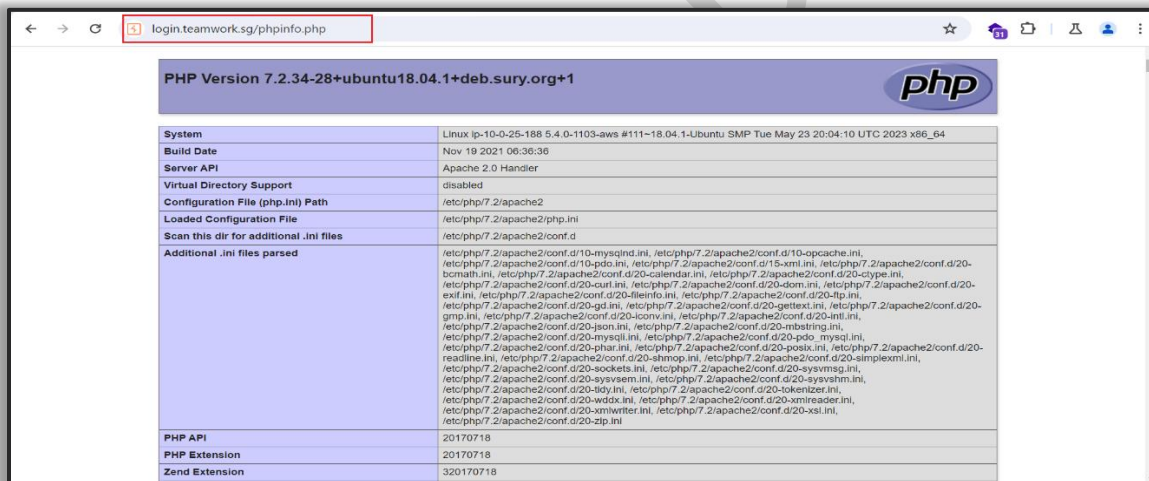
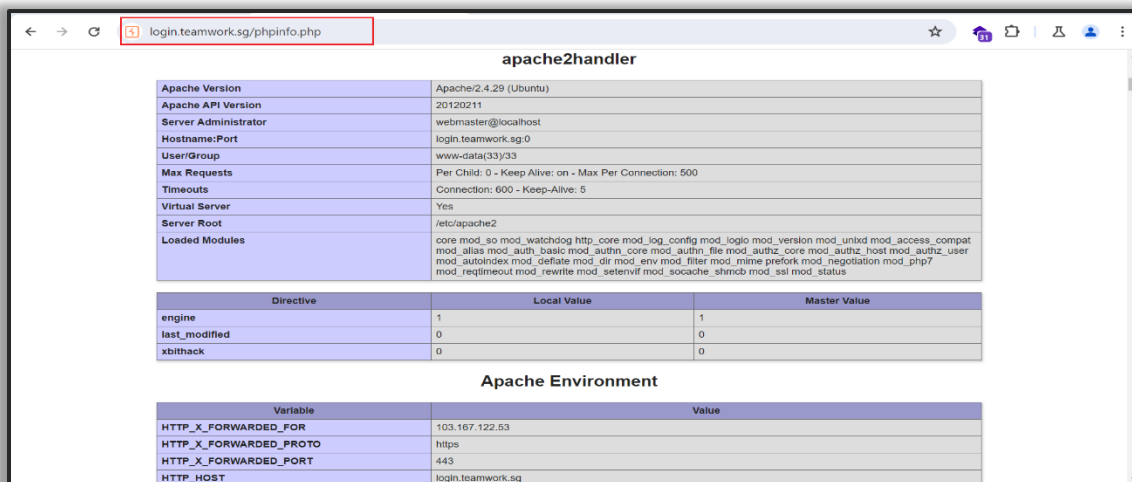


FIGURE 27 - DISCLOSURE OF PHPINFO FILE 02



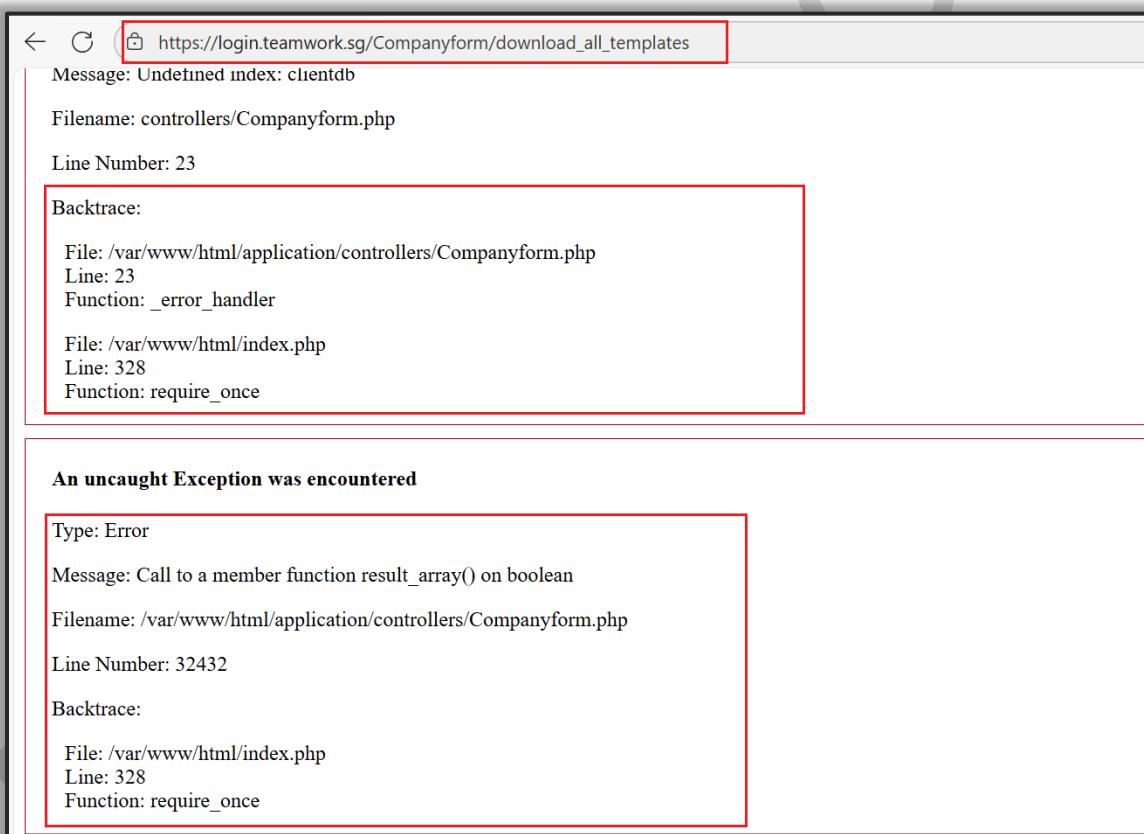
The screenshot shows the output of the phpinfo.php file, which displays various system and PHP configuration details. The information is organized into several sections: 'apache2handler' for Apache-related settings, 'Loaded Modules' for installed PHP modules, a 'Directive' table for core directives, and an 'Apache Environment' table for environment variables.

apache2handler	
Apache Version	Apache/2.4.29 (Ubuntu)
Apache API Version	20120211
Server Administrator	webmaster@localhost
Hostname:Port	login.teamwork.sg:0
User/Group	www-data(33)/33
Max Requests	Per Child: 0 - Keep-Alive: on - Max Per Connection: 500
Timeouts	Connection: 600 - Keep-Alive: 5
Virtual Server	Yes
Server Root	/etc/apache2
Loaded Modules	core mod_so mod_watchdog http_core mod_log_config mod_logio mod_version mod_unixd mod_access_compat mod_alias mod_auth_basic mod_auth_core mod_auth_file mod_authz_core mod_authz_host mod_authz_user mod_autoindex mod_deflate mod_dir mod_env mod_filter mod_inflate mod_negotiation mod_php7 mod_reqtimeout mod_rewrite mod_setenvif mod_socache_shmcb mod_ssl mod_status

Directive	Local Value	Master Value
engine	1	1
last_modified	0	0
xbithack	0	0

Variable	Value
HTTP_X_FORWARDED_FOR	103.167.122.53
HTTP_X_FORWARDED_PROTO	https
HTTP_X_FORWARDED_PORT	443
HTTP_HOST	login.teamwork.sg

FIGURE 28 – DISCLOSURE OF PHPINFO FILE 03



The screenshot displays a PHP error message from a web browser. The error message includes the URL, the message itself, the filename, the line number, a backtrace, and a detailed description of the exception.

Message: Undefined index: clientdb

Filename: controllers/Companyform.php

Line Number: 23

Backtrace:

- File: /var/www/html/application/controllers/Companyform.php
- Line: 23
- Function: _error_handler
- File: /var/www/html/index.php
- Line: 328
- Function: require_once

An uncaught Exception was encountered

Type: Error

Message: Call to a member function result_array() on boolean

Filename: /var/www/html/application/controllers/Companyform.php

Line Number: 32432

Backtrace:

- File: /var/www/html/index.php
- Line: 328
- Function: require_once

FIGURE 29 – DISCLOSURE OF INTERNAL FILE NAME AND PATH 01

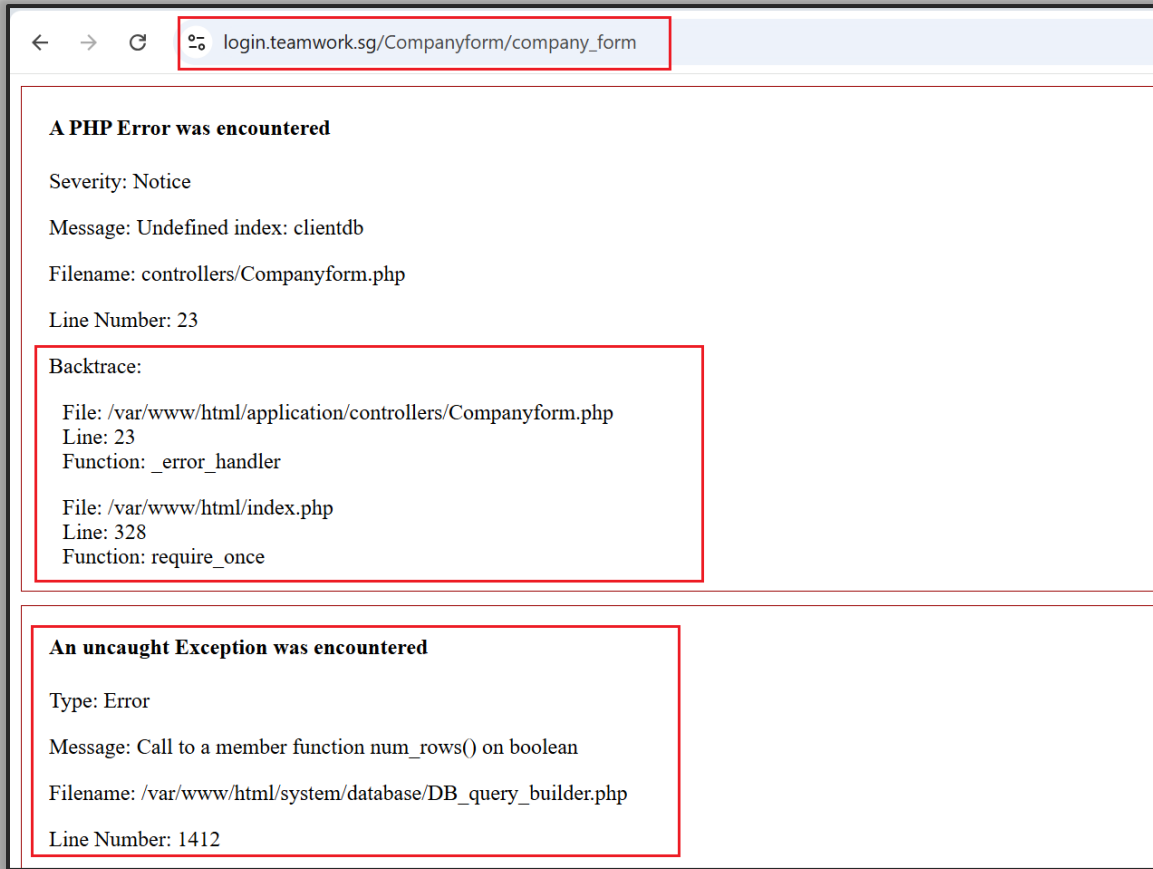


FIGURE 30 – DISCLOSURE OF INTERNAL FILE NAME AND PATH 02

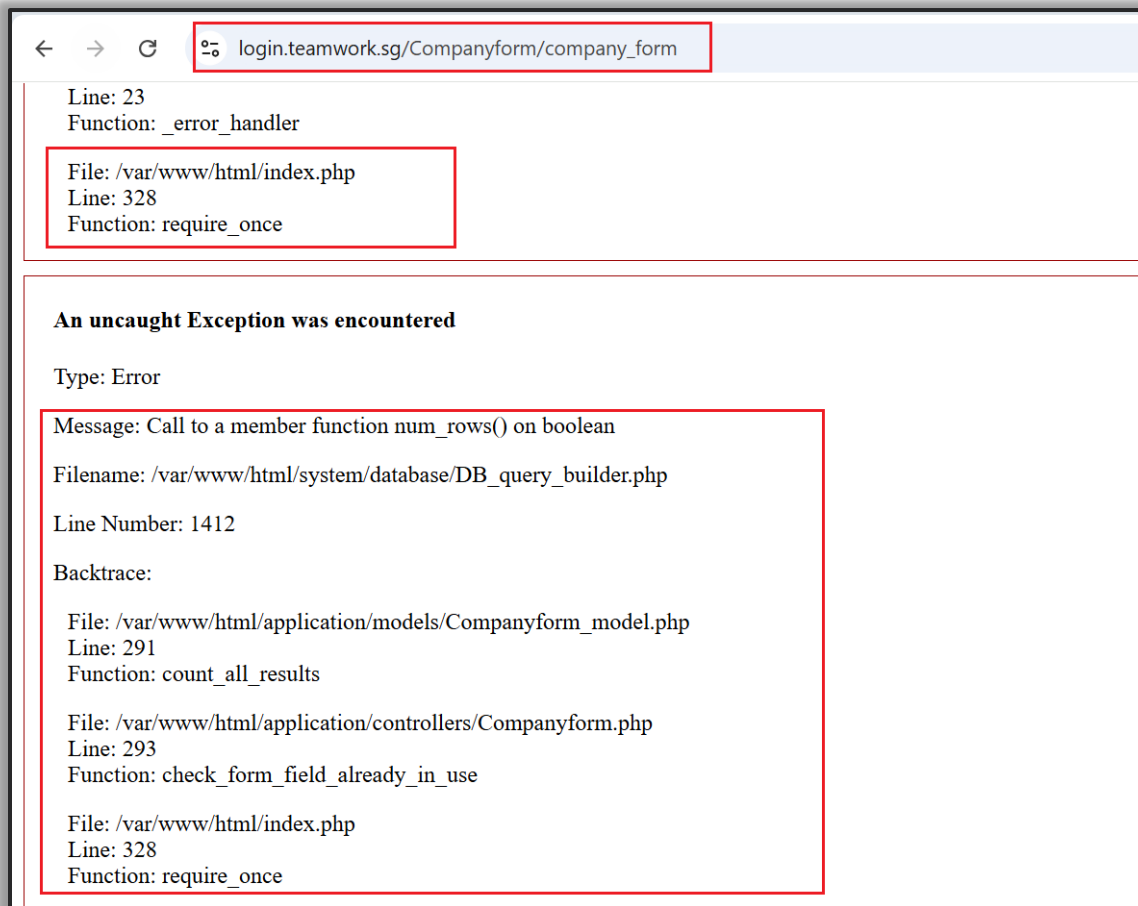


FIGURE 31 – DISCLOSURE OF INTERNAL FILE NAME AND PATH 03

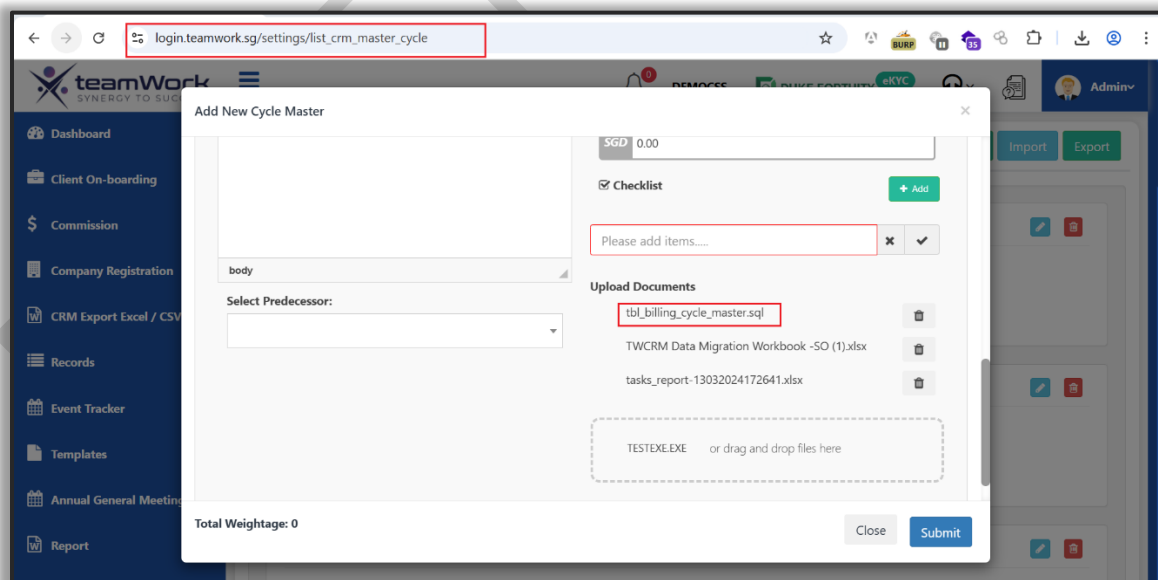


FIGURE 32 – DISCLOSURE OF SQL FILE 01

FIGURE 33 – DISCLOSURE OF SQL FILE 02

FIGURE 34 – DISCLOSURE OF SQL FILE 02



3.1.8 Improper session management

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A07:2021 – Identification and Authentication Failures
Vulnerability Definition	Improper Session Management occurs when the web application fails to properly manage and invalidate user sessions, exposing the application to session hijacking.
Vulnerability Description	During assessment, it was observed that the application does not implement proper session timeout mechanisms. Specifically: The session does not expire after 5-15 minutes of user inactivity. The session remains active even after closing and reopening the browser window. The session remains active even after log-out. This behavior indicates that session tokens remain valid indefinitely or for a very long duration, increasing the risk of session hijacking if the token is stolen or left on a shared/public computer.
Root Cause	• Lack of server-side session timeout configuration based on inactivity period. • Session cookies are set without proper Expires or Max-Age attributes.
Affected URL	• https://login.teamwork.sg/invoice/invoice_settings • https://login.teamwork.sg/my_profile
Impact	• Increased risk of session hijacking or misuse if session tokens are leaked or intercepted. • Users on shared or public machines can have their session misused after they leave.
References	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
Recommendation	• Configure the server to expire sessions after a defined inactivity period (e.g., 5–15 minutes). • Use Session Expiry on Browser Close. • Ensure that sessions are fully invalidated on logout, including server-side session store cleanup.
Proof of Concept	

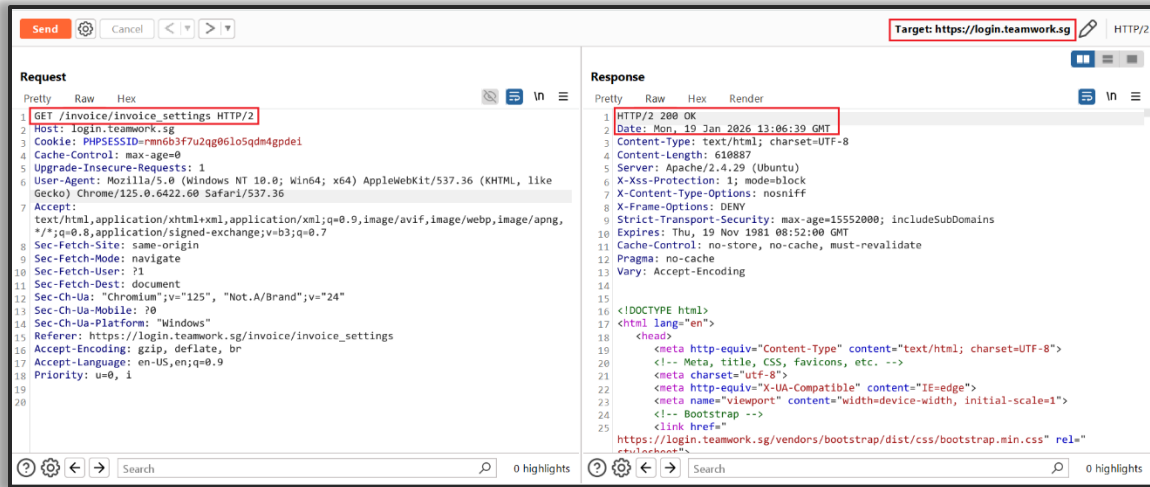


FIGURE 35: HIGH SESSION INACTIVITY 01

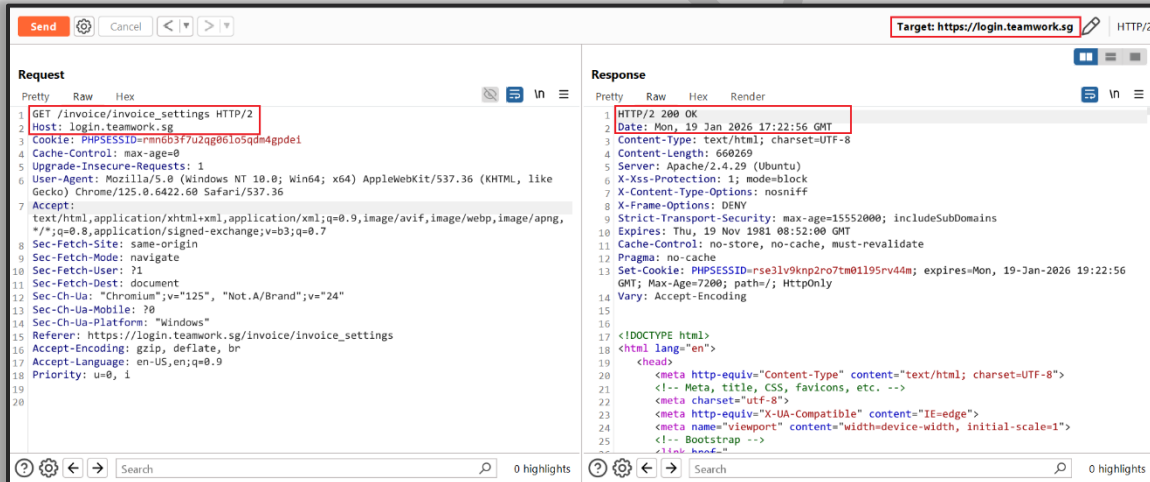


FIGURE 36: HIGH SESSION INACTIVITY 02

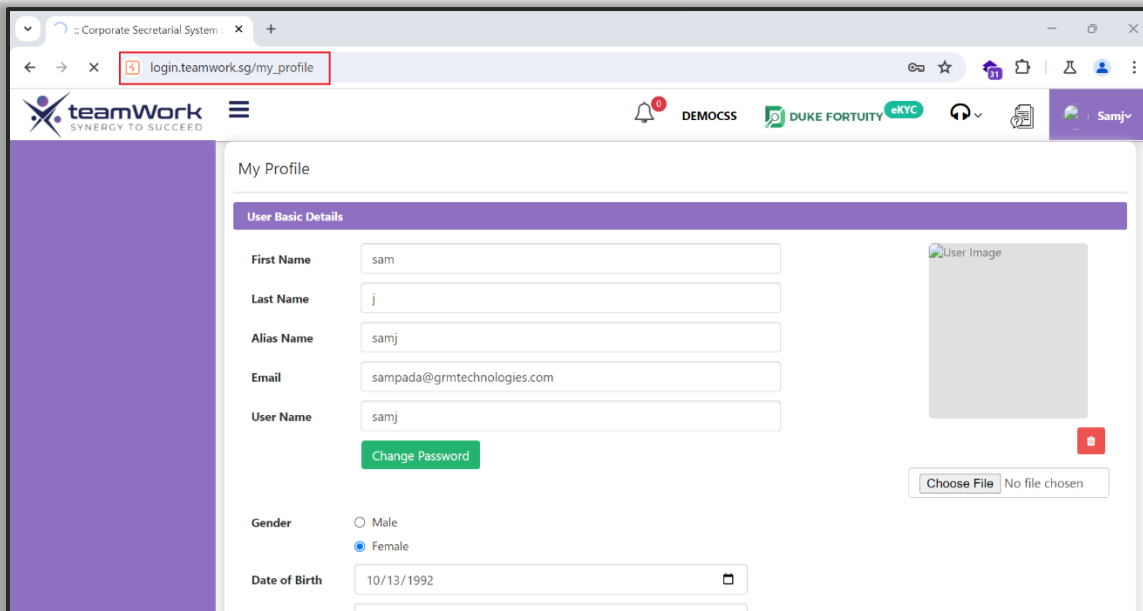


FIGURE 37: SESSION REMAINS ACTIVE AFTER LOG-OUT 01

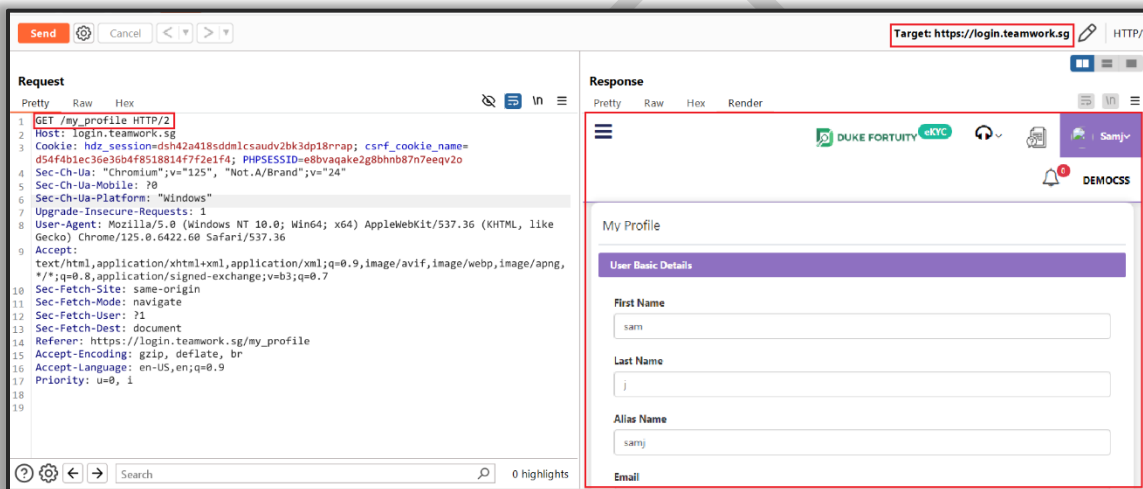


FIGURE 38: SESSION REMAINS ACTIVE AFTER LOG-OUT 02

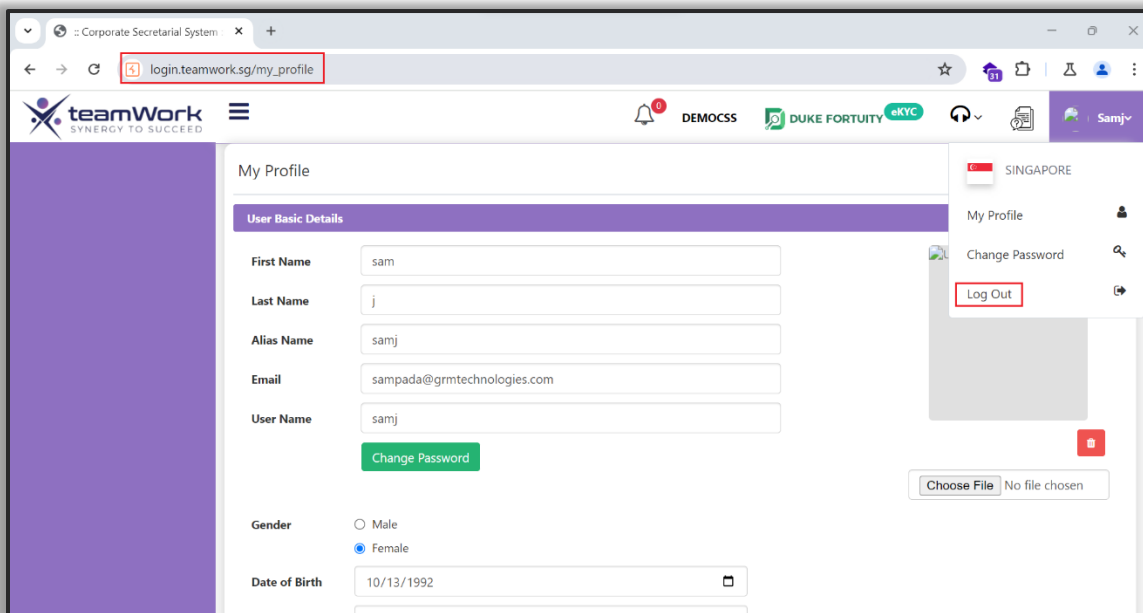


FIGURE 39: SESSION REMAINS ACTIVE AFTER LOG-OUT 03

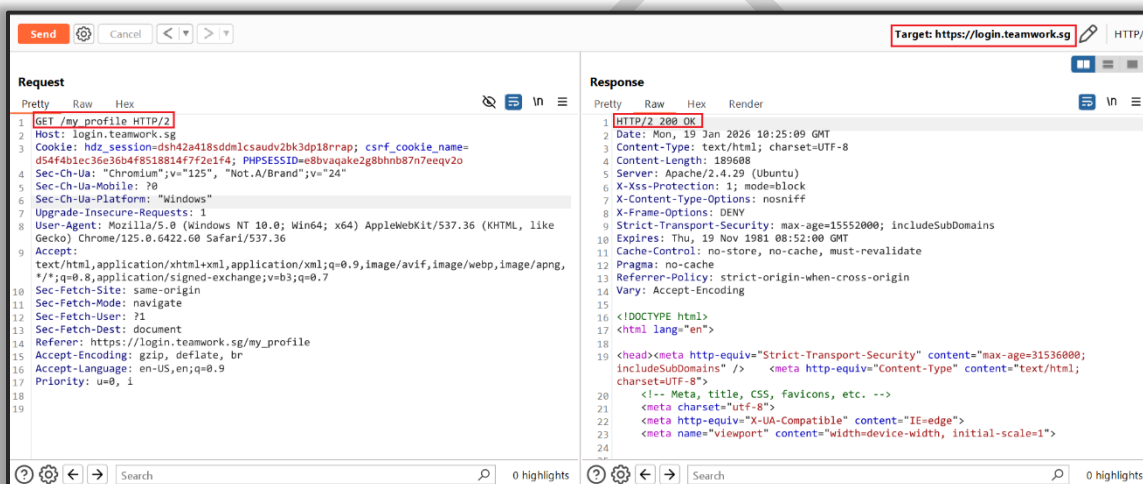


FIGURE 40: SESSION REMAINS ACTIVE AFTER LOG-OUT 04

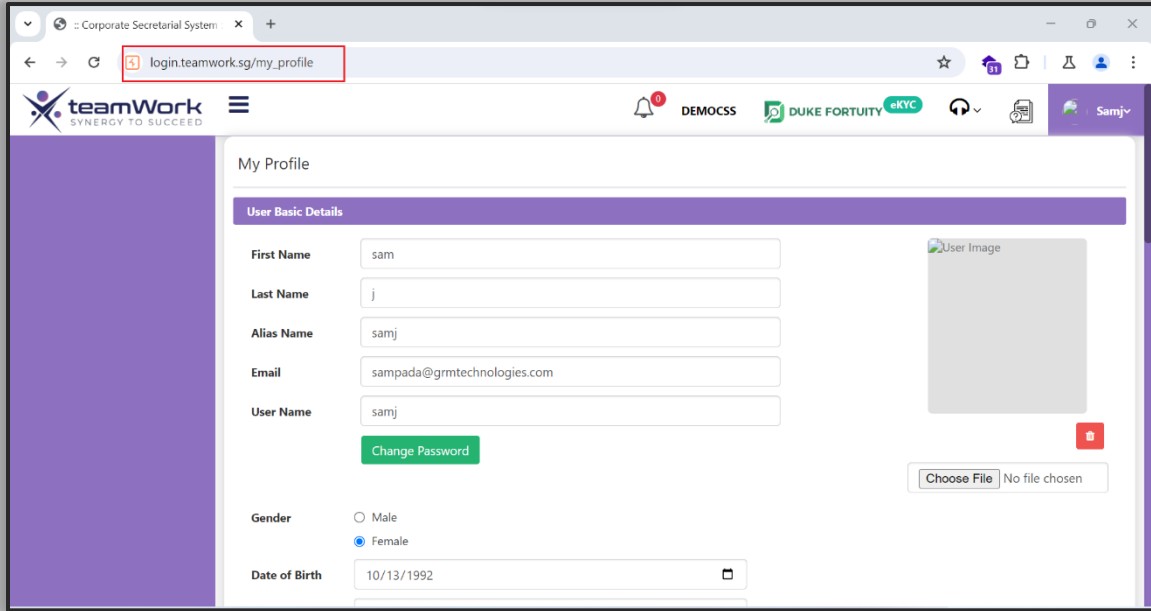


FIGURE 41: SESSION REMAINS ACTIVE AFTER CLOSING AND REOPENING THE BROWSER WINDOW 01

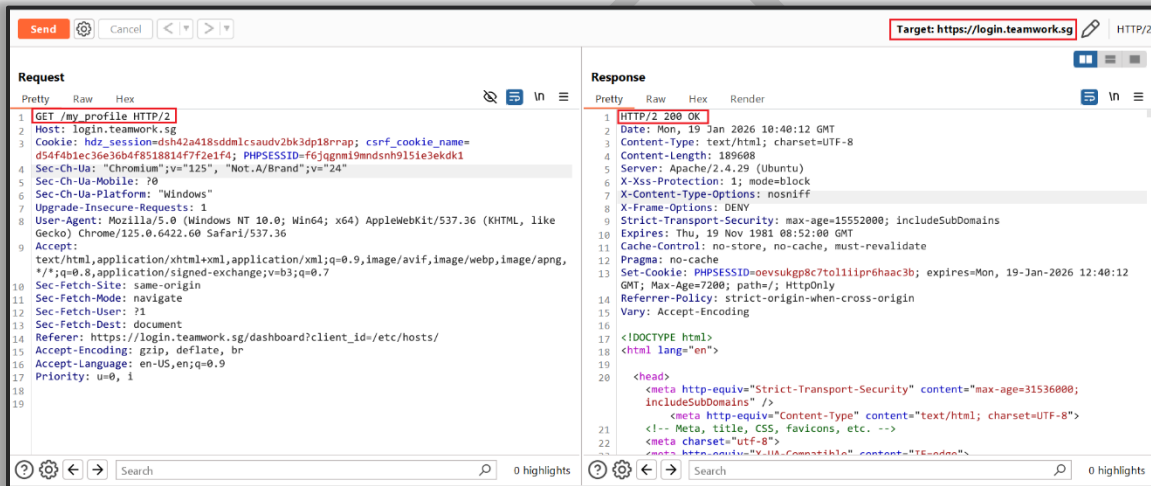


FIGURE 42: SESSION REMAINS ACTIVE AFTER CLOSING AND REOPENING THE BROWSER WINDOW 02

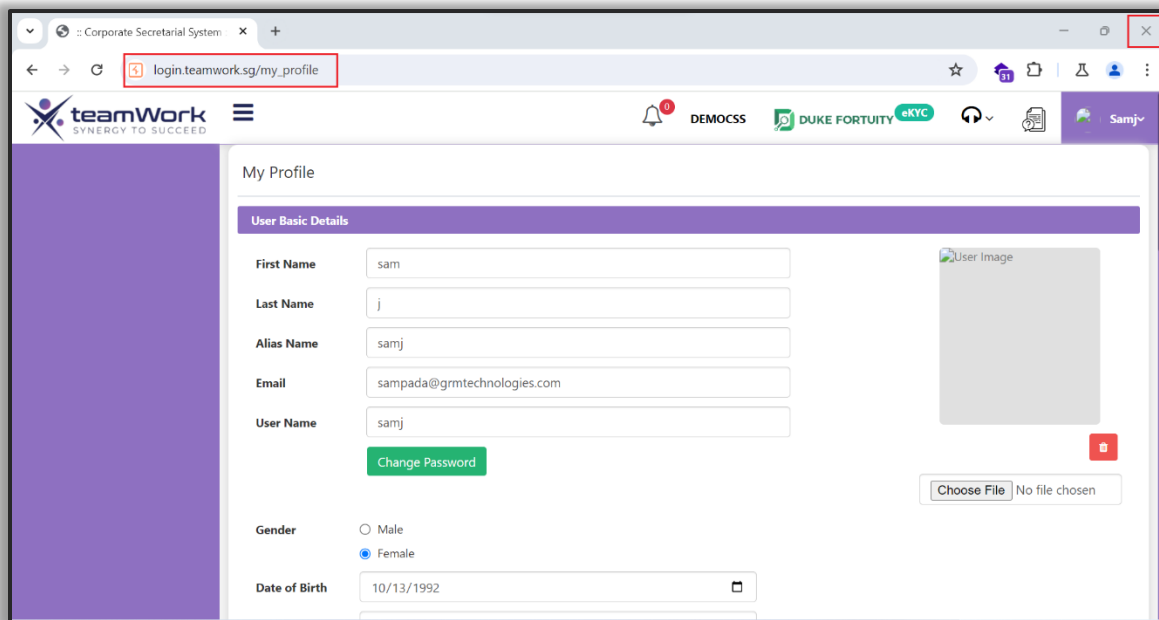


FIGURE 43: SESSION REMAINS ACTIVE AFTER CLOSING AND REOPENING THE BROWSER WINDOW 03

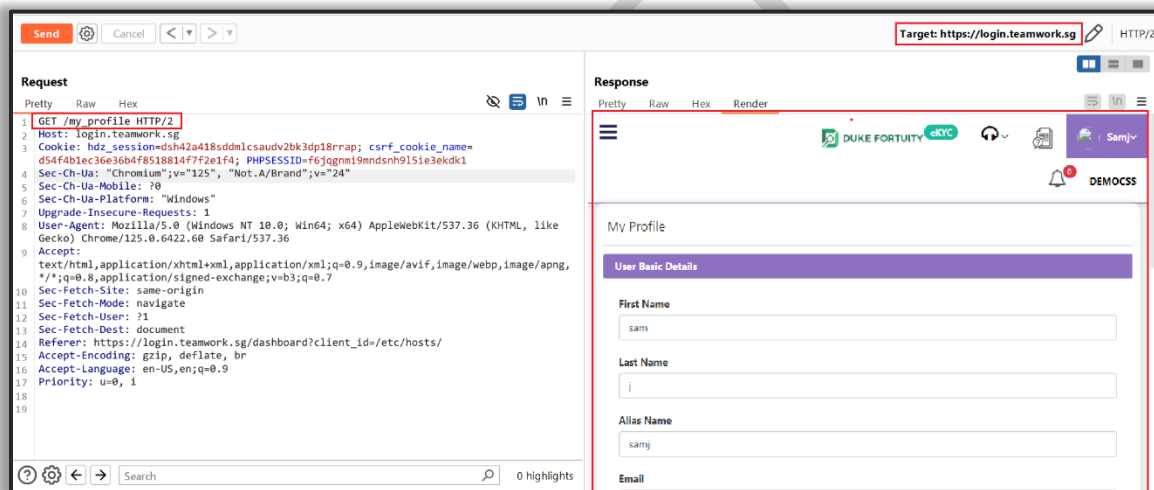
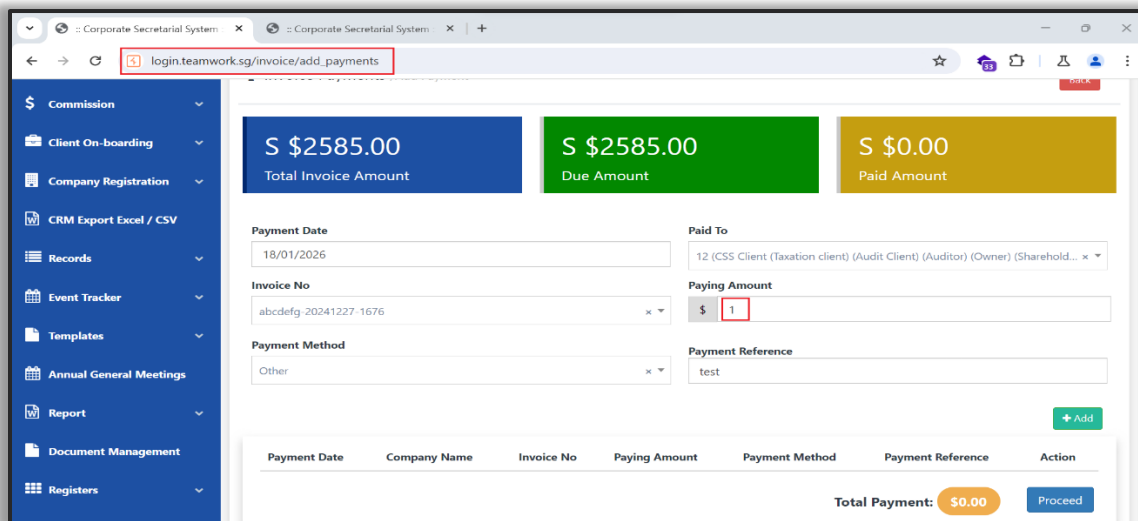


FIGURE 44: SESSION REMAINS ACTIVE AFTER CLOSING AND REOPENING THE BROWSER WINDOW 04



3.1.9 Business Logic Flaw

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A05:2021 – Security Misconfiguration
Vulnerability Definition	A business logic flaw occurs when an application fails to enforce intended workflows or rules, allowing attackers to manipulate the system in ways that break its normal business processes.
Vulnerability Description	During the assessment, it was observed that the application accepted negative values on fixed deposit income. By intercepting the request in Burp Suite and changing the income to \$-1, the response reflected as success, confirming that fixed deposit income and other financial rules are not validated on the server.
Root Cause	The application does not have server-side validation to enforce business rules for deposit income. It relies solely on client-side checks, which can easily be bypassed using tools like Burp Suite.
Affected URL	• https://login.teamwork.sg/invoice/add_payments • https://login.teamwork.sg/invoice/proceed_payments • https://login.teamwork.sg/invoice/
Impact	If exploited, attackers could manipulate FD income -related data and submit invalid or negative values. This could lead to incorrect financial calculations, data integrity issues, and potential financial loss for the organization.
References	• https://owasp.org/www-community/vulnerabilities/Business_logic_vulnerability • https://owasp.org/www-project-top-10-for-business-logic-abuse/
Recommendation	The application should enforce strict server-side validation to ensure only valid and logical values are accepted. Negative or unrealistic values must be rejected, and all business rules should be applied consistently on both the client and server side.
Proof of Concept	



Payment Date	Company Name	Invoice No	Paying Amount	Payment Method	Payment Reference	Action
18/01/2026		abcdefg-20241227-1676	\$ 1	Other	test	

Total Payment: \$0.00

FIGURE 45 – ORIGINAL REQUEST

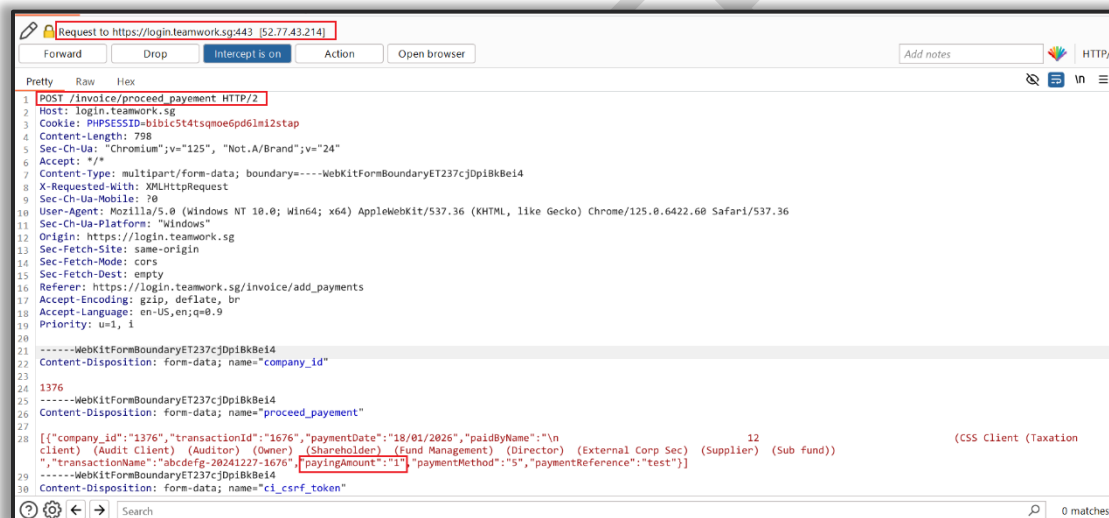


FIGURE 46 – INTERCEPTING THE REQUEST WITH BURPSUITE.

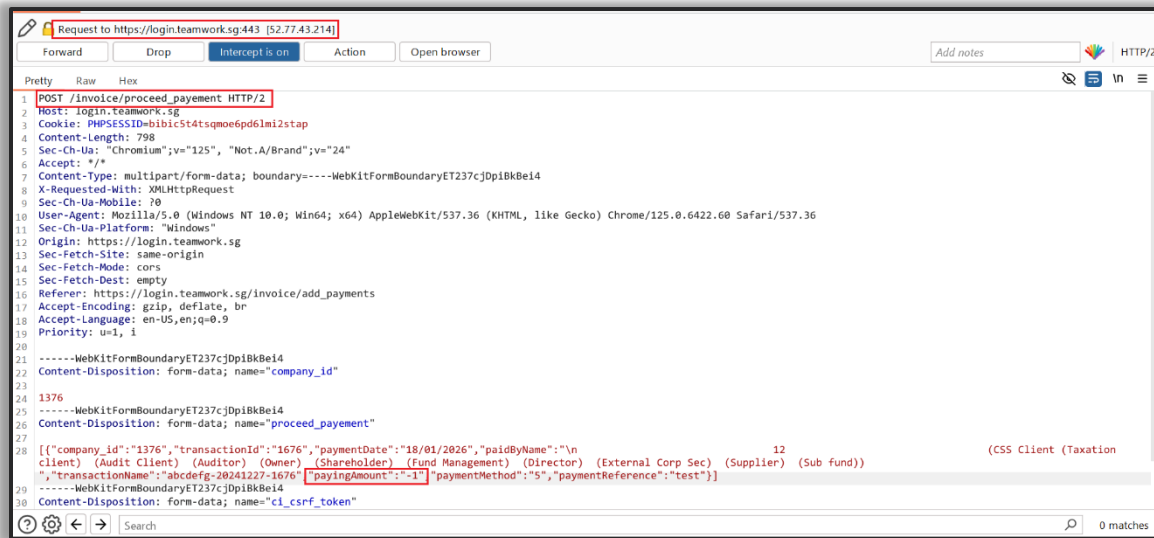
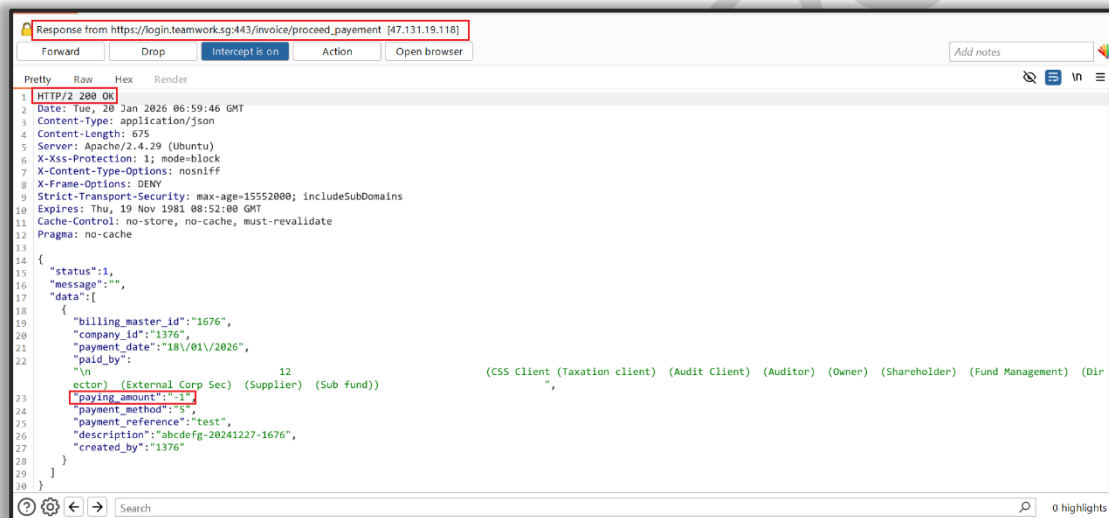
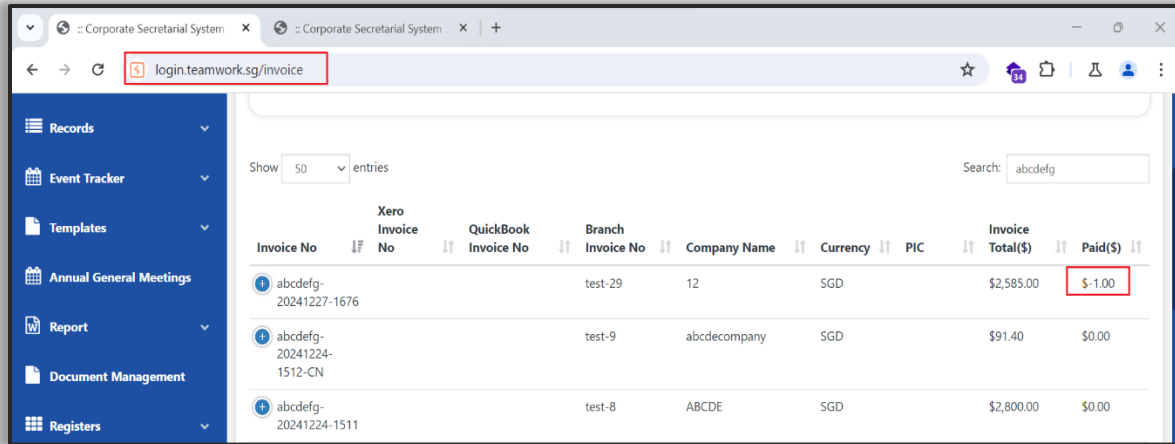


FIGURE 47 – MODIFIED PAYING AMOUNT IN MINUS 01



FIGURE

48- MODIFIED PAYING AMOUNT IN MINUS 02



login.teamwork.sg/invoice

Show 50 entries Search: abcdefg

Invoice No	Xero Invoice No	QuickBook Invoice No	Branch Invoice No	Company Name	Currency	PIC	Invoice Total(\$)	Paid(\$)
abcdefg-20241227-1676			test-29	12	SGD		\$2,585.00	\$-1.00
abcdefg-20241224-1512-CN			test-9	abcdec company	SGD		\$91.40	\$0.00
abcdefg-20241224-1511			test-8	ABCDE	SGD		\$2,800.00	\$0.00

FIGURE 49 – SUCCESSFULLY ACCEPTANCE OF PAYING AMOUNT IN MINUS



3.1.10 Password Disclosure in Response

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A02:2021 – Cryptographic Failures
Vulnerability Definition	Password disclosure in response occurs when an application returns user passwords or sensitive credential information directly in server responses.
Vulnerability Description	During the assessment, we observed that the application returns the user's password in the server response during certain requests (such as login, profile, or API responses). Exposing passwords in responses is a critical security issue, as sensitive credentials should never be transmitted or displayed in plain text.
Root Cause	Improper handling of sensitive data and lack of secure credential storage and response filtering.
Affected URL	• https://login.teamwork.sg/edit_admin/382 • https://login.teamwork.sg/my_profile
Impact	An attacker can easily obtain user passwords by intercepting responses, leading to account compromise, unauthorized access, identity theft, and potential reuse of credentials across other services.
References	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
Recommendation	• Never return passwords or sensitive credentials in any response • Store passwords using strong one-way hashing algorithms (e.g., bcrypt, Argon2) • Mask or remove sensitive fields from API responses • Conduct security reviews to ensure sensitive data is not exposed
Proof of Concept	

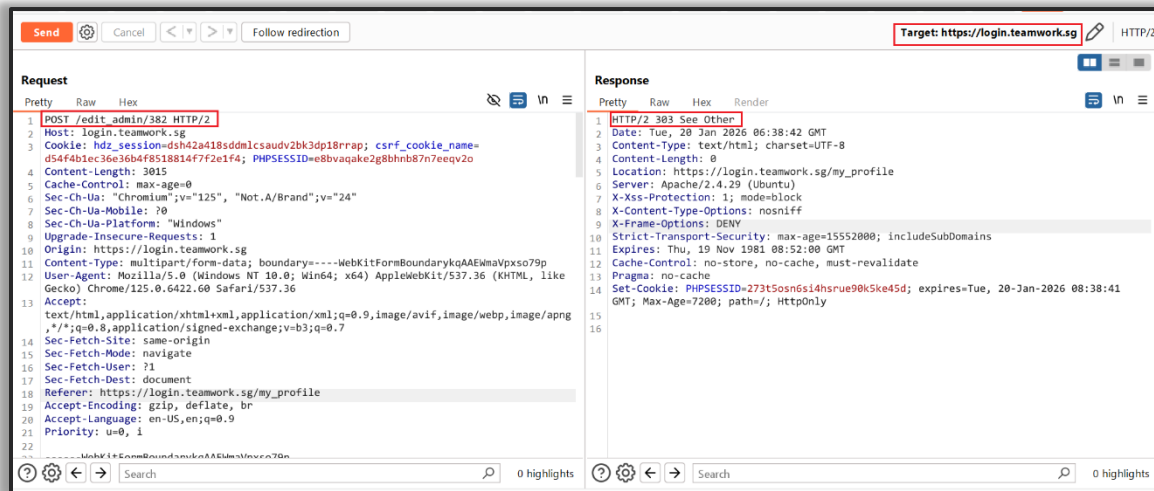


FIGURE 50: ORIGINAL REQUEST AND RESPONSE

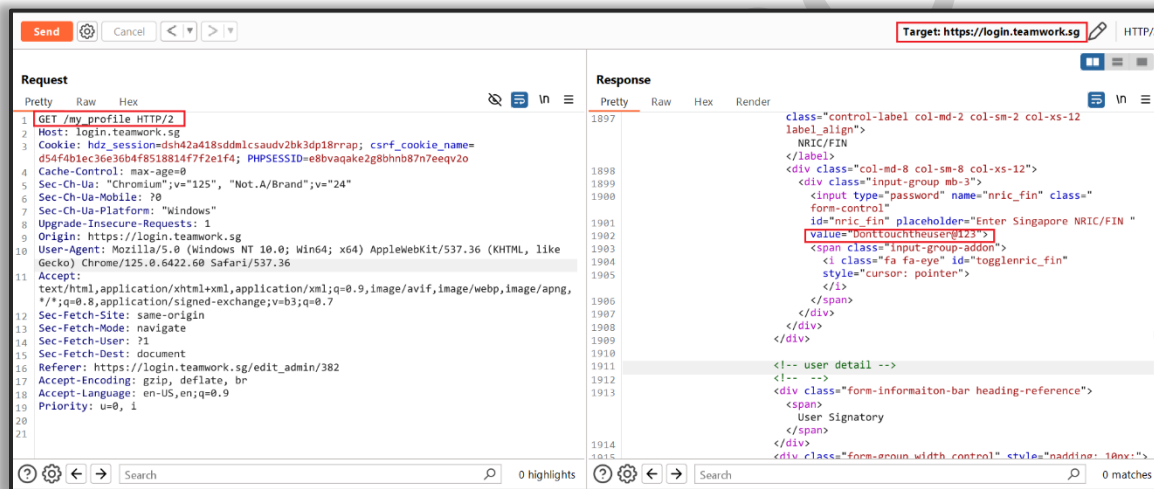


FIGURE 51: PASSWORD DISCLOSED IN REPONSE



3.1.11 Weak Password Policy

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A02:2021 – Cryptographic Failures
Vulnerability Definition	A weak password policy is a security issue where the application allows users to create passwords that are easy to guess or crack due to insufficient complexity requirements
Vulnerability Description	During the assessment, we observed that the application enforces a weak password policy. The application allows users to set passwords with minimal complexity, that is application accepting the guessable passwords like Admin@123, Password@123. This makes user accounts more susceptible to brute-force and credential-guessing attacks.
Root Cause	Insufficient password complexity and strength requirements enforced by the application.
Affected URL	https://login.teamwork.sg/welcome/index?accessToken=
Impact	Weak passwords can be easily guessed or cracked, leading to account compromise, unauthorized access, data exposure, and potential misuse of the application.
References	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html#password-security
Recommendation	• Avoid default and guessable passwords. • Enforce strong password requirements (minimum length and complexity) • Require a mix of uppercase, lowercase, numbers, and special characters • Prevent reuse of previous passwords • Implement account lockout or rate limiting for repeated failed attempts • Encourage the use of password managers
Proof of Concept	

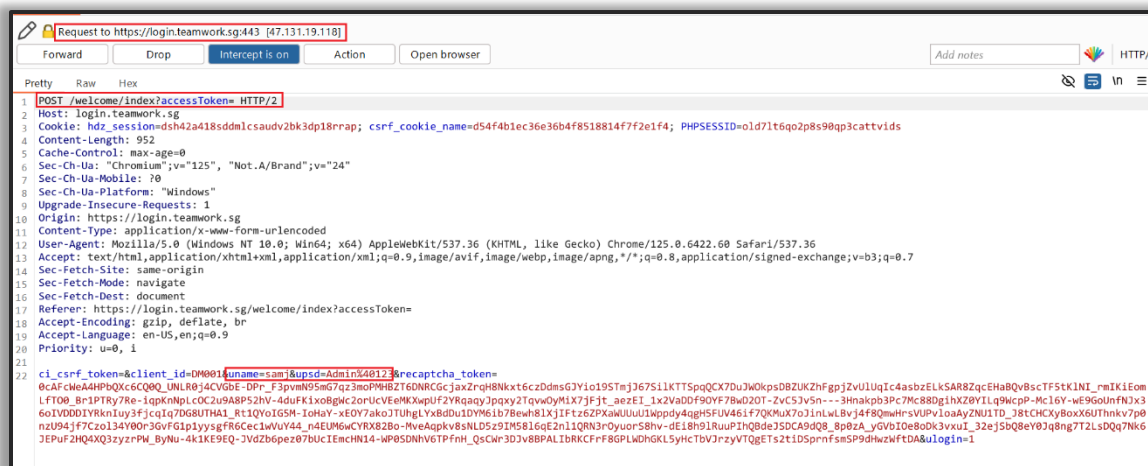


FIGURE 52- ALLOWS WEAK PASSWORD 01

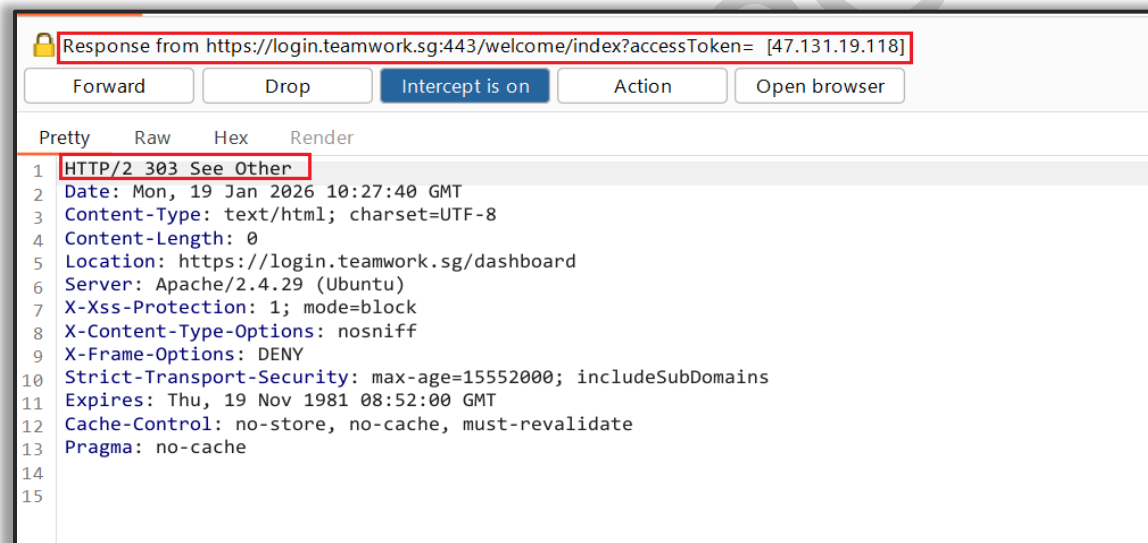


FIGURE 53- ALLOWS WEAK PASSWORD 02



3.1.12 User Enumeration

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A07:2021 – Identification and Authentication Failures
Vulnerability Definition	User Enumeration is a vulnerability where an application reveals whether a username, email, or account exists through different responses or behaviors.
Vulnerability Description	During the assessment, we observed that the application provides different responses when valid and invalid usernames are submitted during authentication or account-related functions. These differences allow an attacker to identify which user accounts exist in the application, confirming a user enumeration issue.
Root Cause	Different error messages or response behaviors for valid and invalid users.
Affected URL	https://login.teamwork.sg
Impact	An attacker can identify valid user accounts, which can be leveraged for targeted brute-force attacks, credential stuffing, phishing, or social engineering attacks.
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/03-Identity_Management_Testing/04-Testing_for_Account_Enumeration_and_Guessable_User_Account
Recommendation	• Use generic and consistent error messages for authentication and account-related actions • Return the same HTTP status codes and response times for valid and invalid users • Implement rate limiting and monitoring on authentication endpoints • Avoid revealing user existence in password reset or registration flows
Proof of Concept	

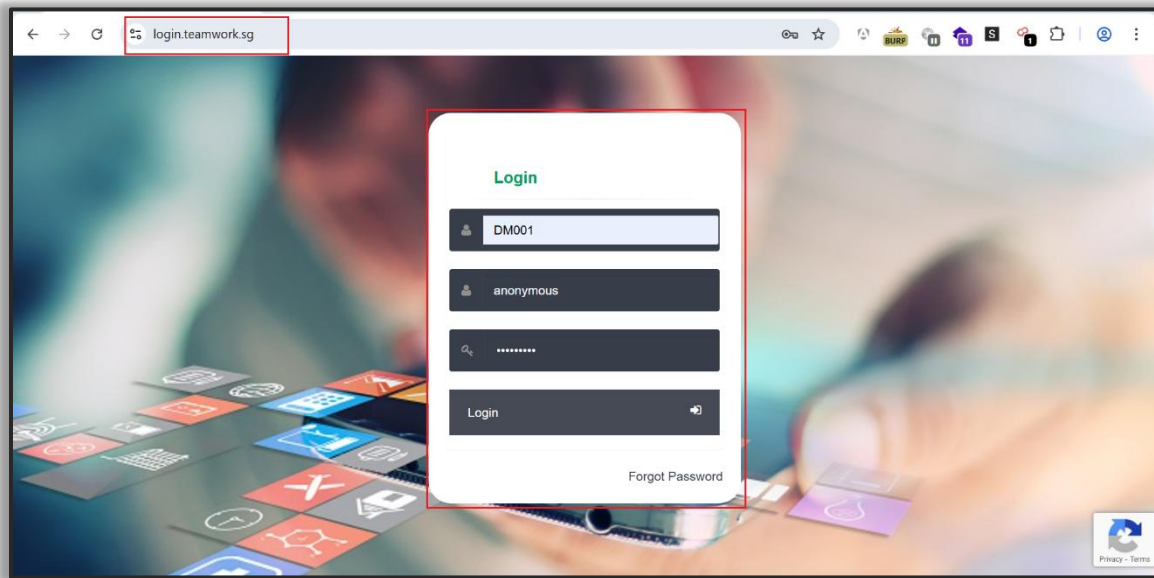


FIGURE 54 - USER ENUMERATION 01

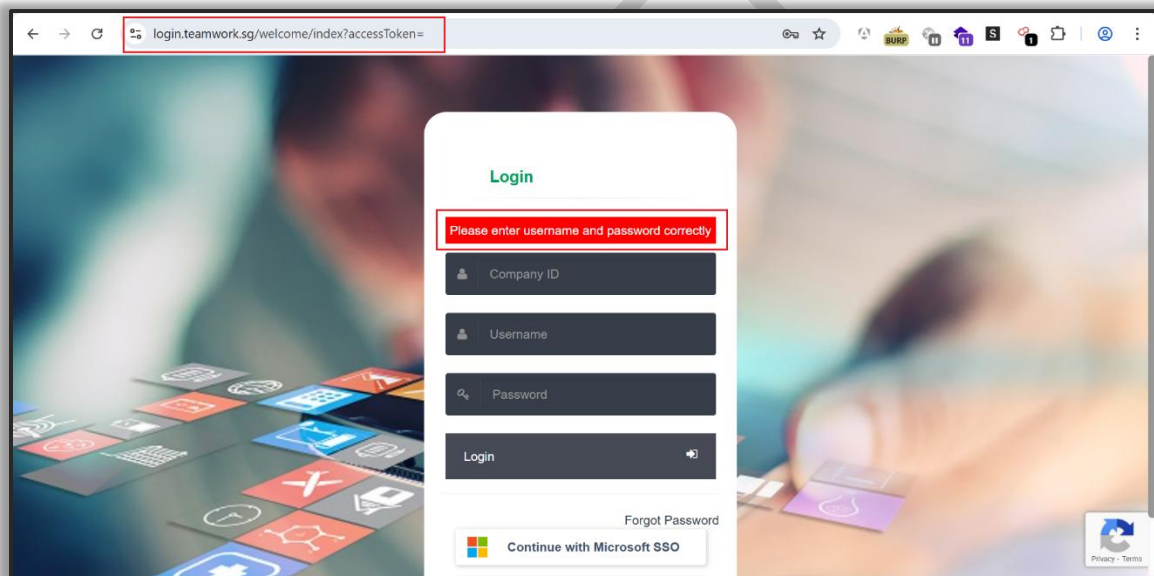


FIGURE 55 - USER ENUMERATION 02

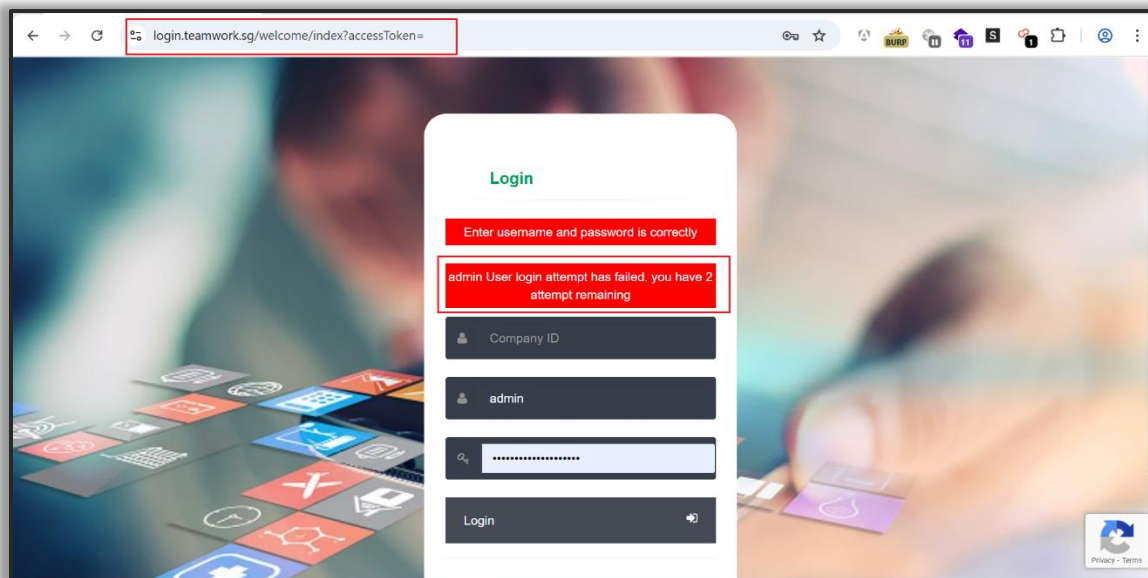


FIGURE 56 - USER ENUMERATION 03



3.1.13 Session Fixation

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A07:2021 – Identification and Authentication Failures
Vulnerability Definition	Session Fixation is a vulnerability where an application does not issue a new session identifier after user authentication, allowing an attacker to reuse a known session ID.
Vulnerability Description	During the assessment, we observed that the application does not regenerate the session ID after successful login. A session identifier provided before authentication remained valid even after the user logged in. This behavior allows an attacker to fix a session ID and potentially hijack an authenticated user session.
Root Cause	Session identifiers are not regenerated or invalidated after authentication.
Affected URL	• https://login.teamwork.sg/ • https://login.teamwork.sg/dashboard • https://login.teamwork.sg/mainadmin/logout
Impact	An attacker can hijack a victim's authenticated session, gain unauthorized access to user accounts, and perform actions on behalf of the victim.
References	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
Recommendation	• Regenerate session IDs immediately after successful login • Invalidate existing sessions on authentication and logout • Set secure cookie attributes (HttpOnly, Secure, SameSite) • Use strong, random, server-generated session identifiers
Proof of Concept	

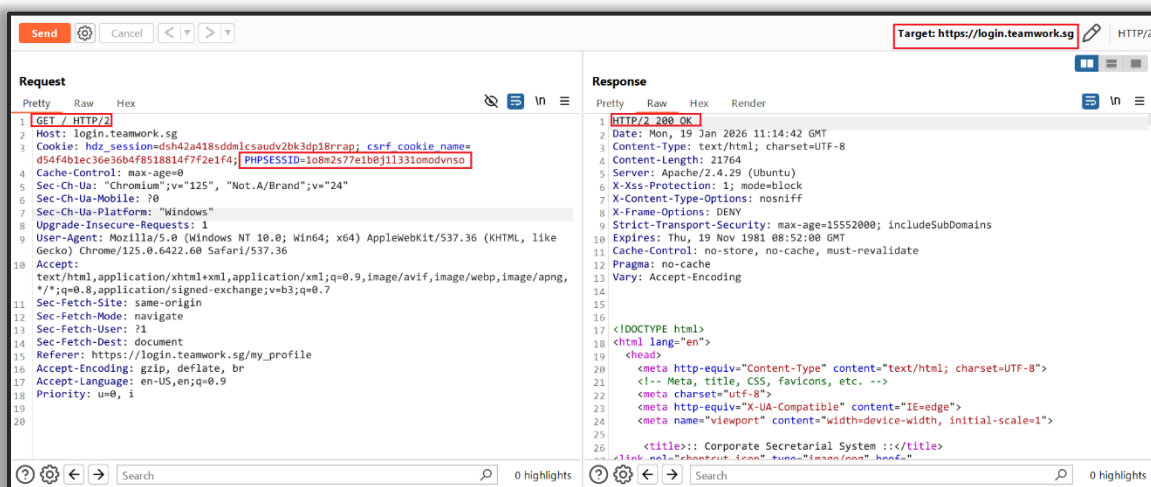


FIGURE 57 - PHPSESSIONID BEFORE AUTHENTICATION

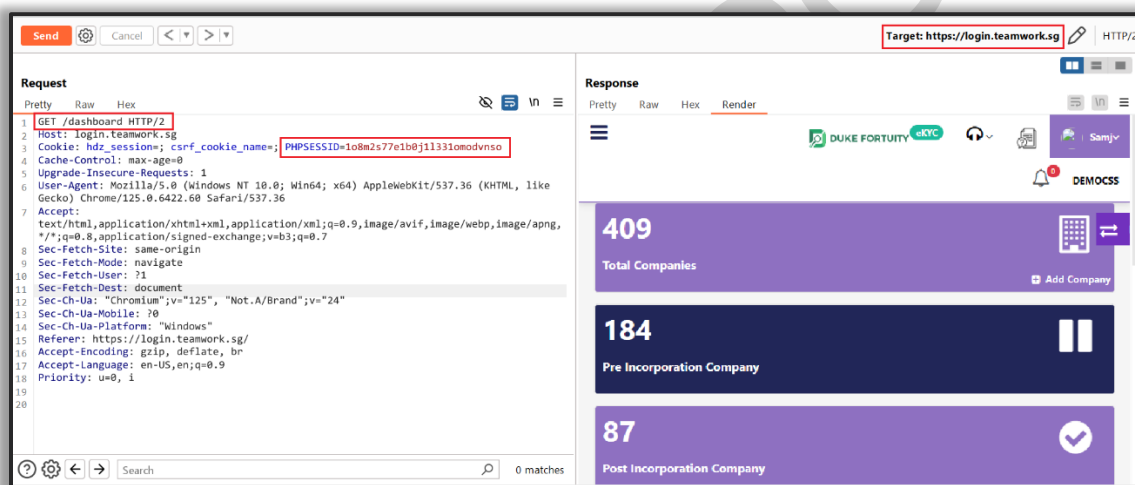


FIGURE 58- PHPSESSIONID REMAINS UNCHANGED AFTER AUTHENTICATION

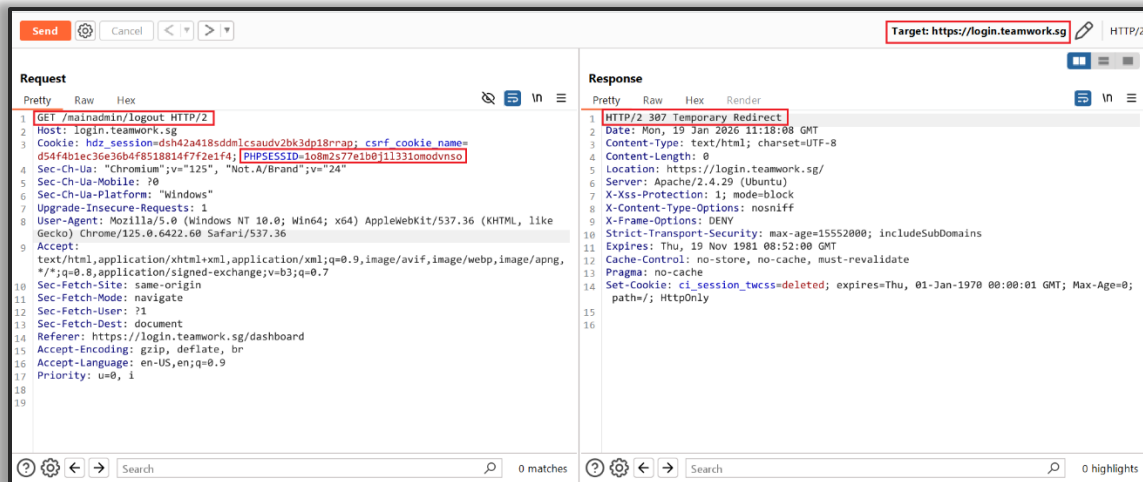


FIGURE 59 - THE SAME PHPSESSIONID ID PERSISTS EVEN AFTER USER LOGOUT

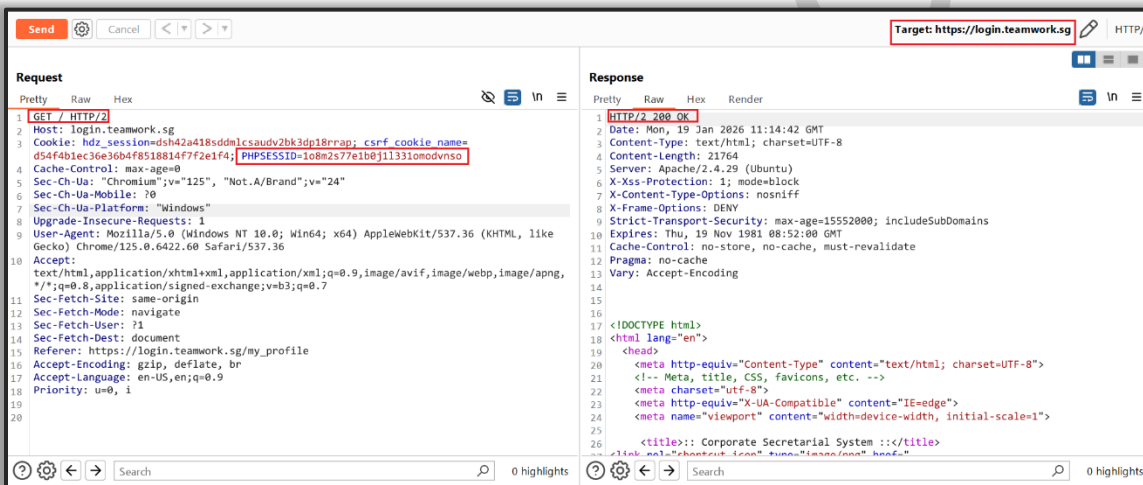


FIGURE 60 - THE APPLICATION RESUES THE SAME PHPSESSIONID UPON RE-LOGIN.



3.1.14 Improper OTP Configuration

Parameters	Description
Vulnerability Severity	Medium
OWASP Rating	A07:2021 – Identification and Authentication Failures
Vulnerability Definition	Weak OTP Length is a vulnerability where the application uses an OTP that is too short, making it easier to guess or brute-force.
Vulnerability Description	During the assessment, we observed that the application uses a 4-digit OTP for authentication or verification. Due to the limited number of possible combinations, this OTP length provides insufficient security and can be easily brute forced, especially if proper rate limiting is not enforced.
Root Cause	Use of a short OTP length with insufficient entropy for authentication.
Affected URL	https://login.teamwork.sg/welcome/index?accessToken=
Impact	An attacker can brute force the OTP, bypass authentication mechanisms, gain unauthorized access to user accounts, and potentially compromise sensitive data.
References	https://medium.com/@sammyasopa/building-a-secure-otp-based-user-authentication-system-f06102afef8d
Recommendation	• Increase OTP length to at least 6 digits • Implement rate limiting and attempt restrictions on OTP verification • Set OTP expiration time (short validity window) • Lock or temporarily block verification after multiple failed attempts
Proof of Concept	

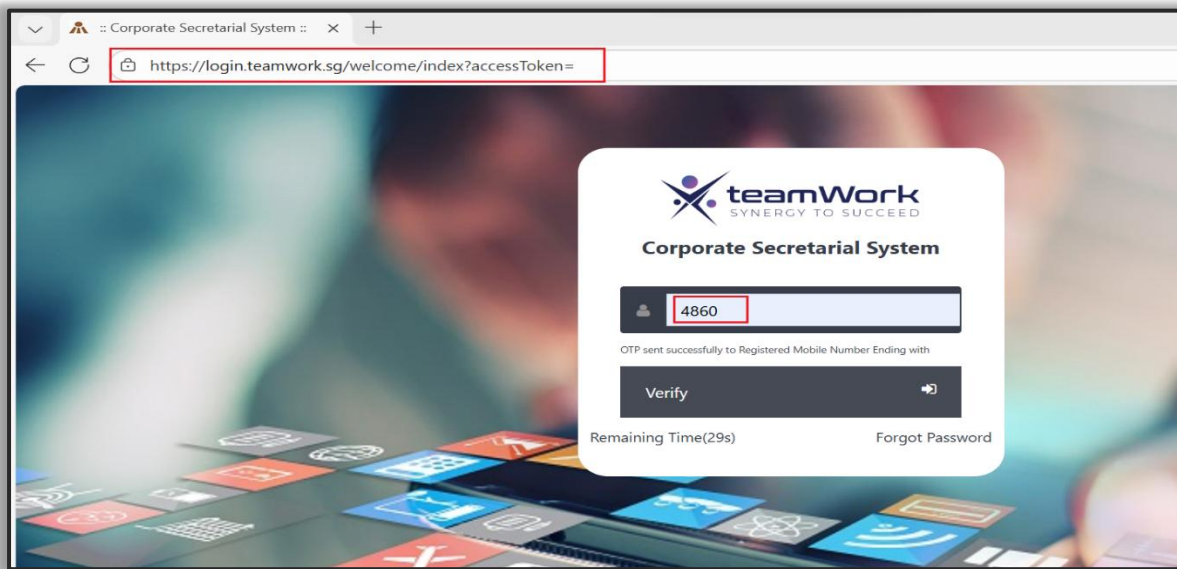


FIGURE 61: WEAK OTP LENGTH CONFIGURATION.



3.1.15 Missing Autocomplete

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Missing Autocomplete is a security issue where sensitive input fields do not properly disable browser autocomplete, allowing credentials or personal data to be stored and reused by the browser.
Vulnerability Description	During the assessment, we observed that sensitive input fields such as username, password, and other personal data fields do not have the autocomplete attribute disabled. Due to this, the browser may store and automatically fill sensitive information, which can be accessed by unauthorized users on shared or compromised systems.
Root Cause	The autocomplete attribute is not properly configured for sensitive input fields.
Affected URL	https://login.teamwork.sg
Impact	An attacker or unauthorized user with access to the same browser or device may retrieve stored credentials or sensitive data, leading to account compromise or information leakage.
References	https://portswigger.net/kb/issues/00500800_password-field-with-autocomplete-enabled
Recommendation	• Disable autocomplete for sensitive fields using: • <code>autocomplete="off"</code> or appropriate values like <code>autocomplete="new password"</code> • Apply secure handling for authentication and PII input fields • Follow browser and OWASP recommendations for form security
Proof of Concept	

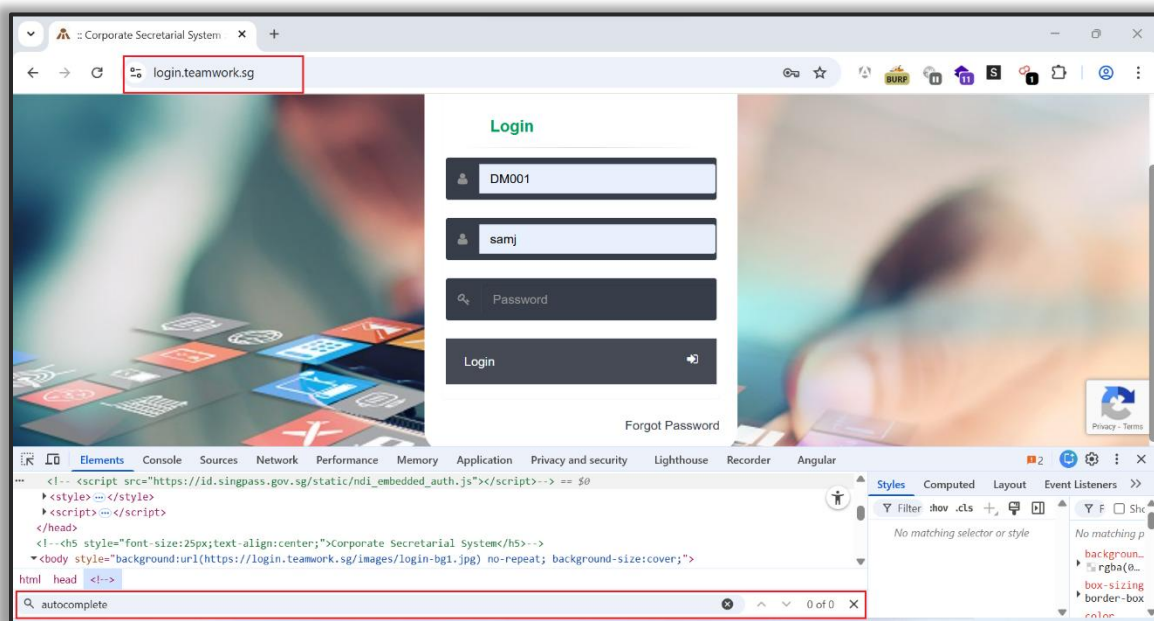


FIGURE 62: AUTOCOMPLETE IS NOT PRESENT



3.1.16 Server Name and Version Disclosure

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Server Name and Version Disclosure occurs when an application reveals details about the underlying web server software and its version through HTTP response headers or error messages.
Vulnerability Description	During the assessment, we observed that the application exposes server information such as the server's name and version in HTTP response. Disclosing this information provides attackers with insights into the technology stack used, which can help them target known vulnerabilities specific to the disclosed server version.
Root Cause	Default or misconfigured server settings expose server identification headers.
Affected URL	• https://login.teamwork.sg/ • https://login.teamwork.sg/files • https://login.teamwork.sg/crm/checknew_notificationcount
Impact	An attacker can use the disclosed server name and version to fingerprint the application, identify known vulnerabilities, and plan targeted attacks against the server.
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server
Recommendation	• Remove or obfuscate server identification headers • Disable unnecessary response headers such as Server and X-Powered-By • Keep server software and dependencies up to date • Apply secure server configuration best practices
Proof of Concept	

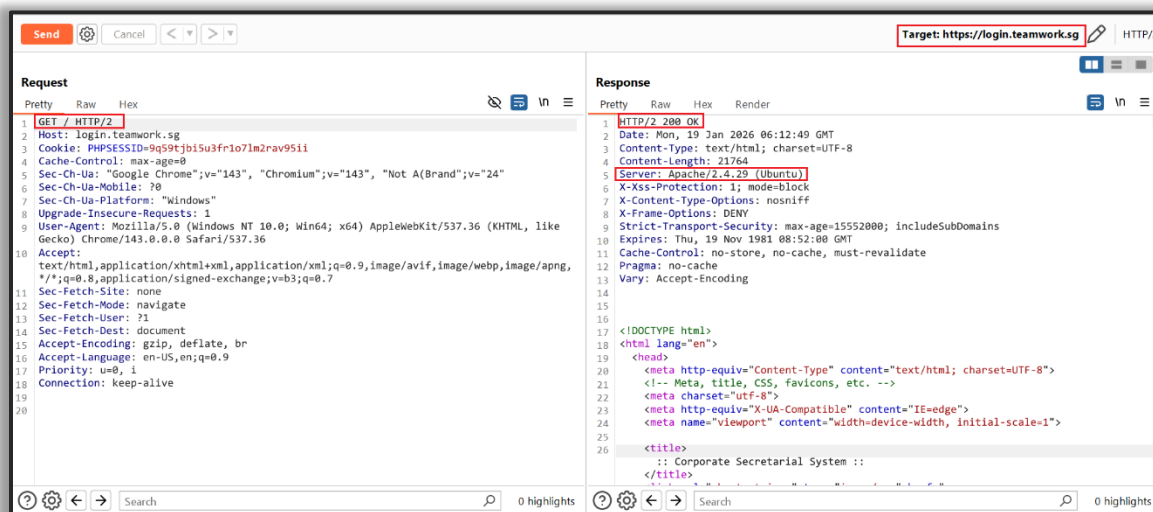


FIGURE 63: SERVER NAME AND VERSION DISCLOSURE 01

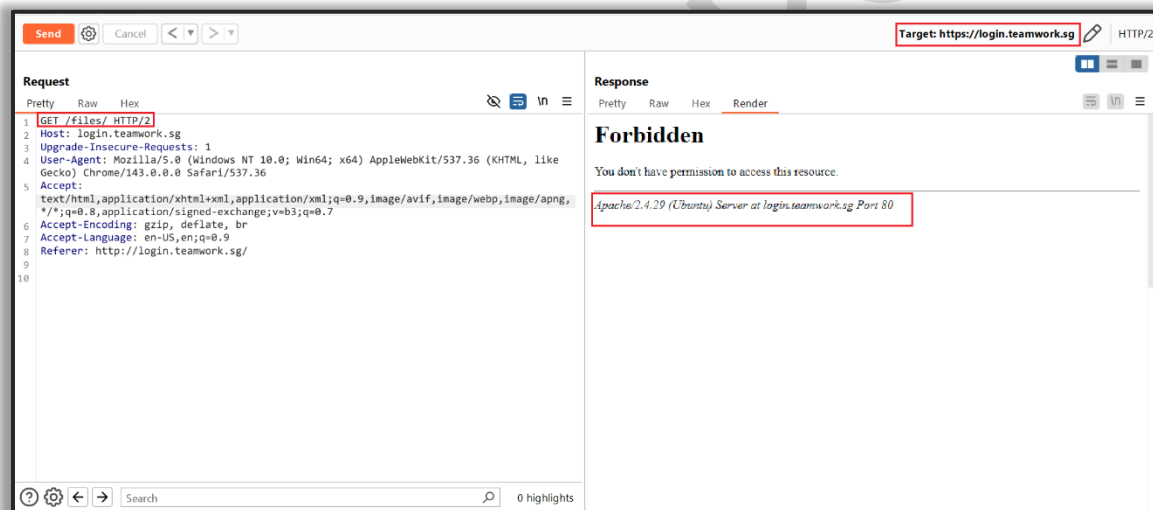


FIGURE 64: SERVER NAME AND VERSION DISCLOSURE 02

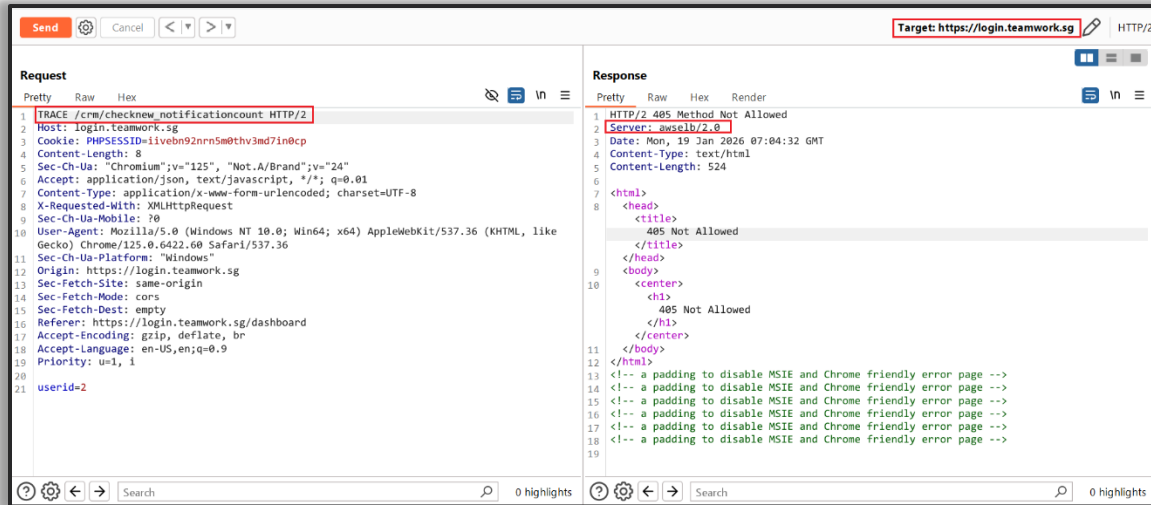


FIGURE 65: SERVER NAME AND VERSION DISCLOSURE 03



3.1.17 Improper Error Handling

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Improper Error Handling occurs when an application displays detailed system or application error messages to users, revealing internal implementation details.
Vulnerability Description	During the assessment, we observed that the application returns verbose error messages when invalid input or unexpected conditions occur. These error responses disclose internal details such as application logic, database information, or stack traces, which should not be visible to end users.
Root Cause	Lack of centralized and secure error handling mechanisms.
Affected URL	https://login.teamwork.sg/edit_admin/382
Impact	An attacker can use the exposed error information to understand the application's internal structure, making it easier to identify vulnerabilities, craft targeted attacks, or bypass security controls
References	https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html
Recommendation	• Display generic error messages to users • Log detailed errors securely on the server side • Disable debug and verbose error modes in production • Implement centralized exception handling
Proof of Concept	

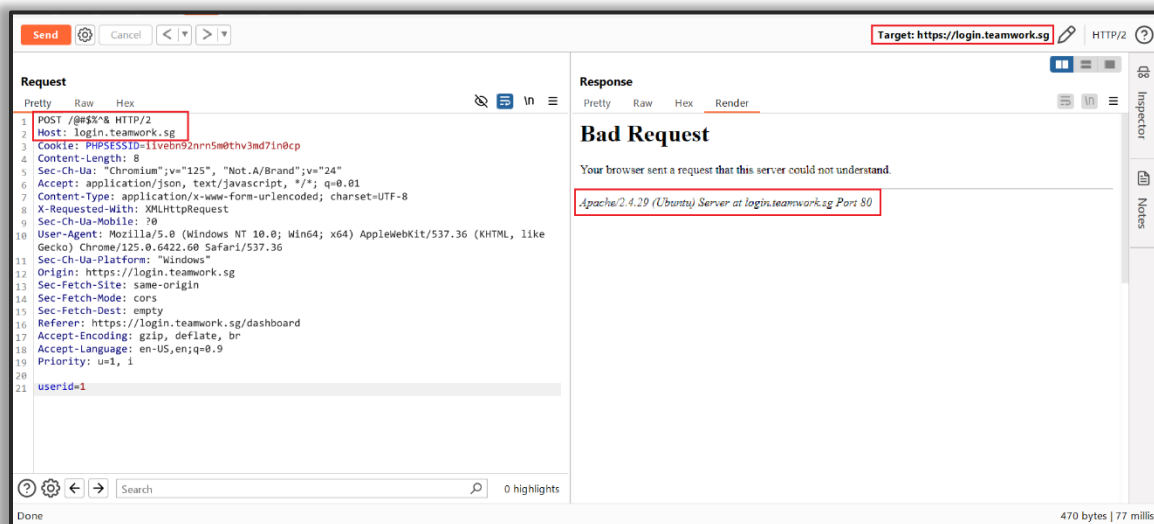


FIGURE 66: IMPROPER ERROR HANDLING, SERVER NAME AND VERSION DISCLOSURE

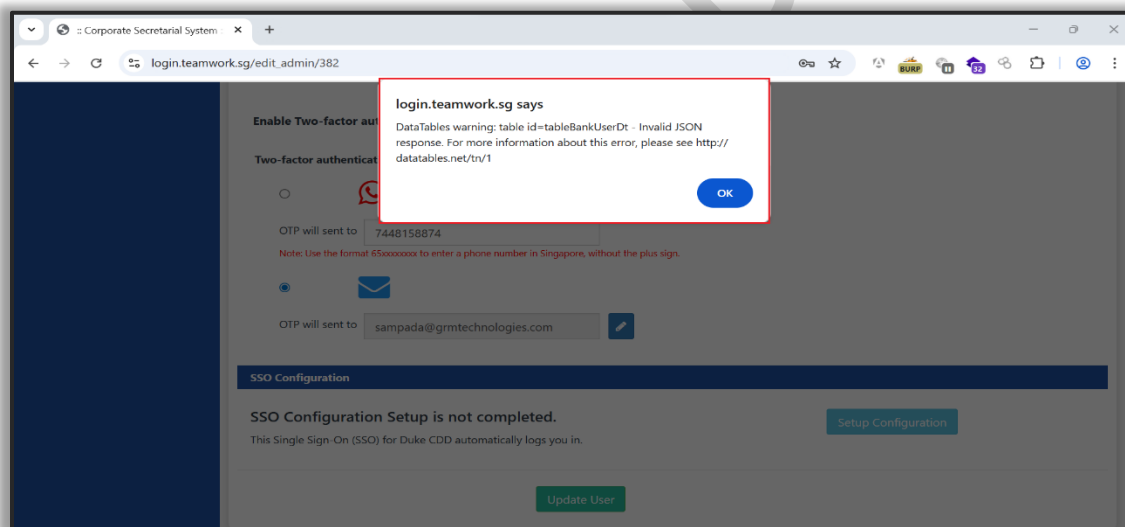


FIGURE 67: THE APPLICATION REVEALS DETAILED ERROR MESSAGES



3.1.18 Clear Text Submission of Credentials

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A02:2021 – Cryptographic Failures
Vulnerability Definition	Clear text submission of credentials occurs when an application transmits sensitive information such as usernames and passwords without encryption, allowing attackers to intercept and read them.
Vulnerability Description	During the assessment, we observed that the application submits the username and password in clear text during the login process. The credentials were transmitted without proper encryption, making them visible in the request. This indicates that secure communication mechanisms such as HTTPS/TLS are not properly enforced for authentication requests.
Root Cause	Lack of enforced HTTPS/TLS encryption for transmitting sensitive authentication data.
Affected URL	https://login.teamwork.sg/welcome/index?accessToken=
Impact	An attacker can intercept login credentials through network sniffing or man-in-the-middle attacks, potentially leading to account compromise, unauthorized access, and further exploitation of the application.
References	https://owasp.org/www-community/vulnerabilities/Password_Plaintext_Storage
Recommendation	• Enforce HTTPS (TLS) across the entire application • Redirect all HTTP traffic to HTTPS • Use secure cookies with Secure and HttpOnly flags • Implement HSTS (HTTP Strict Transport Security) • Avoid transmitting credentials in URLs or plain text requests
Proof of Concept	

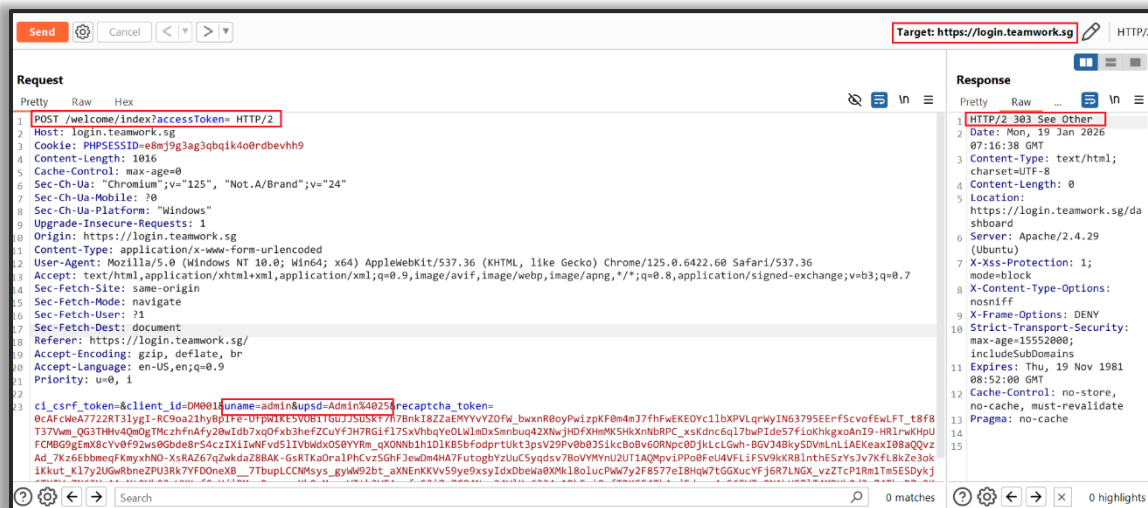


FIGURE 68 – USER CREDENTIALS ARE SUBMITTED IN CLEARTEXT



3.1.19 Missing Security Headers

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Missing Security Headers is a vulnerability where an application does not include recommended HTTP security headers, reducing protection against common web attacks.
Vulnerability Description	During the assessment, we observed that the application responses do not include important HTTP security headers such as Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy. The absence of these headers weakens the application's defense against attacks like XSS, clickjacking, and data leakage.
Root Cause	Security-related HTTP response headers are not properly configured on the server.
Affected URL	https://login.teamwork.sg/
Impact	An attacker may exploit the lack of security headers to perform client-side attacks such as cross-site scripting, clickjacking, MIME-type sniffing, or information leakage, increasing the overall attack surface of the application.
References	https://owasp.org/www-project-secure-headers/
Recommendation	Implement recommended security headers: • Content-Security-Policy (CSP) • X-Frame-Options • X-Content-Type-Options • Referrer-Policy • Strict-Transport-Security (HSTS) Regularly review and test header configurations Use secure defaults provided by modern frameworks or web servers
Proof of Concept	

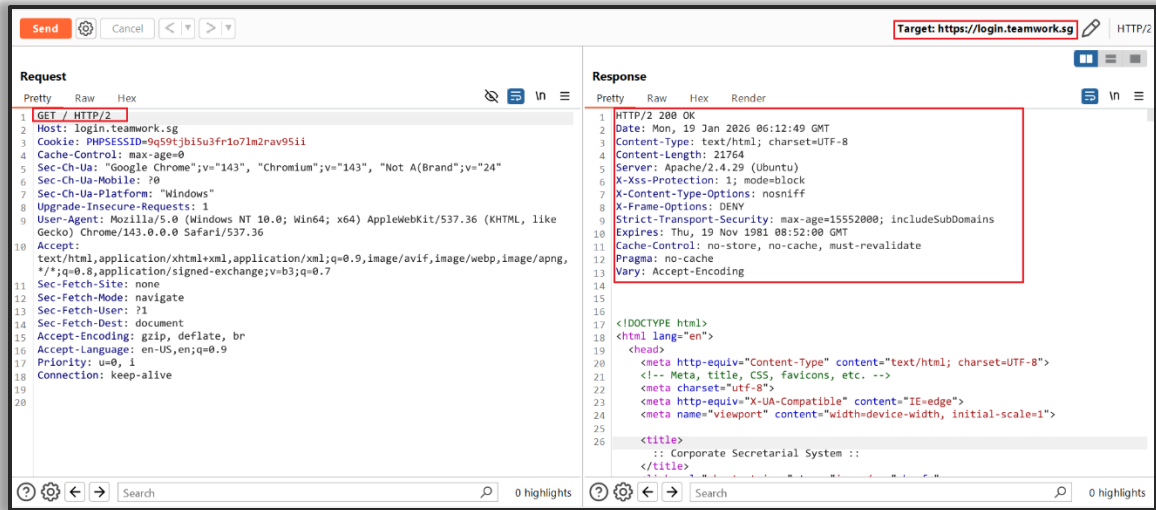


FIGURE 69: MISSING SECURITY HEADERS 01

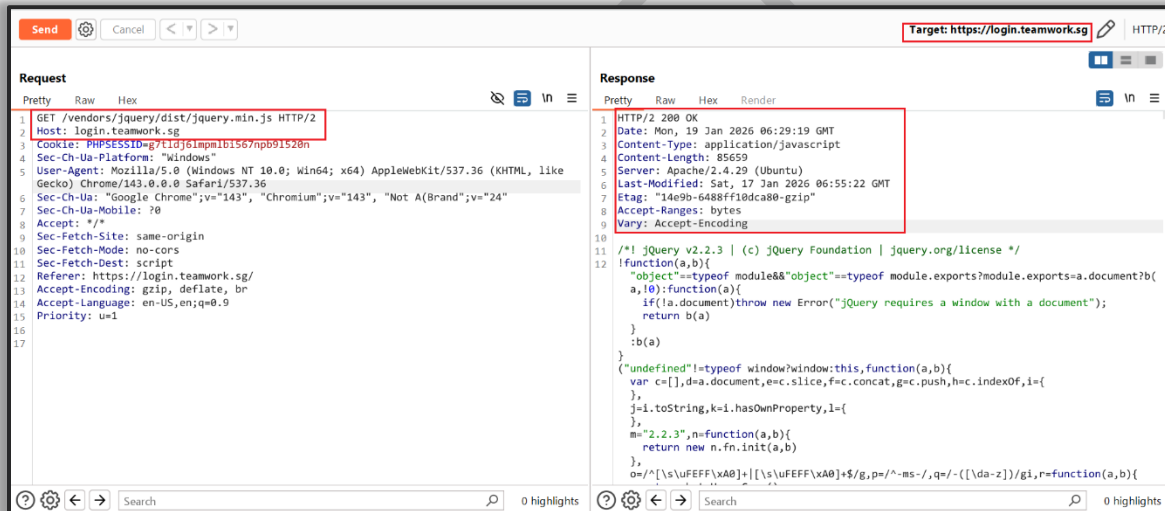


FIGURE 70: MISSING SECURITY HEADERS 01



3.1.20 Clickjacking Attack

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Clickjacking Attack: Clickjacking is an attack that tricks a user into clicking a webpage element which is invisible or disguised as another element. This can cause users to unwittingly download malware, visit malicious web pages, provide credentials or sensitive information, transfer money, or purchase products online.
Vulnerability Description	During the assessment, we discovered that our application is vulnerable to clickjacking attack.
Root Cause	Security Misconfiguration
Affected URL	https://login.teamwork.sg/help/
Impact	An attacker may also choose to redirect the clicks to download malware or gain access to vital systems as a starting point for an advanced persistent threat (APT). This spells trouble for any organizations that rely on protecting sensitive data and intellectual property.
References	https://owasp.org/www-community/attacks/Clickjacking
Recommendation	It is recommended to prevent clickjacking attacks is to ask the browser to block any attempt to load your website within an iframe. You can do it by sending the X-Frame-Options HTTP header.
Proof of Concept	

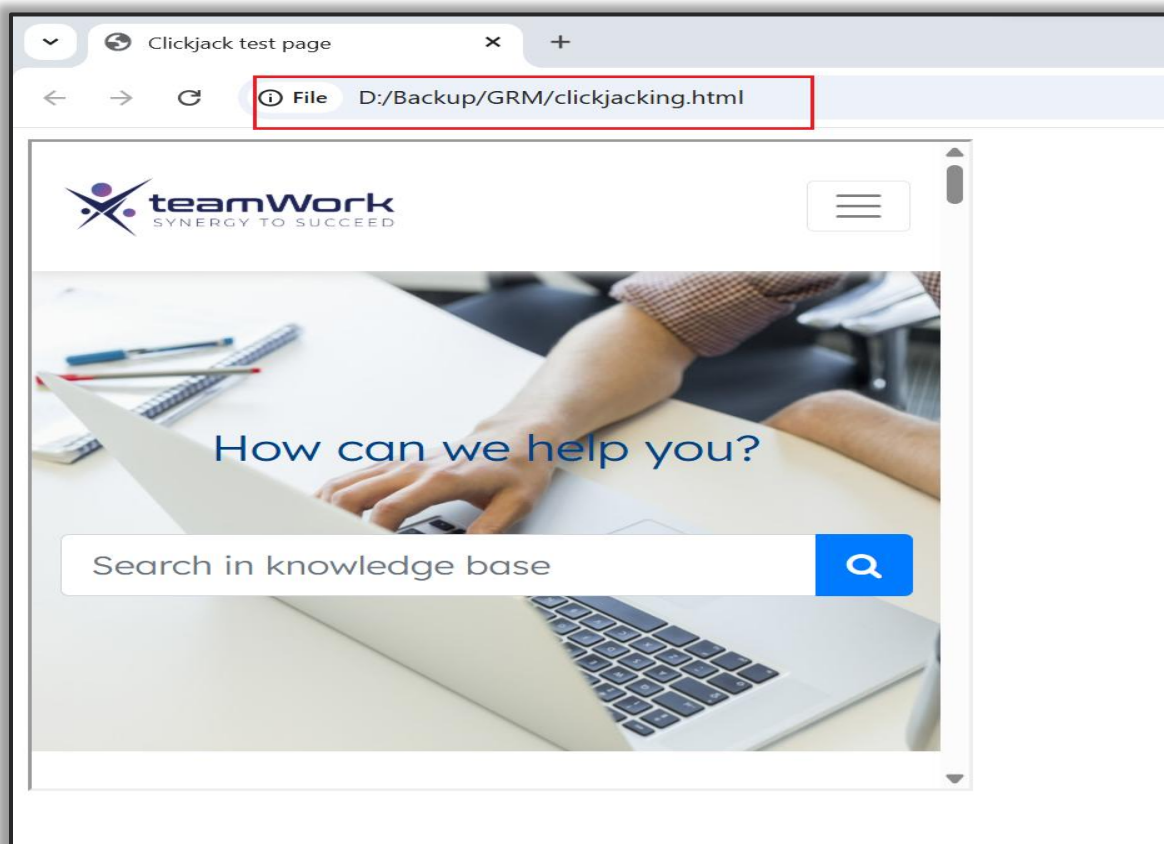


FIGURE 71: CLICKJACKING



3.1.21 Improper Input Validation

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Improper Input Validation is a vulnerability where an application does not correctly validate, sanitize, or restrict user-supplied input before processing it.
Vulnerability Description	During the assessment, we observed that the application accepts user input without proper validation or sanitization. Malformed or unexpected input was processed by the application, indicating that input validation controls are insufficient. This behavior may allow attackers to inject malicious input or manipulate application functionality.
Root Cause	Missing or weak validation of user-supplied input on the server side.
Affected URL	- https://login.teamwork.sg/add_company - https://login.teamwork.sg/crm/insert_bank_of_user
Impact	Improper input validation can lead to multiple security issues such as SQL Injection, XSS, HTML Injection, command injection, application errors, and data manipulation.
References	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
Recommendation	- Implement strict server-side input validation - Use allow-lists (whitelisting) for expected input formats - Apply proper encoding and sanitization based on context - Validate input length, type, and format - Use secure frameworks and built-in validation mechanisms
Proof of Concept	

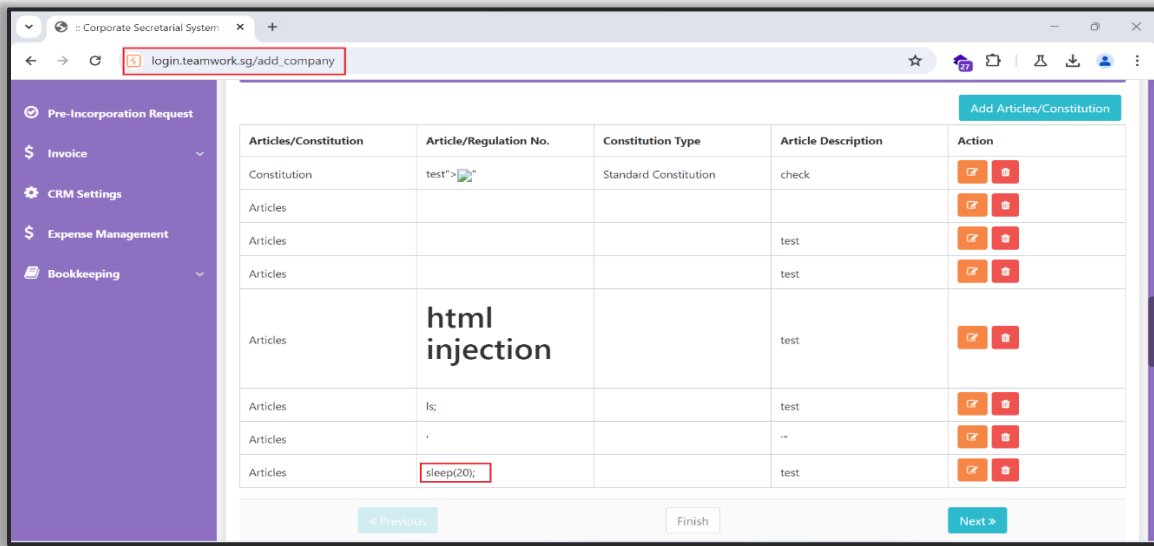


FIGURE 72: IMPROPER INPUT VALIDATION 01

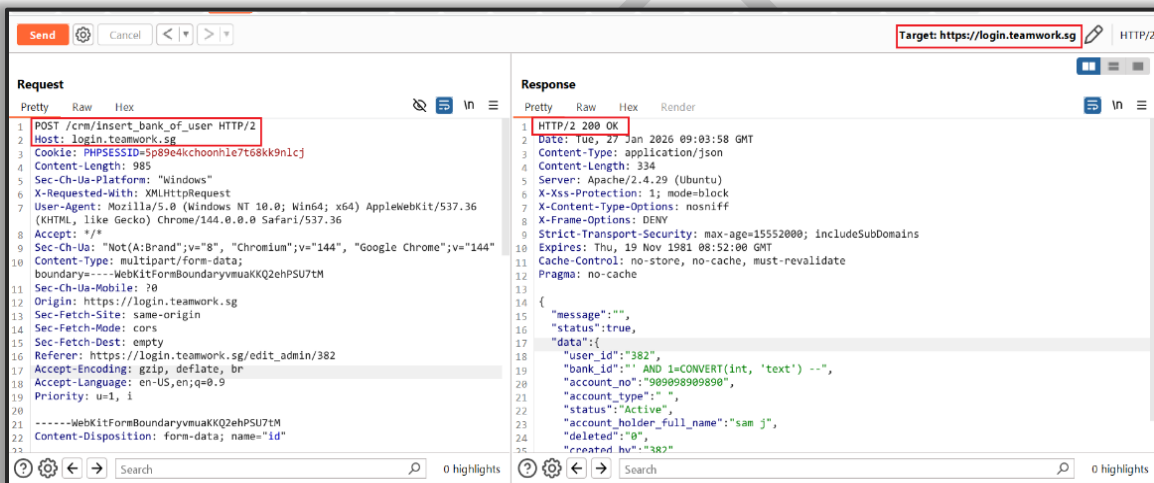


FIGURE 73: IMPROPER INPUT VALIDATION 02

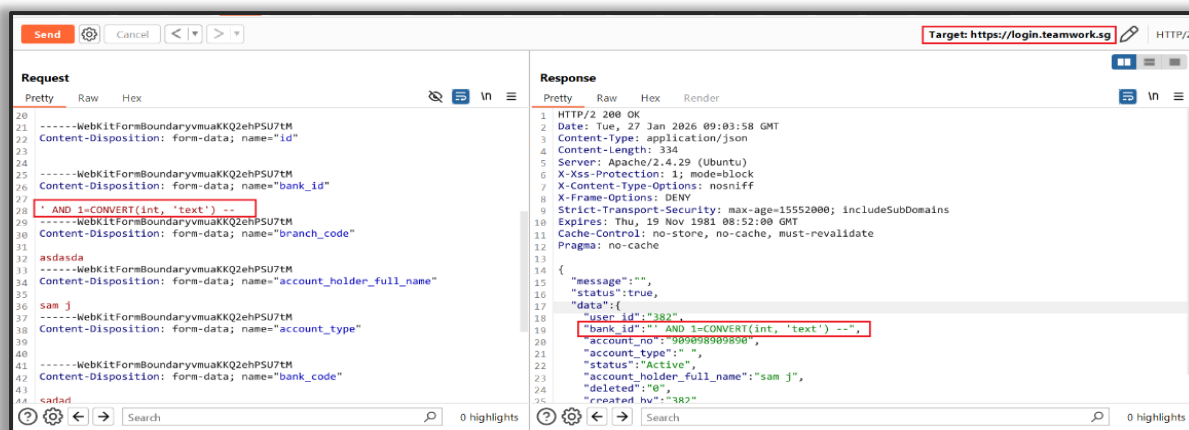


FIGURE 74: IMPROPER INPUT VALIDATION 03



3.1.22 Vulnerable and Outdated Component

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A06:2021 – Vulnerable and Outdated Components
Vulnerability Definition	Vulnerable and Outdated Components is a security issue where an application uses software libraries, frameworks, or components with known security vulnerabilities or unsupported versions.
Vulnerability Description	During the assessment, we observed that the application uses outdated or vulnerable third-party components. These components are running on older versions with publicly known security issues. Using such components increases the risk of exploitation, as attackers can leverage known vulnerabilities to compromise the application.
Root Cause	Lack of regular patching, updates, and dependency management for third-party components.
Affected URL	• https://login.teamwork.sg/ • https://login.teamwork.sg/help/assets/components/jquery/jquery.min.js • https://login.teamwork.sg/help/assets/components/select2/js/select2.min.js • https://login.teamwork.sg/js/datatables_export_button/js/jquery.dataTables.min.js • https://login.teamwork.sg/js/datatables_export_button/js/pdfmake.min.js • https://login.teamwork.sg/js/jquery.validate.js • https://login.teamwork.sg/js/moment/moment.min.js • https://login.teamwork.sg/js/new_js/bootstrap.min.js • https://login.teamwork.sg/settings/terms • https://login.teamwork.sg/vendors/bootstrap/dist/js/bootstrap.min.js • https://login.teamwork.sg/vendors/ckeditor/ckeditor.js • https://login.teamwork.sg/vendors/custom_js/datatables/js/jquery.dataTables.js • https://login.teamwork.sg/vendors/datatables.net/js/jquery.dataTables.min.js • https://login.teamwork.sg/vendors/moment/min/moment.min.js



	https://login.teamwork.sg/vendors/select2/dist/js/select2.full.min.js
Impact	An attacker can exploit known vulnerabilities in outdated components to gain unauthorized access, execute malicious code, cause data breaches, or disrupt application availability.
References	https://owasp.org/Top10/2021/A06_2021-Vulnerable_and_Outdated_Components/index.html
Recommendation	- Regularly update all third-party libraries, frameworks, and components - Remove unused or unsupported components - Monitor dependencies for known vulnerabilities (using tools like dependency scanners) - Maintain an inventory of all application components and their versions

Proof of Concept

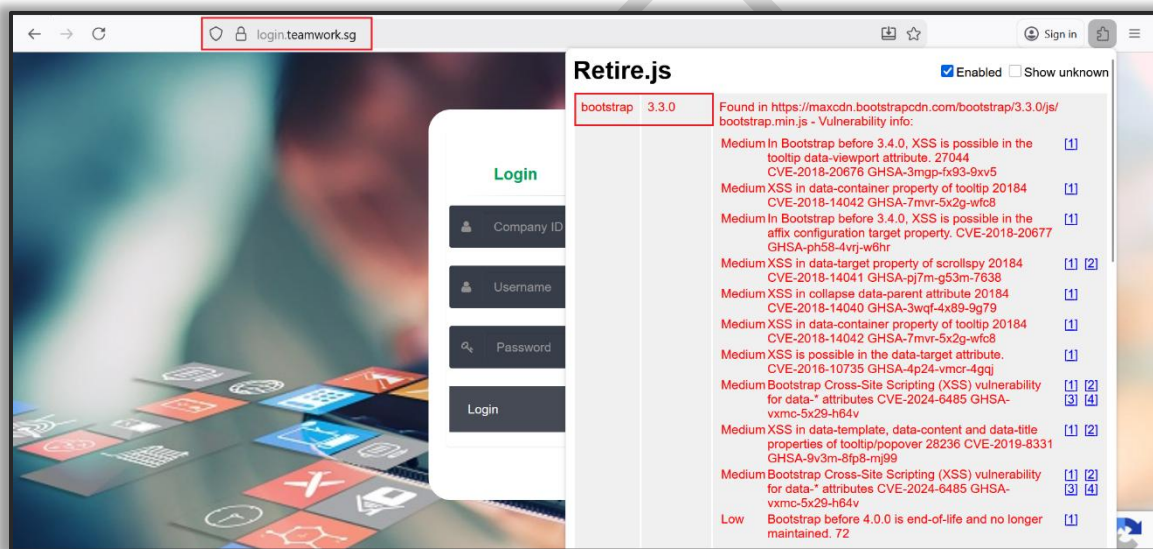


FIGURE 75: VULNERABLE AND OUTDATED COMPONENTS 01

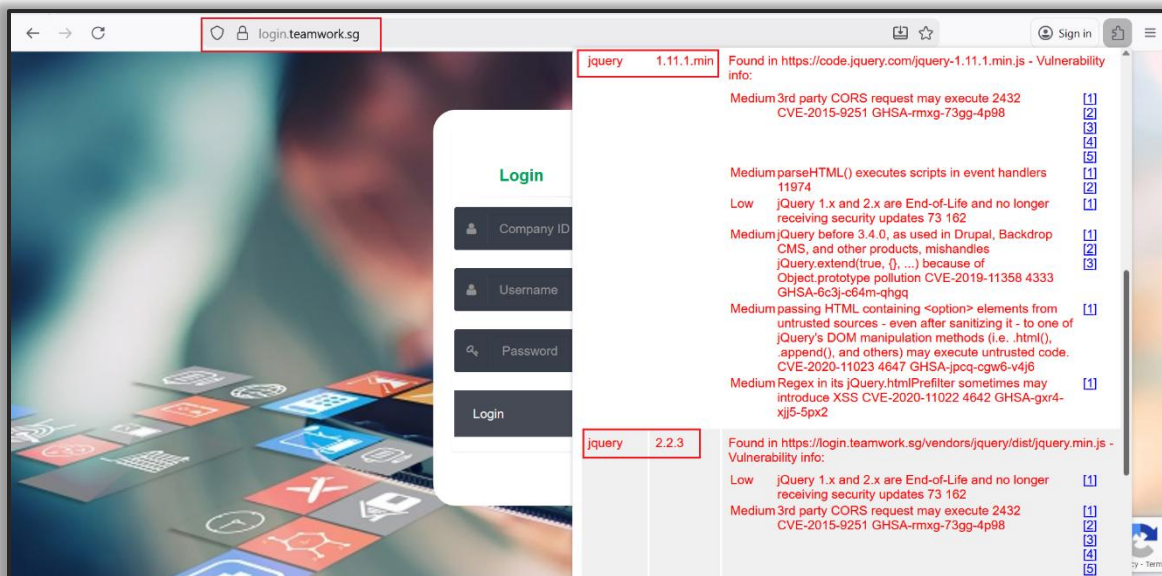


FIGURE 76: VULNERABLE AND OUTDATED COMPONENTS 02

Issue detail

We observed a vulnerable JavaScript library.

We detected **jquery** version **3.3.1**, which has the following vulnerabilities:

- [CVE-2019-11358](#): jQuery before 3.4.0, as used in Drupal, Backdrop CMS, and other products, mishandles `jQuery.extend(true, {}, ...)` because of Object.prototype pollution
- [CVE-2020-11023](#): passing HTML containing `<option>` elements from untrusted sources - even after sanitizing it - to one of jQuery's DOM manipulation methods (i.e. `.html()`, `.append()`, and others) may execute untrusted code.
- [CVE-2020-11022](#): Regex in its `jQuery.htmlPrefilter` sometimes may introduce XSS

FIGURE 77: VULNERABLE AND OUTDATED COMPONENTS 03

Issue detail

We observed a vulnerable JavaScript library.

We detected **select2** version **4.0.5**, which has the following vulnerability:

- [CVE-2016-10744](#): Improper Neutralization of Input During Web Page Generation in Select2

FIGURE 78: VULNERABLE AND OUTDATED COMPONENTS 04

**Issue detail**

We observed a vulnerable JavaScript library.

We detected **jquery.datatables** version **1.10.13**, which has the following vulnerabilities:

- [CVE-2020-28458](#): datatables.net vulnerable to Prototype Pollution due to incomplete fix
- prototype pollution
<https://cdn.datatables.net/1.10.22/>
- prototype pollution
<https://cdn.datatables.net/1.10.23/>
<https://github.com/DataTables/DataTablesSrc/commit/a51cbe99fd3d02aa5582f97d4af1615d11a1ea03>
- possible XSS
<https://cdn.datatables.net/1.11.3/>
<https://github.com/DataTables/Dist-DataTables/commit/59a8d3f8a3c1138ab08704e783bc52bfe88d7c9b>
- [CVE-2021-23445](#): Cross site scripting in datatables.net

FIGURE 79: VULNERABLE AND OUTDATED COMPONENTS 05

Issue detail

We observed a vulnerable JavaScript library.

We detected **lodash** version **4.17.4**, which has the following vulnerabilities:

- [CVE-2018-3721](#): Prototype Pollution in lodash
- [CVE-2018-16487](#): Prototype Pollution in lodash
- [CVE-2019-1010266](#): Regular Expression Denial of Service (ReDoS) in lodash
- [CVE-2019-10744](#): Prototype Pollution in lodash
- [CVE-2020-8203](#): Prototype Pollution in lodash
- [CVE-2020-28500](#): Regular Expression Denial of Service (ReDoS) in lodash
- [CVE-2021-23337](#): Command Injection in lodash

FIGURE 80: VULNERABLE AND OUTDATED COMPONENTS 06

Issue detail

We observed a vulnerable JavaScript library.

We detected **moment.js** version **2.9.0**, which has the following vulnerabilities:

- [CVE-2016-4055](#): reDOS - regular expression denial of service
- Regular Expression Denial of Service (ReDoS)
<https://security.snyk.io/vuln/npm:moment:20161019>
- [CVE-2017-18214](#): Regular Expression Denial of Service (ReDoS)
- [CVE-2022-24785](#): This vulnerability impacts npm (server) users of moment.js, especially if user provided locale string, eg fr is directly used to switch moment locale.

FIGURE 81: VULNERABLE AND OUTDATED COMPONENTS 07

Issue detail

We observed a vulnerable JavaScript library.

We detected **bootstrap** version **3.4.1**, which has the following vulnerability:

- Bootstrap before 4.0.0 is end-of-life and no longer maintained.
<https://github.com/twbs/bootstrap/issues/20631>

FIGURE 82: VULNERABLE AND OUTDATED COMPONENTS 08

Issue detail

We observed a vulnerable JavaScript library.

We detected **jquery-ui** version **1.12.1**, which has the following vulnerabilities:

- [CVE-2021-41182](#): XSS in the `altField` option of the Datepicker widget
- [CVE-2021-41184](#): XSS in the `of` option of the `position()` util
- [CVE-2021-41183](#): XSS Vulnerability on text options of jQuery UI datepicker
- [CVE-2022-31160](#): XSS when refreshing a checkboxradio with an HTML-like initial text label

FIGURE 83: VULNERABLE AND OUTDATED COMPONENTS 09

Issue detail

We observed 2 vulnerable JavaScript libraries.

We detected **jquery-ui** version **1.10.1**, which has the following vulnerabilities:

- [CVE-2021-41182](#): XSS in the `altField` option of the Datepicker widget
- [CVE-2021-41184](#): XSS in the `of` option of the `position()` util
- [CVE-2021-41183](#): XSS Vulnerability on text options of jQuery UI datepicker
- [CVE-2022-31160](#): XSS when refreshing a checkboxradio with an HTML-like initial text label

We also detected **jquery-ui-dialog** version **1.10.1**, which has the following vulnerability:

- [CVE-2016-7103](#): XSS Vulnerability on closeText option

FIGURE 84: VULNERABLE AND OUTDATED COMPONENTS 10

**Issue detail**

We observed a vulnerable JavaScript library.

We detected **jquery-validation** version **1.12.0**, which has the following vulnerabilities:

- [CVE-2021-21252](#): Regular Expression Denial of Service vulnerability
- [CVE-2021-43306](#): ReDoS vulnerability in URL2 validation
- [CVE-2022-31147](#): ReDoS vulnerability in url and URL2 validation
- Potential XSS via showLabel
<https://github.com/jquery-validation/jquery-validation/blob/master/changelog.md#1200--2023-10-10>

FIGURE 85: VULNERABLE AND OUTDATED COMPONENTS 11

Issue detail

We observed a vulnerable JavaScript library.

We detected **jquery-ui** version **1.12.1**, which has the following vulnerabilities:

- [CVE-2021-41182](#): XSS in the `altField` option of the DatePicker widget
- [CVE-2021-41184](#): XSS in the `of` option of the `.position()` util
- [CVE-2021-41183](#): XSS Vulnerability on text options of jQuery UI datepicker
- [CVE-2022-31160](#): XSS when refreshing a checkboxradio with an HTML-like initial text label

FIGURE 86: VULNERABLE AND OUTDATED COMPONENTS 12

Issue detail

We observed a vulnerable JavaScript library.

We detected **bootstrap** version **3.3.6**, which has the following vulnerabilities:

- [CVE-2018-20676](#): In Bootstrap before 3.4.0, XSS is possible in the tooltip data-viewport attribute.
- [CVE-2018-14042](#): XSS in data-container property of tooltip
- [CVE-2018-20677](#): In Bootstrap before 3.4.0, XSS is possible in the affix configuration target property.
- [CVE-2018-14041](#): XSS in data-target property of scrollspy
- [CVE-2016-10735](#): XSS is possible in the data-target attribute.
- [CVE-2019-8331](#): XSS in data-template, data-content and data-title properties of tooltip/popover
- Bootstrap before 4.0.0 is end-of-life and no longer maintained.
<https://github.com/twbs/bootstrap/issues/20631>

FIGURE 87: VULNERABLE AND OUTDATED COMPONENTS 13

**Issue detail**

We observed a vulnerable JavaScript library.

We detected **ckeditor** version **4.11.4**, which has the following vulnerabilities:

- XSS
<https://github.com/ckeditor/ckeditor4/blob/major/CHANGES.md#ckeditor-414>
- XSS-type attack inside CKEditor 4 by persuading a victim to paste a specially crafted HTML code into the Color Button dialog
<https://github.com/ckeditor/ckeditor4/blob/major/CHANGES.md#ckeditor-4151>
- ReDoS vulnerability in Autolink plugin and Advanced Tab for Dialogs plugin
<https://ckeditor.com/cke4/release/CKEditor-4.16.0>
- [CVE-2021-32808](#): XSS vulnerability in the Widget plugin
- [CVE-2021-32809](#): XSS vulnerability in the Clipboard plugin
- [CVE-2021-37695](#): XSS vulnerability in the Fake Objects plugin
- [CVE-2021-41164](#), [CVE-2021-41165](#): XSS vulnerabilities in the core module
- [CVE-2022-24728](#): Inject malformed URL to bypass content sanitization for XSS
- [CVE-2023-28439](#): cross-site scripting vulnerability has been discovered affecting Iframe Dialog and Media Embed packages. The vulnerability may trigger a JavaScript code

FIGURE 88: VULNERABLE AND OUTDATED COMPONENTS 14

Issue detail

We observed a vulnerable JavaScript library.

We detected **jquery.datatables** version **1.10.10**, which has the following vulnerabilities:

- [CVE-2020-28458](#): datatables.net vulnerable to Prototype Pollution due to incomplete fix
- prototype pollution
<https://cdn.datatables.net/1.10.22/>
- prototype pollution
<https://cdn.datatables.net/1.10.23/>
<https://github.com/DataTables/DataTablesSrc/commit/a51cbe99fd3d02aa5582f97d4af1615d11a1ea03>
- possible XSS
<https://cdn.datatables.net/1.11.3/>
<https://github.com/DataTables/Dist-DataTables/commit/59a8d3f8a3c1138ab08704e783bc52bfe88d7c9b>
- [CVE-2021-23445](#): Cross site scripting in datatables.net

FIGURE 89: VULNERABLE AND OUTDATED COMPONENTS 15

**Issue detail**

We observed a vulnerable JavaScript library.

We detected **jquery.datatables** version **1.10.11**, which has the following vulnerabilities:

- [CVE-2020-28458](#): datatables.net vulnerable to Prototype Pollution due to incomplete fix
- prototype pollution
<https://cdn.datatables.net/1.10.22/>
- prototype pollution
<https://cdn.datatables.net/1.10.23/>
<https://github.com/DataTables/DataTablesSrc/commit/a51cbe99fd3d02aa5582f97d4af1615d11a1ea03>
- possible XSS
<https://cdn.datatables.net/1.11.3/>
<https://github.com/DataTables/Dist-DataTables/commit/59a8d3f8a3c1138ab08704e783bc52bfe88d7c9b>
- [CVE-2021-23445](#): Cross site scripting in datatables.net

FIGURE 90: VULNERABLE JAVASCRIPT LIBRARY lodash 4.17.4

Issue detail

We observed a vulnerable JavaScript library.

We detected **moment.js** version **2.13.0**, which has the following vulnerabilities:

- Regular Expression Denial of Service (ReDoS)
<https://security.snyk.io/vuln/npm:moment:20161019>
- [CVE-2017-18214](#): Regular Expression Denial of Service (ReDoS)
- [CVE-2022-24785](#): This vulnerability impacts npm (server) users of moment.js, especially if user provided locale string, eg fr is directly used to switch moment locale.

FIGURE 91: VULNERABLE AND OUTDATED COMPONENTS 16

Issue detail

We observed a vulnerable JavaScript library.

We detected **select2** version **4.0.2**, which has the following vulnerability:

- [CVE-2016-10744](#): Improper Neutralization of Input During Web Page Generation in Select2

FIGURE 92: VULNERABLE AND OUTDATED COMPONENTS 17



3.1.23 Method Interchange

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	HTTP Method Interchange is a vulnerability where an application allows sensitive actions to be performed using an unintended HTTP method, such as processing authenticated actions through GET instead of POST.
Vulnerability Description	During the assessment, it was observed that the application initially processes authenticated requests using the POST method. However, as the user navigates further into the application workflow (such as accessing intermediate pages and the dashboard), subsequent requests are handled using the GET method. The application continues to process these requests successfully without enforcing strict HTTP method validation. This behavior indicates that sensitive application functionality is accessible through unintended HTTP methods, confirming the presence of an HTTP Method Interchange vulnerability.
Root Cause	Improper enforcement of HTTP methods for sensitive application functionality.
Affected URL	https://login.teamwork.sg/welcome/index?accessToken=
Impact	An attacker may exploit this issue to bypass security controls, perform unauthorized actions, expose sensitive data through URLs, enable CSRF attacks, or manipulate application behavior by abusing GET-based requests.
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/06-Test_HTTP_Methods
Recommendation	It is recommended to: • Enforce strict HTTP method validation on the server side • Use POST for all sensitive actions such as authentication and data modification • Reject requests using unexpected HTTP methods • Implement CSRF protection for state-changing requests • Avoid passing sensitive data via GET parameters

Proof of Concept

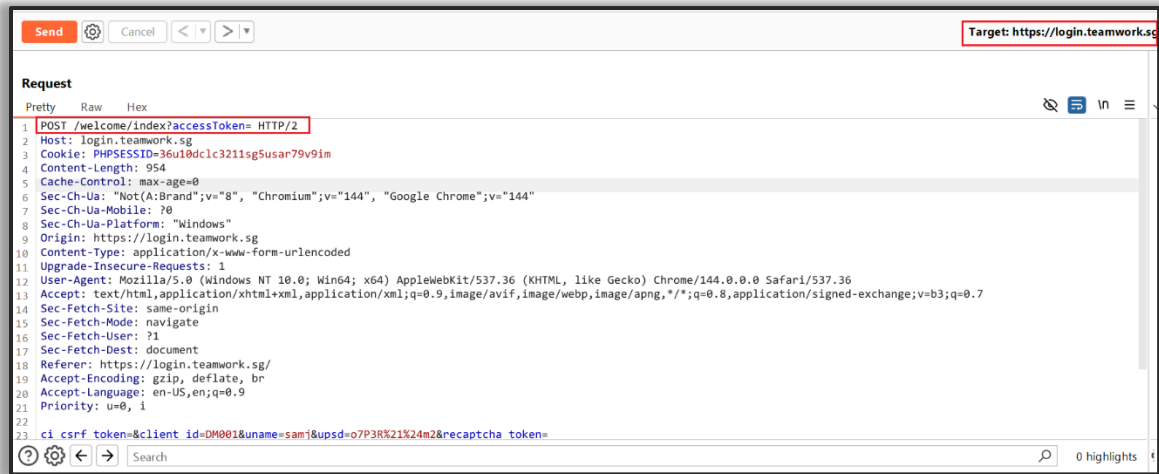


FIGURE 93: METHOD INTERCHANGE 01

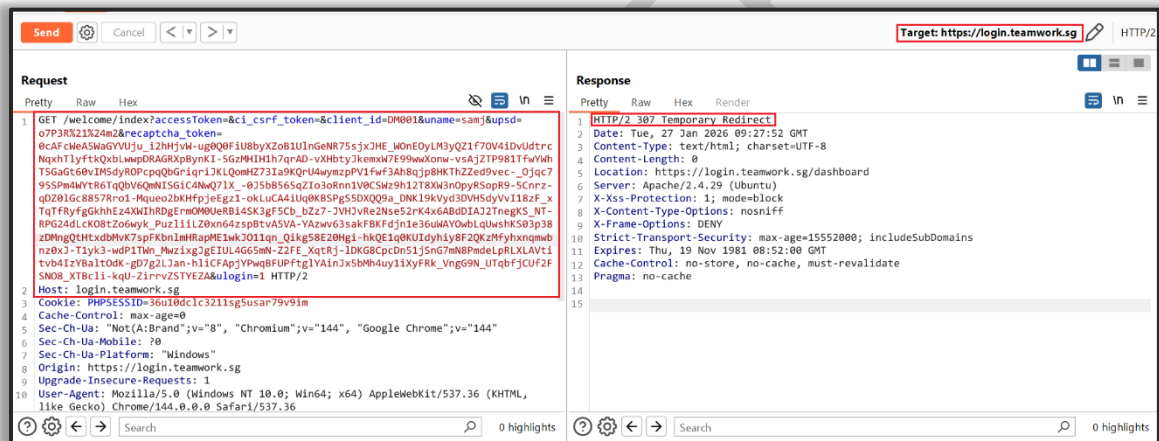


FIGURE 94: METHOD INTERCHANGE 02

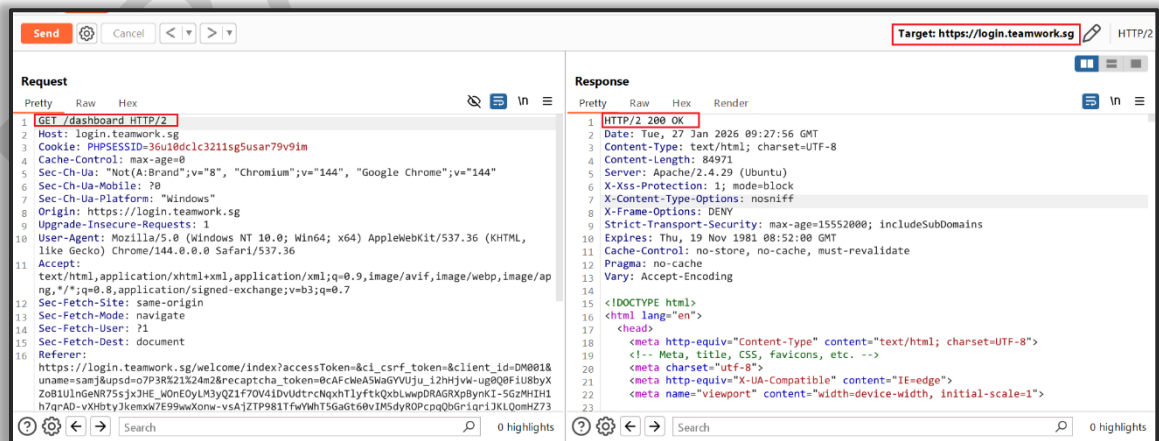


FIGURE 95: METHOD INTERCHANGE 03



3.1.24 Missing CAPTCHA

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A07:2021 – Identification and Authentication Failures
Vulnerability Definition	Missing CAPTCHA is a security issue where an application does not implement human-verification controls on sensitive functionalities, allowing automated abuse.
Vulnerability Description	During the assessment, we observed that the application does not implement CAPTCHA protection on sensitive actions such as login, Forgot Password page, registration, or form submission. These functionalities can be accessed repeatedly without any human-verification mechanism, allowing automated scripts or bots to interact with the application without restriction.
Root Cause	Absence of CAPTCHA or bot-detection mechanisms on sensitive endpoints.
Affected URL	https://login.teamwork.sg/welcome/forgot_password
Impact	An attacker can automate brute-force attacks, credential stuffing, fake account creation, spam submissions, or abuse application resources, potentially leading to account compromise or service disruption.
References	https://www.cobalt.io/vulnerability-wiki/v2-authentication/no-rate-limiting-or-captcha-on-login-page
Recommendation	• Implement CAPTCHA on sensitive functionalities (login, OTP, registration, password reset) • Use adaptive or risk-based CAPTCHA mechanisms • Combine CAPTCHA with rate limiting and monitoring • Apply temporary blocking after multiple failed attempts
Proof of Concept	

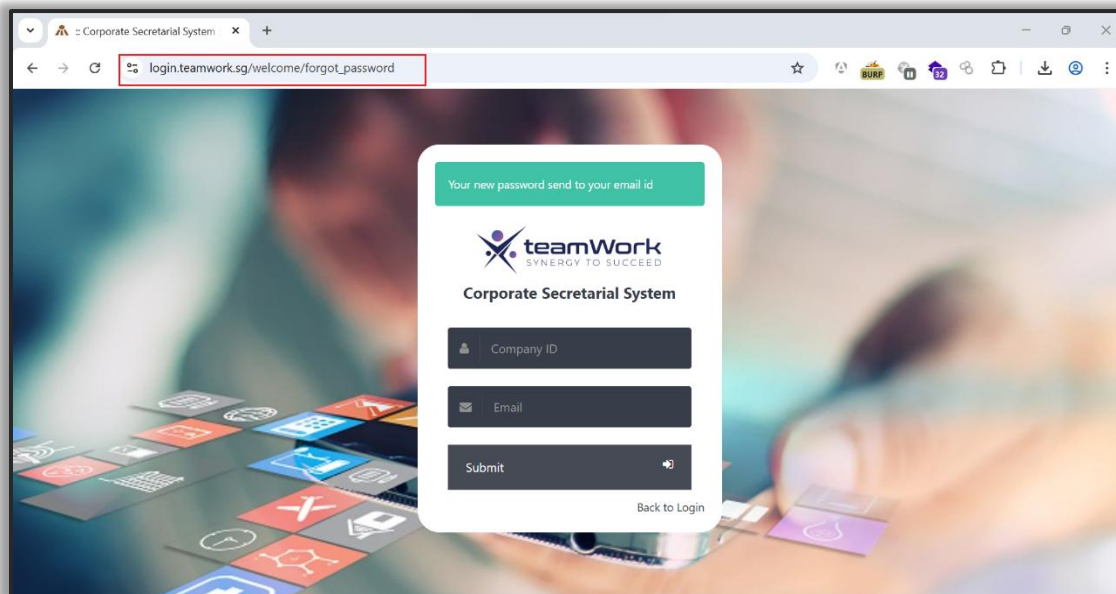


FIGURE 96-CAPTCHA NOT IMPLEMENTED



3.1.25 Sensitive Information Disclosure via URL

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Sensitive Information Disclosure via URL occurs when confidential data such as authentication or access tokens are exposed in URL parameters, making them visible and easily accessible.
Vulnerability Description	During the assessment, we observed that the application includes sensitive information such as access tokens in the URL parameters. These tokens were visible in the browser address bar and could be captured through browser history, logs, bookmarks, referrer headers, or shared links. Exposing tokens in URLs increases the risk of unauthorized access and session hijacking.
Root Cause	Improper handling of sensitive tokens by placing them in URL parameters instead of secure headers or cookies.
Affected URL	https://login.teamwork.sg/preincorporation_request/edit/1176/?token=caDAI2i10I
Impact	An attacker can steal exposed tokens and reuse them to impersonate users, gain unauthorized access, and perform actions on behalf of the victim without authentication.
References	https://owasp.org/www-community/vulnerabilities/Information_exposure_through_query_strings_in_url
Recommendation	• Never include sensitive tokens in URLs • Transmit tokens using secure HTTP headers (e.g., Authorization header) or secure cookies • Apply HttpOnly, Secure, and SameSite cookie flags • Rotate and invalidate tokens frequently • Implement token expiration and revocation mechanisms
Proof of Concept	

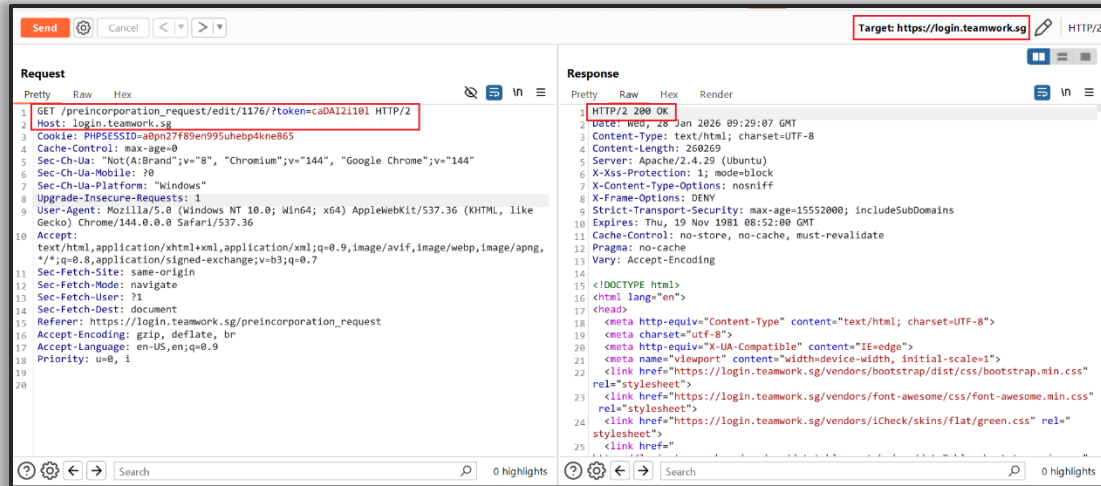


FIGURE 97-SENSITIVE INFORMATION DISCLOSURE 01



3.1.26 Open Redirection

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A01:2021 – Broken Access Control
Vulnerability Definition	Open Redirect is a vulnerability where an application redirects users to an external or untrusted URL based on user-controlled input without proper validation.
Vulnerability Description	During the assessment, we observed that the application accepts a user-supplied URL parameter and redirects the user to the provided destination without validating or restricting it. By manipulating this parameter, users can be redirected to arbitrary external websites, confirming the presence of an Open Redirect vulnerability.
Root Cause	Lack of validation or allow-listing of redirect URLs provided by user input.
Affected URL	https://login.teamwork.sg/tasks?type=http%3a%2f%2fattacker.com
Impact	An attacker can exploit this issue to perform phishing attacks, redirect users to malicious websites, steal credentials, or abuse the application's trusted domain to increase attack credibility.
References	https://portswigger.net/kb/issues/00500100_open-redirection-reflected
Recommendation	• Validate and restrict redirect URLs using an allow-list • Avoid using user-controlled input directly in redirect functions • Use relative paths instead of absolute URLs for redirection • Display a warning or confirmation page before redirecting to external sites
Proof of Concept	

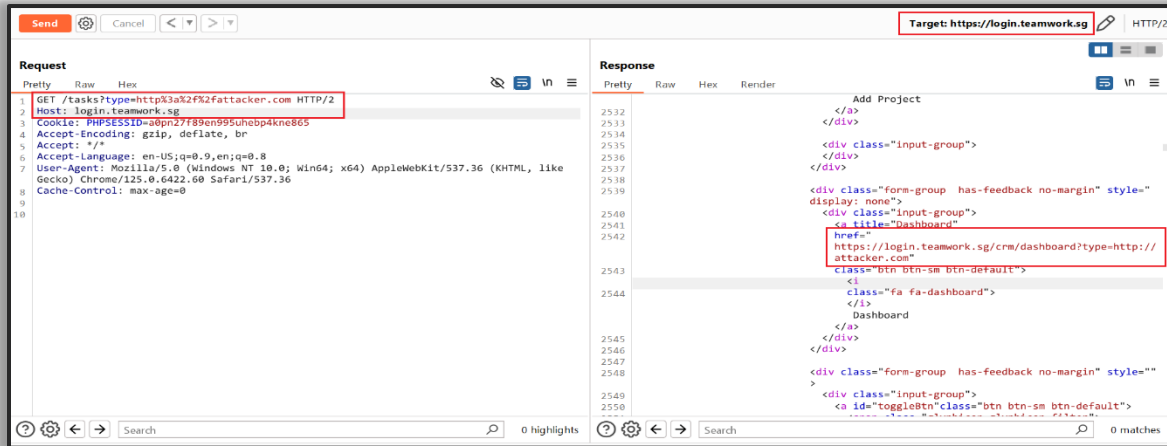


FIGURE 98 –OPEN REDIRECTION ISSUE



3.1.27 JSON Web Key set Disclosed

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A02:2021 – Cryptographic Failures
Vulnerability Definition	JSON Web Key Set (JWKS) Disclosure occurs when an application publicly exposes its cryptographic keys or key metadata, which can be misused to validate or potentially forge JSON Web Tokens (JWTs).
Vulnerability Description	During the assessment, we observed that the application exposes its JSON Web Key Set (JWKS) endpoint publicly. This endpoint discloses cryptographic key information used for JWT signing or verification. If improperly configured or combined with weak token validation, this exposure may allow attackers to analyze token signing mechanisms and potentially craft or abuse JWTs.
Root Cause	Improper access control and configuration of the JWKS endpoint.
Affected URL	https://login.teamwork.sg/jwks.json
Impact	An attacker can use the disclosed key information to analyze JWT structure, validate tokens, or attempt token manipulation. In certain scenarios, this may lead to authentication bypass, privilege escalation, or unauthorized access.
Reference	https://datatracker.ietf.org/doc/html/rfc7517
Recommendation	• Restrict access to the JWKS endpoint where possible • Ensure only public keys (never private keys) are exposed • Enforce strict JWT validation (issuer, audience, algorithm) • Rotate signing keys regularly • Use secure and recommended JWT algorithms
Proof of Concept	

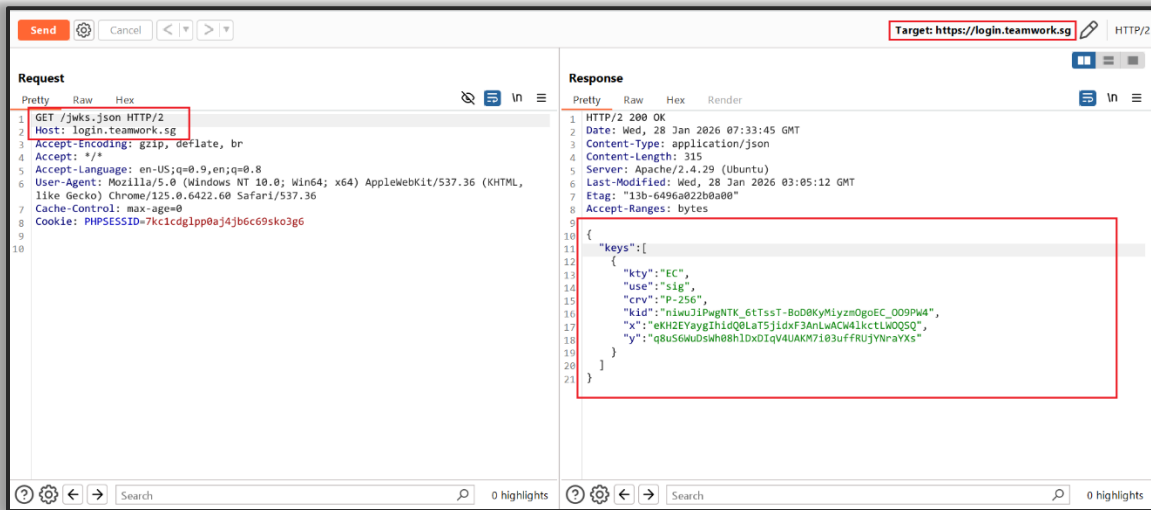


FIGURE 99 – JSON WEB KEY SET (JWKS) DISCLOSED



3.1.28 Missing Cookie Attribute

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021 – Security Misconfiguration
Vulnerability Definition	Missing Cookie Attributes is a security issue where cookies are not configured with recommended security flags, increasing the risk of session hijacking and client-side attacks.
Vulnerability Description	During the assessment, we observed that cookies used by the application are missing important security attributes such as HttpOnly, Secure, and SameSite. Due to this misconfiguration, cookies may be accessed by client-side scripts, transmitted over unencrypted connections, or sent in cross-site requests, exposing them to potential attacks.
Root Cause	Improper configuration of cookie security attributes on the server.
Affected URL	https://login.teamwork.sg/dashboard
Impact	An attacker may steal or manipulate session cookies through XSS, man-in-the-middle attacks, or CSRF, leading to session hijacking, unauthorized access, and account compromise.
References	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie
Recommendation	• Set HttpOnly to prevent client-side script access • Set Secure to ensure cookies are sent only over HTTPS • Set SameSite (Strict or Lax) to reduce CSRF risk • Review and apply secure defaults for all authentication cookies
Proof of Concept	

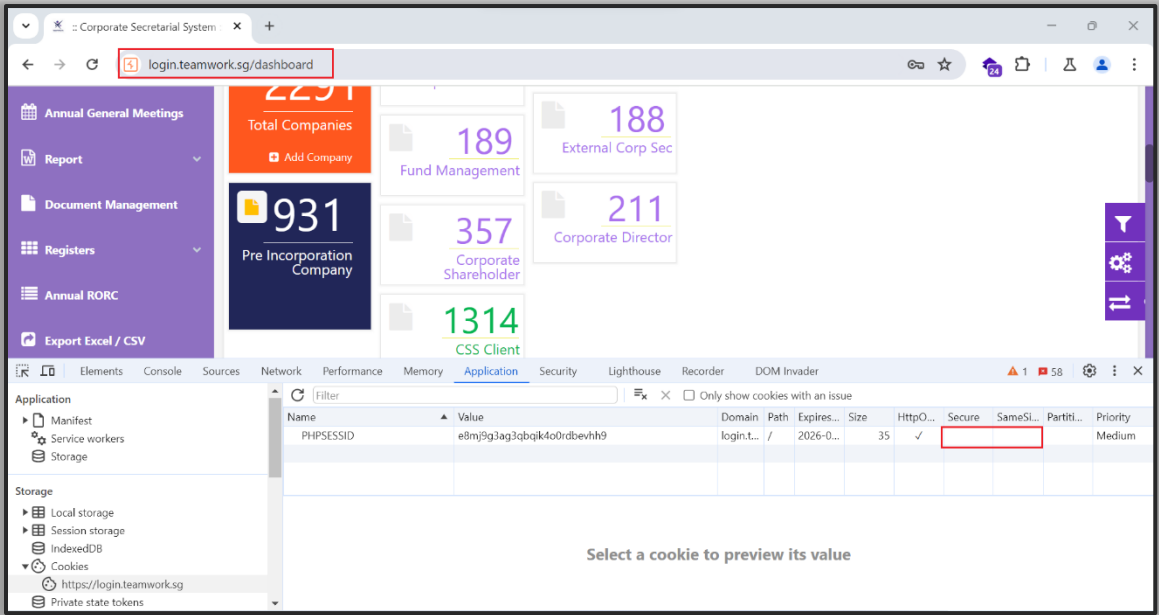


FIGURE 100 – MISSING COOKIE ATTRIBUTES



3.1.29 Concurrent Login Session

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A07:2021 Identification and Authentication Failures
Vulnerability Definition	Concurrent Session Login is a security issue where an application allows the same user account to be logged in from multiple devices or browsers at the same time without restriction.
Vulnerability Description	During our assessment, we observed that it permits a user to have multiple simultaneous active sessions across various devices or browsers.
Root Cause	Improper Design
Affected URL	https://login.teamwork.sg
Impact	Concurrent sessions can increase the risk of unauthorized access if session tokens are compromised. Attackers may gain persistent access, making it harder to detect suspicious activity, which could lead to data breaches and account misuse.
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/11-Testing_for_Concurrent_Sessions
Recommendation	It is recommended to allow the termination of active sessions for enhanced user control.
Proof of Concept	

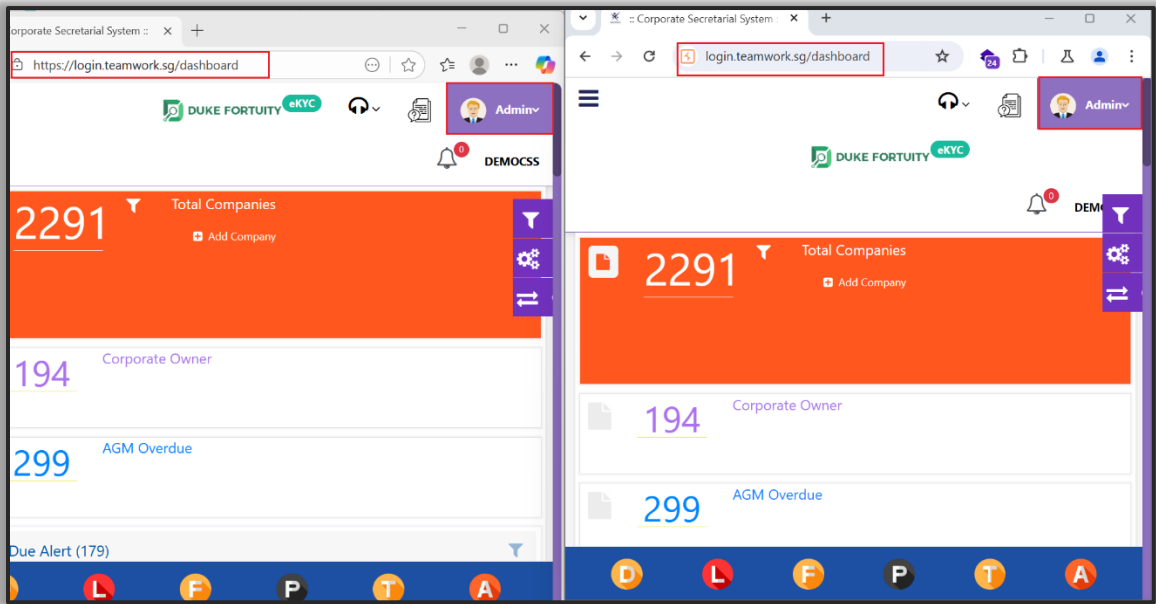


FIGURE 101 – CONCURRENT LOGIN



3.1.30 Breach & Lucky13 Attack

Parameters	Description
Vulnerability Severity	Low
OWASP Rating	A05:2021-Security Misconfiguration
Vulnerability Definition	Lucky Thirteen: A timing side-channel attack targeting CBC-mode ciphers in TLS, allowing an attacker to infer information about encrypted data by measuring subtle timing differences during MAC validation.
Vulnerability Description	During our assessment it was observed that the web application vulnerable to lucky 13 attack.
Root Cause	- HTTP compression enabled indiscriminately on sensitive responses. Secrets not properly separated from attacker-controlled input. - Use of CBC-mode ciphers in TLS configuration without mitigations such as constant-time MAC/padding validation.
Affected URL	https://login.teamwork.sg
Impact	- Disclosure of authentication tokens, session IDs. - Session hijacking and unauthorized actions. - Exposure of sensitive data such as credentials or session information.
References	https://www.openssl.org/~bodo/tls-cbc.txt
Recommendation	- Disable HTTP compression for sensitive responses (e.g., authentication endpoints). - Apply constant-time MAC and padding validation in TLS implementation.

Proof of Concept

```

Testing vulnerabilities
Heartbleed (CVE-2014-0160) not vulnerable (OK), no heartbeat extension
CCS (CVE-2014-0224) not vulnerable (OK)
Ticketbleed (CVE-2016-9244), experiment. not vulnerable (OK), reply empty
Opussum (CVE-2025-49812) not vulnerable (OK)
ROBOT not vulnerable (OK)
Secure Renegotiation (RFC 5746) supported (OK)
Secure Client-Initiated Renegotiation not vulnerable (OK)
CRIME-TLS (CVE-2012-4929) not vulnerable (OK)
BREACH (CVE-2013-3587) potentially NOT ok, "gzip" HTTP compression detected, - only supplied "/" tested
Can be ignored for static pages or if no secrets in the page
POODLE, SSL (CVE-2014-3566) not vulnerable (OK), no SSLv3 support
TLS_FALLBACK_SCSV (RFC 7507) No fallback possible (OK), no protocol below TLS 1.2 offered
SWEET32 (CVE-2016-2183, CVE-2016-6329) not vulnerable (OK)
FREAK (CVE-2015-0204) not vulnerable (OK)
DROWN (CVE-2016-0808, CVE-2016-0703) not vulnerable on this host and port (OK)
make sure you don't use this certificate elsewhere with SSLv2 enabled services, see
https://search.censys.io/search?resource=hosts&virtual_hosts=INCLUDE&q=88865851DD568335E2055F1A46E3717CAD3B30F4B543F3568C791F78A5F15D8
LOGJAM (CVE-2015-4000), experimental not vulnerable (OK): no DH EXPORT ciphers, no DH key detected with <= TLS 1.2
BEAST (CVE-2011-3389) not vulnerable (OK), no SSL3 or TLS1
LUCKY13 (CVE-2013-0169), experimental potentially VULNERABLE, uses cipher block chaining (CBC) ciphers with TLS. Check patches
WINSHOCK (CVE-2015-0321), experimental not vulnerable (OK)
RC4 (CVE-2013-2566, CVE-2015-2808) no RC4 ciphers detected (OK)

```

FIGURE 102 - BREACH AND LUCKY 13 ATTACK



Annexure A – Security Assessment Test Case Sheet

The following sheet contains information of the various test cases used while conducting Web application security assessment.

Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
1	Reflected Cross Site Scripting	In Reflected XSS attack, the injected code is reflected off the web server, such as in popup from executed script, error message, or any other response that includes some or all the input sent to the server as part of the request. When a user is tricked into clicking on a malicious link or submitting a specially crafted form, the injected code travels to the vulnerable web server, which reflects the attack back to the user's browser.	Vulnerable
2	Stored Cross Site Scripting	In Stored XSS attacks, the injected malicious code is permanently stored on the target servers, such as in a database, in a message forum, visitor log, comment field, etc. when victim requests to the stored information, the malicious script gets executed on his browser.	Not Vulnerable
3	DOM based Cross Site Scripting	In the DOM Based XSS attack, malicious payload is executed as a result of modifying the DOM (Document Object Model) "environment" in the victim's browser used by the original client-side script, so that the client-side code runs in an "unexpected" manner.	Not Vulnerable
4	Injectons - SQL, LDAP, Code Injectons	Injection is an attack where user input is directly appended to the query without any validation. The attacker may be able to inject control characters into the query and change the entire structure of the query. Additional conditions can be added to the query, or parts of the query can be commented out. This can lead to leakage of data that the attacker is not authorized to see by allowing the attacker to run any arbitrary SQL statement against the database.	Not Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
5	HTML Injection	HTML Injection refers to injecting HTML code into a web server's response to alter the content to the end user.	Vulnerable
6	Vertical & Horizontal Privilege Escalation	Privilege escalation happens when it is possible to access resources granted to more privileged account via changing some parameters.	Not Vulnerable
7	Cross Site Request Forgery Testing	A CSRF attack forces a logged-on victim's browser to send a forged HTTP request, including the victim's session cookie and any other automatically included authentication information, to a vulnerable web application. This allows the attacker to force the victim's browser to generate requests the vulnerable application thinks are legitimate requests from the victim.	Not Vulnerable
8	HTTP Response Splitting	HTTP response splitting occurs when the data is included in an HTTP response header sent to a web user without being validated for malicious characters.	Not Vulnerable
9	Forced Browsing	In this attack an attacker tries to enumerate and access resources that are not referenced by the application but are still accessible. This attack is performed manually when the application index directories and pages are based on number generation or predictable values or using automated tools for common files and directory names.	Not Vulnerable
10	Session Fixation	Session Fixation is an attack technique that forces a user's session ID to an explicit value. The attacker then causes the victim to authenticate against the server using the same session identifier, giving the attacker access to the user's account through the active session.	Vulnerable
11	Arbitrary File Upload Possible	In this attack, the affected application allows user to upload any arbitrary file through upload module. An attacker may possibly upload malicious ".exe" files on to the server.	Vulnerable
12	Installed application platforms are vulnerable	In this attack, the installed application platform is already vulnerable to several vulnerabilities. An attacker can use those to craft an attack on the application.	Vulnerable
13	File Inclusion Vulnerability	File Inclusion attack possible When web applications take user input (URL, parameter	Not Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
		value, etc.) and pass them into file include commands, the web application might be tricked into including remote files with malicious code.	
14	Unrestricted URL Access	In this attack, an attacker can get authenticated pages by just accessing the URL through browser without having credentials.	Vulnerable
15	Bypassing client-side validation for change password	The applications, which rely on client-side validations, are prone to this attack. In this attack, an attacker tries to change the password while bypassing the old password field for validation.	Not Vulnerable
16	Query Parameter in GET request	The get request is known as unsecure method as all the parameters through this request transmitted in URL. If application transfers any sensitive information through GET request, an attacker may exploit it to get sensitive information.	Vulnerable
17	Unhandled Error	Applications can unintentionally leak information about their configuration, internal workings, or violate privacy through a variety of application errors. Attackers may use this weakness to steal sensitive data or conduct more serious attacks.	Vulnerable
18	Parallel session and authorization check	An attacker would be unnoticed, if application allows concurrent user session to login and perform the actions.	Not Vulnerable
19	Application Shows Last User Login Date & time	As per the security best practices, it is recommended to show last user login information for End user's notice.	Not Vulnerable
20	Parameter Tampering	The Web Parameter Tampering attack is based on the manipulation of parameters exchanged between client and server in order to modify application data, such as user credentials and permissions, price, and quantity of products, etc. Usually, this information is stored in cookies, or URL Query Strings, and is used to increase application functionality and control.	Vulnerable
21	Hidden Parameter validation	In this attack, an attacker tries to change the values in hidden parameter through proxy or another tool. If application does not validate hidden parameter at server level, attacker may	Not Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
		use this to modify sensitive hidden field's values.	
22	Change "To Mail Id" while send mail	If application does not validate "To Mail Id" parameter while sending mail through application, then an attacker may be able to send mails to anyone from any email id.	Vulnerable
23	Check for password history	If application allows setting the last used passwords for reuse, it would be easy for an attacker to guess the credentials.	Not Vulnerable
24	Password Complexity	As per the security best practices, an application must check for complexity of password.	Vulnerable
25	Weak SSL or Cipher testing	If the target server supports the use of weak SSL ciphers or weak algorithm, an attacker may deprecate the communication between the affected application and its client.	Vulnerable
26	Backup and unreferenced files used	If an application contains unreferenced and/or forgotten files on production server that can be used to obtain critical information about either the infrastructure or the credentials.	Vulnerable
27	Access to Admin interfaces	If application does not restrict open admin interface to authorized IPs, it may possible that an attacker may access it remotely and preform malicious activity.	Vulnerable
28	Unnecessary HTTP Methods enabled	If application allows to unnecessary HTTP methods like PUT, Delete, Trace etc. an attacker can misuse these methods to take control on server.	Vulnerable
29	Credential's transport over an unencrypted channel	If SSL is not implemented on the web server, sensitive information such as Username, password goes across the wire in clear text. This allows determined attacker to perform man in the middle attack on web application for capturing the credentials.	Vulnerable
30	User enumeration possible	If application through different error message for wrong username and wrong password, an attacker would be able to get a list of users on system.	Vulnerable
31	Guessable user account found	If default and guessable credentials are being used in the application, then an attacker can get access to them and may lead to compromise the application.	Not Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
32	Weak Password Forgot Functionality	If applications forgot password functionality is properly secure, an attacker may be able to get the password of another user by using weakness in it.	Not Vulnerable
33	By-passing authentication Mechanism through Injections	If application's login page is affected from Injection attacks, the authentication mechanism can be bypassed.	Not Vulnerable
34	AutoComplete is not disabled	If the application does not prohibit autocomplete feature, the victim's browser memorizes the username and passwords in clear text, which can be easily compromised.	Vulnerable
35	Improper Cache Management	Web caching is the caching of web documents (e.g., HTML pages, images) in order to reduce bandwidth usage, server load, and perceived lag. A web cache stores copies of documents passing through it; subsequent requests may be satisfied from the cache if certain conditions are met. Caching should only be used on static content that does not require "real-time" access.	Not Vulnerable
36	Weak Captcha implementation on Login pages	If Captcha is not implemented properly, an attacker can easily guess the Captcha and can bypass it.	Not Vulnerable
37	Bypass Multiple Factors Authentication	If the Process of validating the second factor authentication is not properly implemented, an attacker may be able to bypass it.	Not Vulnerable
38	Weak Session generation algorithm	Session algorithm is used for creating unique and random session for each login session, which used for maintaining authenticated session. If the session algorithm is weak then an attacker may guess new session id and may access to authenticated pages.	Not Vulnerable
39	Session cookie mechanism for Pre & Post Authentication	If web application does not change the Session ID that is assigned to user's browser after successfully login and use it as authenticated session id. Then an attacker with pre-authenticated session id would be able to access the authenticated pages.	Vulnerable
40	Cookies attributes not set properly	Each cookie contains some attribute as secure, path, domain etc. with it. If application does not set these attributes in secure manner, an attacker may be able to get access the cookie information.	Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
41	Session Cookies in Clear text	If cookies are not set secure; an attacker may access it whether application using http or https.	Not Vulnerable
42	Path Traversal testing	A Path Traversal attack aims to access files and directories that are stored outside the web root folder. By manipulating variables that reference files with "dot-dot-slash (../)" sequences and its variations, it may be possible to access arbitrary files and directories stored on file system, including configuration and critical system files etc.	Not Vulnerable
43	Buffer overflow Testing	Buffer overflow errors are characterized by the overwriting of memory fragments of the process, which should have never been modified intentionally or unintentionally. Overwriting values of the IP (Instruction Pointer), BP (Base Pointer) and other registers causes exceptions, segmentation faults, and other errors to occur. Usually, these errors end execution of the application in an unexpected way.	Not Vulnerable
44.	Credential Lockout Policy	If application does not lock the wrong credentials account attempts at login page, an attacker can brute force for the valid credentials and if gets succeed, may access authenticated pages.	Not Vulnerable
45	XPath Injection	XPath Injection attacks occur when a web site uses user-supplied information to construct an XPath query for XML data. By sending intentionally malformed information into the web site, an attacker can find out how the XML data is structured, or access data that he may not normally have access to.	Not Vulnerable
46	WSDL authentication testing	If application is not using authentication for Web services, an attacker may get access to exposed methods and may lead to further exploits.	Not Vulnerable
47	Banner Disclosed Sensitive Information	The default banners leak the version information of the application, platform etc. This information may prove vital for launch further attacks.	Vulnerable
48	Directory Indexing Enabled	If the web server was configured improperly, it may be possible to retrieve a directory listing by sending a request for a specific directory.	Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
49	Cryptographic Storage	If web application does not properly protect sensitive data, such as SSNs, and authentication credentials, with appropriate encryption or hashing. Attackers may steal or modify such weakly protected data to conduct identity theft, or other crimes.	Not Vulnerable
50	Other Business Logic Flaws	If application has functional flaws like logout module is not implemented etc., an attacker may use such flaws to lead further attacks.	Vulnerable
51	HTTP Security headers	HTTP security headers are a set of lines that one can add to your website's code. It helps protect it from malicious attacks. They tell the browser what is allowed and what isn't. They can also serve as a warning against possible threats. One can use these headers alone or in conjunction with other security measures such as an SSL certificate for extra protection.	Vulnerable
52	HTTPS Connection	HTTPS, the communication protocol is encrypted using Transport Layer Security (TLS) or, formerly, Secure Sockets Layer (SSL). The protocol is therefore also referred to as HTTP over TLS	Not Vulnerable
53	Content Negotiation Flaws	The Content-Type header is used to indicate the media type of the resource. The media type is a string sent along with the file indicating the format of the file. The HTTP Accept header is a request type header. The Accept header is used to inform the server by the client that which content type is understandable by the client expressed as MIME-types. By using the Content-negotiation the server selects a proposal of the content type and informs the client of its choice with the Content-type response header.	Not Vulnerable
54	Hardcoded Sensitive Information	The software contains a hard-coded password, which it uses for its own inbound authentication or for outbound communication to external components. The use of a hard-coded cryptographic key significantly increases the possibility that encrypted data may be recovered.	Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
55	Sensitive information submitted as URL Parameter	Information exposure through query strings in URL is when sensitive data is passed to parameters in the URL. This allows attackers to obtain sensitive data such as usernames, passwords, tokens (authX), database details, and any other potentially sensitive data. Simply using HTTPS does not resolve this vulnerability.	Vulnerable
56	Server DEBUG Mode Enabled	Debug Mode, debuggers are software tools which enable the programmer to monitor the execution of a program, stop it, restart it, set breakpoints, and change values in memory.	Not Vulnerable
57	Fingerprinting HTTP Headers Enabled	Web server fingerprinting is one of the critical tasks for a penetration tester. By knowing the version and type of a running web server allows testers to determine known vulnerabilities and the appropriate exploits to use during testing.	Not Vulnerable
58	Broken Authentication	Broken authentication is an umbrella term for several vulnerabilities that attackers exploit to impersonate legitimate users online. Broadly, broken authentication refers to weaknesses in two areas: session management and credential management.	Not Vulnerable
59	Broken Authorization	Broken Authorization happens when an application does not correctly confirm that the user performing the request has the required privileges to access a resource of another user.	Vulnerable
60	Excessive Data Exposure	Looking forward to generic implementations, developers tend to expose all object properties without considering their individual sensitivity, relying on clients to perform the data filtering before displaying it to the user.	Vulnerable
61	Lack of Rate Limiting	APIs or applications do not impose any restrictions on the size or number of resources that can be requested by the client/user. Not only can this impact the API server performance, leading to Denial of Service (DoS), but also leaves the door open to authentication flaws such as brute force	Vulnerable
62	Mass Assignment	Binding client provided data (e.g., JSON) to data models, without proper properties filtering based on an allowlist, usually leads to Mass Assignment. Either guessing objects properties, exploring other API endpoints,	Not Vulnerable



Sr. No.	Test cases	Description	Test Result (Vulnerable / Not Vulnerable)
		reading the documentation, or providing additional object properties in request payloads, allows attackers to modify object properties they are not supposed to.	
63	Improper Assets Management	APIs tend to expose more endpoints than traditional web applications, making proper and updated documentation highly important. Proper hosts and deployed API versions inventory also play an important role to mitigate issues such as deprecated API versions and exposed debug endpoints.	Not Vulnerable
64	Insufficient Session or Token Expiration	Insufficient Session Expiration is when a web site permits an attacker to reuse old session credentials or session IDs for authorization	Vulnerable
65	Insecure Direct Object References	Insecure direct object references (IDOR) are a type of access control vulnerability that arises when an application uses user-supplied input to access objects directly.	Not Vulnerable
66	Unvalidated Redirects and Forwards	This issue occurs when a web application redirects or forwards users to untrusted locations based on unvalidated user input, which could be used for phishing attacks.	Not Vulnerable
67	CSRF Token Missing or Incorrect Implementation	Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. Check whether the application uses CSRF tokens to prevent CSRF attacks and if the token implementation is secure.	Not Vulnerable
68	Test for Default Credentials	Applications or services that use default credentials (username and password) left unchanged may be exploited by attackers to gain unauthorized access.	Not Vulnerable
69	Insufficient Logging and Monitoring	This vulnerability occurs when web applications lack proper logging mechanisms, making it difficult to detect and respond to malicious activities or security breaches.	Not Vulnerable
70	Cross-Origin Resource Sharing (CORS)	Improper CORS configurations may allow an attacker to retrieve restricted resources from a web application using unauthorized domains	Not Vulnerable



Annexure B - Tools Used for Assessments

The following tools were used for this Security Assessment:

- Acunetix/ Burp Suite Pro - is a licensed comprehensive vulnerability-scanning program. Its goal is to detect potential vulnerabilities on the tested systems. For example:
- Vulnerabilities that allow a remote cracker to control or access sensitive data on a system
- Default passwords, a few common passwords, and blank/absent passwords on some system accounts. Nessus can also call Hydra (an external tool) to launch a dictionary attack
- Acunetix run on multiple platforms and support all Operating Systems for technical/application vulnerability assessment.
- Burp Suite Professional is an integrated platform for performing security testing of web applications. Its various tools work seamlessly together to support the entire testing process, from initial mapping and analysis of an application's attack surface, through to finding and exploiting security vulnerabilities.

END OF THE REPORT